

Apresentação do componente.

DESENVOLVIMENTO PARA SERVIDORES I (4)²⁴

Objetivos gerais. Esta disciplina fornecerá uma visão geral da linguagem *script* PHP associado a um gerenciador de banco de dados que utilize a linguagem SQL e como usar essas tecnologias para gerar sites dinâmicos. Introduzir práticas de codificação seguras.

Objetivos específicos. Os estudantes deverão ser capazes de desenvolver um CMS simples ou um aplicativo Web completo (lado cliente e lado servidor).

Ementa. PHP histórico e emprego. Instalação e configuração básica do PHP e um IDE. Sintaxe básica do PHP. Usando o PHP como um mecanismo de modelo simples. Panorama das melhores práticas com PHP. Conceitos de programação HTTP. Codificação de caracteres. Localidades, fusos horários e funções de tempo. *Strings*. Uso de Array e funções de matriz. Orientação a objetos em PHP (Classes, objetos, herança, encapsulamento, polimorfismo, agregação, composição e métodos). Tratamento de exceções de erro. Arquitetura do lado do servidor. Manipulação de dados postados. Enviando e-mail. Sessões e autenticação. Cookies. Arquivo manuseio e armazenamento de dados em arquivos de texto. Gerenciador de banco de dados e suas funções. *Frameworks*. Web Services, API, RSS, JSON e Ajax. Hospedagem compartilhada.

Bibliografia básica

BEIGHLEY, L; MORRISON, M. *Use a cabeça! PHP & MySQL*. São Paulo: Alta Books, 2011.

DALL'OGGIO, P. *PHP - programando com orientação a objetos*. São Paulo: Novatec, 2009.

YANK, K. *Build your own database driven web site using PHP*. New York: Oreilly & Assoc, 2012.

Bibliografia complementar

DOYLE, M. *Beginning PHP 5.3*. Indianapolis: Wiley Pub, 2009.

GILMORE, W. J. *Dominando PHP e MYSQL do iniciante ao profissional*. Rio de Janeiro: Starlin Alta Consult, 2008.

1-) Comunicação de alunos com alunos e professores:

- Um e-mail para a sala é de grande valia para divulgação de material, notícias e etc.
- Criação de grupo nas redes sociais também é interessante.

2-) Uso de celulares:

Para o bom andamento das aulas, recomendo que utilizem os celulares em *vibracall*, para não atrapalhar o andamento da aula.

3-) Material das aulas:

A disciplina trabalha com NOTAS DE AULA que são disponibilizadas ao final de cada aula, para isso vamos utilizar o GitHub, nosso repositório de aula será: <https://github.com/marcossousa33dev/FATECSRDSI202502>

4-) Prazos de trabalhos e atividades:

Toda atividade solicitada terá uma data limite de entrega, de forma alguma tal data será postergada, ou seja, se não for entregue até a data limite a mesma receberá nota 0, isso tanto para atividades entregues de forma impressa ou enviadas ao e-mail da disciplina. Qualquer problema que tenham, me procurem com antecedência para verificarmos o que pode ser realizado.

5-) Qualidade do material de atividades:

- Impressas ou manuscritas:
Muita atenção na qualidade do que será entregue, atividades sem grampear, faltando nome e número de componentes, rasgadas, amassadas, com rebarba de folha de caderno e etc. serão desconsiderados por mim.
- Digitais:
Ao enviarem atividades para o e-mail da disciplina, SEMPRE no assunto deverá ter o nome da atividade que está sendo enviada, e no corpo do e-mail deverá ter o(s) nome(s) do(s) integrante(s) da atividade, sem estar desta forma a atividade será DESCONSIDERADA.

6-) Critérios de avaliação:

Cada trimestres teremos as seguintes formas de avaliação:

- Práticas em Laboratório;
- Assiduidade;
- Outras que se fizerem necessário.

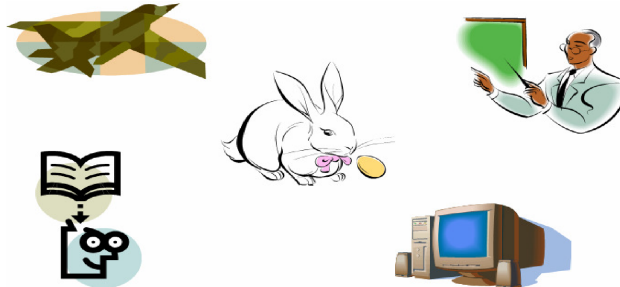
1. Introdução

Iremos relatar nessa fase de nosso curso a orientação à objetos com definições que serão importantes e que podemos implementar em qualquer linguagem de programação que seja OO.

2. Objetos

O Que São Objetos?

De acordo com o dicionário: - Objeto: "1. Tudo que se oferece aos nossos sentidos ou à nossa alma. 2. Coisa material: Havia na estante vários objetos. 3. Tudo que constitui a matéria de ciências ou artes. 4. Assunto, matéria. 5. Fim a que se mira ou que se tem em vista".



A figura destaca uma série de objetos. Objetos podem ser não só coisas concretas como também coisas inanimadas, como por exemplo uma matrícula, as disciplinas de um curso, os horários de aula.

Na programação orientada a objetos, implementa-se um conjunto de classes que definem os objetos presentes no *software*. Cada classe possui um comportamento (definidos pelos métodos) e estados possíveis (valores dos atributos) de seus objetos, assim como o relacionamento com outros objetos.

Há alguns anos, Alan Kay, um dos pais do paradigma da orientação a objetos, formulou a chamada "analogia biológica". Nessa analogia ele imaginou como seria um sistema de software que funcionasse como um ser vivo, no qual cada "célula" interagiria com outras células através do envio de mensagens para realização do objetivo comum.

De uma forma mais geral, Kay pensou em como construir um sistema de software a partir de agentes autônomos que interagem entre si. Ele então estabeleceu os seguintes princípios da orientação a objetos:

- (a) Qualquer coisa é um objeto;
- (b) Objetos realizam tarefas através da requisição de serviços a outros objetos;
- (c) Cada objeto pertence a uma determinada classe. Uma classe agrupa objetos similares;
- (d) A classe é um repositório para comportamento associado ao objeto;
- (e) Classes são organizadas em hierarquias.

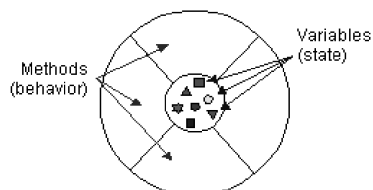
Quando temos um problema e queremos resolvê-lo usando um computador, necessariamente temos que fazer um programa. Este nada mais é do que uma série de instruções que indicam ao computador como proceder em determinadas situações. Assim, o grande desafio do programador é estabelecer a associação entre o modelo que o computador usa e o modelo que o problema lhe apresenta. Isso geralmente leva o programador a modelar o problema apresentado para o modelo utilizado em alguma linguagem. Se a linguagem escolhida for LISP, o problema será traduzido como listas encadeadas. Se for Prolog, o problema será uma cadeia de decisões. Assim, a representação da solução para o problema é característica da linguagem usada, tornando a escrita difícil.

A orientação a objetos tenta solucionar esse problema. A orientação a objetos é geral o suficiente para não impor um modelo de linguagem ao programador, permitindo a ele escolher uma estratégia, representando os aspectos do problema em objetos. Assim, quando se "lê" um programa orientado a objeto, podemos ver não apenas a solução, mas também a descrição do problema em termos do próprio problema. O programador consegue "quebrar" o grande problema em pequenas partes que juntas fornecem a solução.

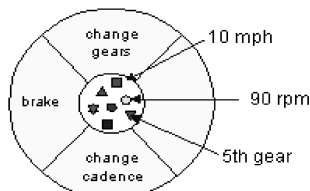
Olhando à sua volta é possível perceber muitos exemplos de objetos do mundo real: seu cachorro, sua mesa, sua televisão, sua bicicleta. Esses objetos têm duas características básicas, que são o **estado** e o **comportamento**. Por exemplo, os cachorros tem nome, cor, raça (estados) e eles balançam o rabo, comem, e latem (comportamento). Uma bicicleta tem 18 marchas, duas rodas, banco (estado) e elas brecam, aceleram e mudam de marcha (comportamento).

De maneira geral, definimos objetos como um conjunto de variáveis e métodos, que representam seus estados e comportamentos.

Veja a ilustração:



Temos aqui representada (de maneira lógica) a ideia que as linguagens orientadas a objeto utilizam. Tudo que um objeto sabe (estados ou variáveis) e tudo que ele podem fazer (comportamento ou métodos) está contido no próprio objeto. No exemplo da bicicleta, poderemos representar o objeto como no exemplo a seguir:



Temos métodos para mudar a marcha, a cadência das pedaladas, e para brecar. A utilização desses métodos altera os valores dos estados. Ao brecar, a velocidade diminui. Ao mudar a marcha, a cadência é alterada. Note que os diagramas mostram que as variáveis do objeto estão no centro do mesmo. Os métodos cercam esses valores e os “escondem” de outros objetos. Deste modo, só é possível ter acesso a essas variáveis através dos métodos. Esse tipo de construção é chamada de encapsulamento, e é a construção ideal para objetos.

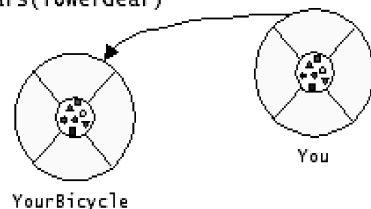
Mas um objeto sozinho geralmente não é muito útil. Ao contrário, em um programa orientado a objetos temos muitos objetos se relacionando entre si. Uma bicicleta encostada na garagem nada mais é que um pedaço de ferro e borracha. Por si mesma, ela não tem capacidade de realizar nenhuma tarefa. Apenas com outro objeto (você) utilizando a bicicleta, ela é capaz de realizar algum trabalho.

A interação entre objetos é feita através de mensagens. Um objeto “chama” os métodos de outro, passando parâmetros quando necessário. Quando você passa a mensagem “mude de marcha” para o objeto bicicleta você precisa dizer qual marcha você deseja.

A figura a seguir mostra os três componentes que fazem parte de uma mensagem:

- Objeto para o qual a mensagem é dirigida (bicicleta)
- Nome do método a ser executado (muda marcha)
- Os parâmetros necessários para a execução do método.

`changeGears(lowerGear)`



A troca de mensagens nos fornece ainda um benefício importante. Como todas as interações são feitas através delas, não importa se o objeto faz parte do mesmo programa; ele pode estar até em outro computador. Existem maneiras de enviar essas mensagens através da rede, permitindo a comunicação remota entre objetos.

Assim, um programa orientado a objetos nada mais é do que um punhado de objetos dizendo um ao outro o que fazer. Quando você quer que um objeto faça alguma coisa, você envia a ele uma "mensagem" informando o que quer fazer, e o objeto faz. Se ele precisar de outro objeto que o auxiliar a realizar o "trabalho", ele mesmo vai cuidar de enviar mensagem para esse outro

objeto. Deste modo, um programa pode realizar atividades muito complexas baseadas apenas em uma mensagem inicial.

Você pode definir vários tipos de objetos diferentes para seu programa. Cada objeto terá suas próprias características, e saberá como interpretar certos pedidos. A parte interessante é que objetos de mesmo tipo se comportam de maneira semelhante. Assim, você pode ter funções genéricas que funcionam para vários tipos diferentes, economizando código.

Vale lembrar que métodos são diferentes de procedimentos e funções das linguagens procedurais, e é muito fácil confundir os conceitos. Para evitar a confusão, temos que entender o que são classes.

3. O que é uma classe?

Assim como os objetos do mundo real, o mundo da Programação Orientada a Objetos (POO) agrupa os objetos pelos comportamentos e atributos comuns.

A biologia classifica todos os cães, gatos, elefantes e seres humanos como mamíferos. Características compartilhadas dão a essas criaturas separadas um senso de comunidade. No mundo do software, as classes agrupam objetos relacionados da mesma maneira.

Uma classe define todas as **características comuns a um tipo de objeto**. Especificamente, a classe **define todos os atributos e comportamentos** expostos pelo objeto. A classe define às quais mensagens o seu objeto responde. Quando um objeto quer exercer o comportamento de outro objeto, ele não faz isso diretamente, mas pede ao outro objeto para que se mude, normalmente baseado em alguma informação adicional. Frequentemente, isso é referido como “envio de mensagem”.

Uma classe define os atributos e comportamentos comuns compartilhados por um tipo de objeto. Imagine a classe como a forma do objeto. Os objetos de certo tipo ou classificação compartilham os mesmos comportamentos e atributos. As classes atuam de forma muito parecida com um cortador de molde ou biscoito, no sentido de que você usa uma classe para criar ou *instanciar* objetos.

Os *atributos* são as características de uma classe visíveis externamente. A cor dos olhos e a cor dos cabelos são exemplos de atributos. Um objeto pode expor um atributo fornecendo um link direto a alguma variável interna ou retornando o valor através de um método.

Comportamento é uma ação executada por um objeto quando passada uma mensagem ou em resposta a uma mudança de estado: é algo que um objeto faz. Um objeto pode executar o *comportamento* de outro, executando uma operação sobre esse objeto. Você pode ver os termos: *chamada de método*, *chamada de função* ou *passar uma mensagem*; usados em vez de executar uma *operação*. O que é importante é que cada uma dessas ações omite o comportamento de um objeto.

A definição de classes e seus inter-relacionamentos é o principal resultado da etapa de projeto de software. Em geral, esse resultado é expresso em termos de alguma linguagem de modelagem, tal como UML.

Uma classe é um gabarito para a definição de objetos. Através da definição de uma classe, descreve-se que propriedades — ou atributos — o objeto terá.

Além da especificação de atributos, a definição de uma classe descreve também qual o comportamento de objetos da classe, ou seja, que funcionalidades podem ser aplicadas a objetos da classe.

Essas funcionalidades são descritas através de métodos. Um método nada mais é que o equivalente a um procedimento ou função, com a restrição que ele manipula apenas suas variáveis locais e os atributos que foram definidos para a classe.

Uma vez que estejam definidas quais serão as classes que irão compor uma aplicação, assim como qual deve ser sua estrutura interna e comportamento, é possível criar essas classes em Java.

Na *Unified Modeling Language* (UML), a representação para uma classe no diagrama de classes é tipicamente expressa na forma gráfica, como mostrado na Figura a seguir.

Como se observa nessa figura, a especificação de uma classe é composta por três regiões: o nome da classe, o conjunto de atributos da classe e o conjunto de métodos da classe.

NomeClasse
visibilidade nomeAtributo : tipo = valor default
visibilidade nomeAtributo : tipo = valor default
visibilidade nomeMétodo(listaArgumentos) : tipoRetorno
...
visibilidade nomeMétodo(listaArgumentos) : tipoRetorno

O nome da classe é um identificador para a classe, que permite referenciá-la posteriormente por exemplo, no momento da criação de um objeto.

O conjunto de atributos descreve as propriedades da classe. Cada atributo é identificado por um nome e tem um tipo associado. Em uma linguagem de programação orientada a objetos pura, o tipo é o nome de uma classe. Na prática, a maior parte das linguagens de programação orientada a objetos oferecem um grupo de tipos primitivos, como inteiro, real e caráter, que podem ser usados na descrição de atributos. O atributo pode ainda ter um valor_default opcional, que especifica um valor inicial para o atributo.

Os métodos definem as funcionalidades da classe, ou seja, o que será possível fazer com objetos dessa classe. Cada método é especificado por uma assinatura, composta por um identificador para o método (o nome do método), o tipo para o valor de retorno e sua lista de argumentos, sendo cada argumento identificado por seu tipo e nome.

Através do mecanismo de sobrecarga (*overloading*), dois métodos de uma classe podem ter o mesmo nome, desde que suas assinaturas sejam diferentes. Tal situação não gera conflito pois o compilador é capaz de detectar qual método deve ser escolhido a partir da análise dos tipos dos argumentos do método. Nesse caso, diz-se que ocorre a ligação prematura (*early binding*) para o método correto.

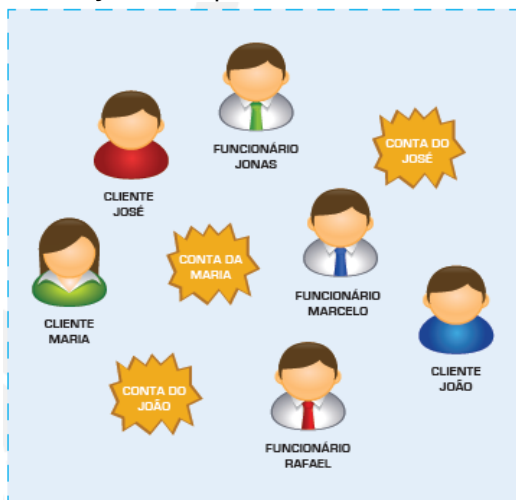
O modificador de visibilidade pode estar presente tanto para atributos como para métodos. Em princípio, três categorias de visibilidade podem ser definidas:

- público, denotado em UML pelo símbolo +: nesse caso, o atributo ou método de um objeto dessa classe pode ser acessado por qualquer outro objeto (visibilidade externa total);
- privativo, denotado em UML pelo símbolo -: nesse caso, o atributo ou método de um objeto dessa classe não pode ser acessado por nenhum outro objeto (nenhuma visibilidade externa);
- protegido, denotado em UML pelo símbolo #: nesse caso, o atributo ou método de um objeto dessa classe poderá ser acessado apenas por objetos de classes que sejam derivadas dessa através do mecanismo de herança.

4. Domínio e Aplicação

Um domínio é composto pelas entidades, informações e processos relacionados a um determinado contexto. Uma aplicação pode ser desenvolvida para automatizar ou tornar factível as tarefas de um domínio. Portanto, uma aplicação é basicamente o “reflexo” de um domínio.

Para exemplificar, suponha que estamos interessados em desenvolver uma aplicação para facilitar as tarefas do cotidiano de um banco. Podemos identificar clientes, funcionários, agências e contas como entidades desse domínio. Assim como podemos identificar as informações e os processos relacionados a essas entidades.



DOMÍNIO BANCÁRIO

Observação:

A identificação dos elementos de um domínio é uma tarefa **difícil**, pois depende fortemente do conhecimento das entidades, informações e processos que o compõem. Em geral, as pessoas que possuem esse conhecimento ou parte dele estão em contato constante com o domínio e não possuem conhecimentos técnicos para desenvolver uma aplicação.

Desenvolvedores de software buscam constantemente mecanismos para tornar mais eficiente o entendimento dos domínios para os quais eles devem desenvolver aplicações.

5. Objetos, Atributos e Métodos

As entidades identificadas no domínio devem ser representadas de alguma forma dentro da aplicação correspondente. Nas aplicações orientadas a objetos, as entidades são representadas por objetos.

- Uma aplicação orientada a objetos é composta por objetos.
- Em geral, um objeto representa uma entidade do domínio.

Para exemplificar, suponha que no domínio de um determinado banco exista um cliente chamado João. Dentro de uma aplicação orientada a objetos correspondente a esse domínio, deve existir um objeto para representar esse cliente.

Suponha que algumas informações do cliente João como nome, data de nascimento e sexo são importantes para o banco. Já que esses dados são relevantes para o domínio, o objeto que representa esse cliente deve possuir essas informações. Esses dados são armazenados nos atributos do objeto que representa o João.

- Um atributo é uma variável que pertence a um objeto.
- Os dados de um objeto são armazenados nos seus atributos.

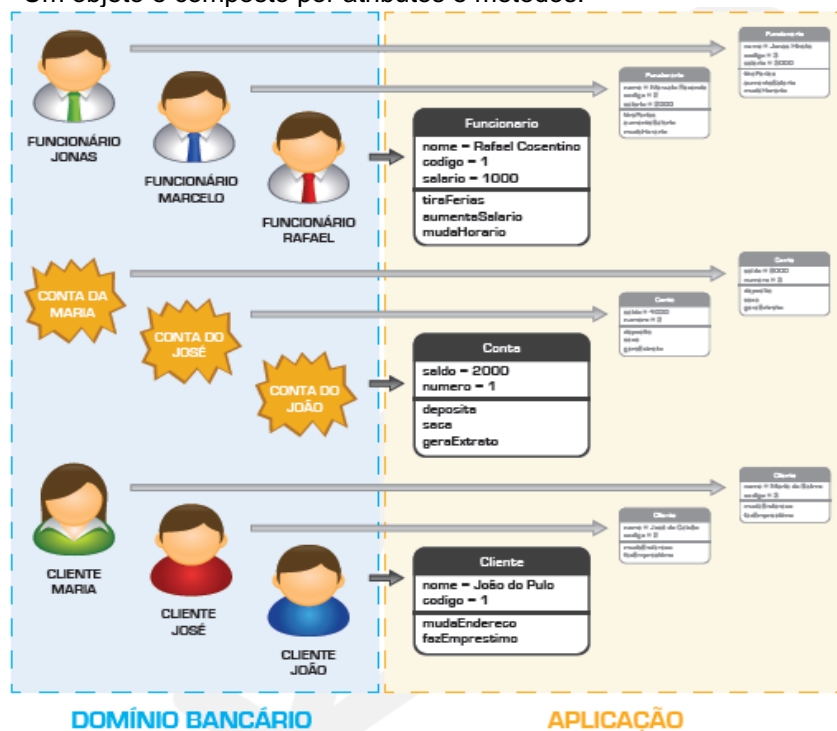
O próprio objeto deve realizar operações de consulta ou alteração dos valores de seus atributos.

Essas operações são definidas nos métodos do objeto.

Os métodos também são utilizados para possibilitar interações entre os objetos de uma aplicação.

Por exemplo, quando um cliente requisita um saque através de um caixa eletrônico do banco, o objeto que representa o caixa eletrônico deve interagir com o objeto que representa a conta do cliente.

- As tarefas que um objeto pode realizar são definidas pelos seus métodos.
- Um objeto é composto por atributos e métodos.



Observações:

- Em geral, não é adequado utilizar o objeto que representa um determinado cliente para representar outro cliente do banco, pois os dados dos clientes podem ser diferentes.

Dessa forma, para cada cliente do banco, deve existir um objeto dentro do sistema para representá-lo.

- Os objetos não representam apenas coisas concretas como os clientes do banco. Eles também devem ser utilizados para representar coisas abstratas como uma conta de um cliente ou um serviço que o banco ofereça.

6. Convenções de nomenclatura: Camel, Pascal, Kebab e Snake case

Se você estuda programação, provavelmente se vê a todo momento nomeando coisas, como variáveis, classes, métodos, funções e muito mais. Mas, afinal, será que existe alguma convenção para ajudar nesse processo de nomeação?

Há diversas maneiras de aplicar boas práticas no seu código, uma delas é a convenção de nomenclatura, que são recomendações para nomear identificadores ajudando a tornar seu código mais limpo e fácil de compreender, mantendo um padrão para o leitor e até mesmo podendo fornecer informações sobre a função do identificador.

Além disso, podemos encontrar outras linguagens, como Go, que dependem muito de você conhecer a diferença entre os casos de uso, pois a convenção utilizada no nome possui um efeito semântico também.

Agora, vamos conhecer os quatro tipos mais comuns de convenções de nomenclatura para combinar palavras em uma única *String*.

6.1. Camel Case

Camel case deve começar com a primeira letra minúscula e a primeira letra de cada nova palavra subsequente maiúscula:



CoisasParaFazer

IdadeDoAmigo

ValorFinal

6.2. Snake case

Em snake case, conhecido também como “*underscore case*”, utilizamos *underline* no lugar do espaço para separar as palavras. Quando o *snake case* está em caixa alta, ele é chamado de “*screaming snake case*”:



coisas-para-fazer

idade-do-amigo

valor-final

PRIMEIRO-NOME

LISTA-INICIAL

6.3. Kebab case

Kebab case utiliza o traço para combinar as palavras. Quando o kebab case está em caixa alta, ele é chamado de “*screaming kebab case*”:

● ● ●

coisas-para-fazer

idade-do-amigo

valor-final

PRIMEIRO-NOME

LISTA-INICIAL

6.4. Convenções Java

O Java possui o *Java Secure Coding Guidelines*, uma documentação disponibilizada pela Oracle, baseada nos padrões de codificação apresentados no *Java Language Specification*. É importante ressaltar que essas convenções de código Java foram escritas em 1999 e não foram atualizadas desde então, portanto algumas informações contidas no documento podem não estar atualizadas. As convenções mais comuns são e que usamos:

- *camelCase* para variáveis, atributos e métodos;
- *PascalCase* para classes;
- *SCREAMING_SNAKE_CASE* para constantes.

● ● ●

```
public class Pessoa{
    public static void main(String[] args) {

        String primeiroNome = "Maria";
        int idade = 22;
        double alturaAtual = 1.65;
        final String MENSAGEM_PADRAO = "Olá";
    }
}
```

7. Semântica e Sintaxe, vamos entender?

Se você respondeu sim! Parabéns!

Na vasta paisagem da programação, duas palavras - semântica e sintaxe - surgem frequentemente, mas nem sempre são entendidas em sua totalidade. A semântica e a sintaxe são como duas faces de uma mesma moeda, ambas essenciais para a compreensão e execução bem-sucedida de um código. No entanto, é fundamental entender a diferença entre elas e como elas se complementam para criar uma experiência de programação poderosa e eficaz.

7.1. O que é Semântica?

A semântica está relacionada ao significado ou interpretação do código. Em outras palavras, trata-se de entender o propósito e a função por trás das instruções escritas em um programa. Uma semântica forte garante que o código não apenas funcione corretamente, mas também faça sentido do ponto de vista lógico.

Por exemplo, ao escrever um código para calcular a média de uma lista de números, a semântica envolveria a compreensão de que estamos tentando encontrar um valor médio e que isso requer a soma de todos os números na lista, dividida pelo total de elementos na lista.

7.2. O que é Sintaxe?

A sintaxe, por outro lado, diz respeito à estrutura e à gramática do código. Ela se concentra em seguir as regras específicas da linguagem de programação para garantir que o código seja escrito de forma correta e compreensível pelo computador. Uma sintaxe correta é fundamental para evitar erros de compilação ou interpretação.

Continuando com o exemplo anterior, a sintaxe seria a forma como escrevemos o código em uma linguagem de programação específica, seguindo suas regras gramaticais. Por exemplo, em Python, a sintaxe para calcular a média de uma lista de números pode envolver o uso de loops e operadores aritméticos de acordo com a estrutura definida pela linguagem.

7.3. O Equilíbrio entre Semântica e Sintaxe

Para escrever um código eficiente e de fácil manutenção, é essencial encontrar o equilíbrio certo entre semântica e sintaxe. Um código com uma semântica forte, mas com uma sintaxe confusa, pode ser difícil de entender e dar manutenção. Da mesma forma, um código com uma sintaxe impecável, mas com uma semântica fraca, pode não atender às necessidades do usuário final.

O verdadeiro poder da programação reside na capacidade de comunicar ideias complexas de forma clara e concisa. Isso requer não apenas um domínio da sintaxe da linguagem de programação escolhida, mas também uma compreensão profunda dos princípios subjacentes e da lógica por trás do problema a ser resolvido.

7.4. Como Melhorar a Semântica e a Sintaxe do seu Código

Compreender o Problema: Antes de começar a escrever código, certifique-se de entender completamente o problema que está tentando resolver. Isso ajudará a orientar tanto a semântica quanto a sintaxe do seu código.

Escolher a Linguagem Adequada: Cada linguagem de programação tem suas próprias nuances de semântica e sintaxe. Escolha a linguagem que melhor se alinha com os requisitos do seu projeto e com a sua própria compreensão.

Praticar Regularmente: A prática leva à perfeição. Dedique tempo para praticar a escrita de código em diferentes cenários para aprimorar tanto a semântica quanto a sintaxe.

Revisar e Refatorar: Sempre revise seu código em busca de possíveis melhorias na semântica e na sintaxe. A refatoração é uma parte essencial do processo de desenvolvimento de software e pode ajudar a tornar seu código mais claro e eficiente.

Em última análise, a semântica e a sintaxe são duas partes igualmente importantes do processo de programação. Ao compreender e aplicar esses conceitos de forma equilibrada, os programadores podem criar soluções poderosas e elegantes para uma ampla gama de problemas.

8. Linguagem Java

Iremos agora abordar nossas aulas uma das maiores linguagens e que demanda um conhecimento específico, por isso que iremos aplicar os conceitos vistos em **Orientação à Objetos** nesta linguagem.

Java é uma linguagem de programação orientada a objeto, multiplataforma que é executada em bilhões de dispositivos em todo o mundo. Ele capacita aplicativos, sistemas operacionais de smartphones, software empresarial e muitos programas conhecidos. Apesar de estar sendo usado há mais de 20 anos, Java é atualmente a linguagem de programação mais popular para desenvolvedores de aplicativos. Java é:

Multiplataforma: Java foi marcado com o slogan "escreva uma vez, execute em qualquer lugar", e isso ainda é válido hoje. O código de programação Java escrito para uma plataforma, como o sistema operacional Windows, pode ser facilmente transferido para outra plataforma, como um sistema operacional de um celular, e vice-versa, sem ser completamente reescrito. O Java funciona em várias plataformas porque quando um programa Java é compilado, o compilador cria um arquivo de código de bytes .class que pode ser executado em qualquer sistema operacional que tenha a Máquina Virtual Java (JVM) instalada nele. Normalmente, é fácil instalar o JVM na maioria dos principais sistemas operacionais, incluindo iOS, o que nem sempre foi o caso.

Orientado a objeto: Java estava entre as primeiras linguagens de programação orientadas a objeto. Uma linguagem de programação orientada a objeto organiza seu código em torno de classes e objetos, em vez de funções e comandos. A maioria das linguagens de programação modernas, incluindo C++, C#, Python e Ruby, é orientada a objeto.

8.1. Quando o Java foi criado?

Java foi inventado por James Gosling em 1995 enquanto ele estava trabalhando na Sun Microsystems. Embora tenha obtido popularidade rapidamente após seu lançamento, o Java não foi iniciado como a linguagem de programação *powerhouse* que é hoje.



James Gosling

O desenvolvimento do que viria a ser o Java começou na Sun Microsystems em 1991. O projeto, inicialmente chamado de Oak, foi originalmente projetado para televisão interativa. Quando o Oak foi considerado avançado demais para a tecnologia de cabo digital disponível na época, Gosling e sua equipe mudaram seu foco para a criação de uma linguagem de programação e renomearam o projeto como Java, em homenagem a um tipo de café da Indonésia. O Gosling viu o Java como uma chance de resolver problemas que ele previu que estavam a caminho de linguagens de programação menos portáteis à medida que mais dispositivos se tornavam conectados em rede.

O Java foi projetado com um estilo de sintaxe semelhante à linguagem de programação C++ para que já fosse familiar para os programadores quando eles comessem a usá-lo. Com o slogan "escreva uma vez, execute em qualquer lugar" em seu núcleo, um programador poderia escrever código Java para uma plataforma que seria executada em qualquer outra plataforma que tivesse um interpretador Java (ou seja, Máquina Virtual Java) instalado. Com o surgimento da Internet e a proliferação de novos dispositivos digitais em meados da década de 1990, o Java foi rapidamente adotado pelos desenvolvedores como uma linguagem de programação verdadeiramente multiplataforma.

A primeira versão pública do Java, Java 1.0, foi lançada em 1996. Em cinco anos, ele tinha 2,5 milhões de desenvolvedores em todo o mundo. Hoje, o Java capacita tudo, desde o sistema operacional móvel Android até software empresarial.

8.2. Para que a linguagem de programação Java é usada?

O Java é uma linguagem de programação extremamente transferível usada entre plataformas e diferentes tipos de dispositivos, de smartphones a TVs inteligentes. É usado para criar aplicativos móveis e Web, software empresarial, dispositivos de Internet das Coisas (IoT), jogos, Big Data, distribuídos e aplicativos baseados em nuvem entre outros tipos. Aqui estão alguns exemplos específicos do mundo real de aplicativos programados com o Java.

8.2.1. Aplicativos móveis

Muitos, se não a maioria dos aplicativos móveis são criados com Java. Java é a linguagem preferida dos desenvolvedores de aplicativos móveis devido à sua plataforma estável e versatilidade. Os aplicativos móveis populares codificados em Java incluem Spotify, Signal e Cash App.

8.2.2. Aplicativos Web

Uma ampla variedade de aplicativos Web é desenvolvida usando Java. O Twitter e o LinkedIn estão entre os mais conhecidos.

8.2.3. Software empresarial

O software empresarial destina-se a atender a um grande grupo ou organização. Inclui softwares como sistemas de cobrança e programas de gerenciamento da cadeia de fornecedores. A alta escalabilidade do Java o torna uma linguagem atraente para desenvolvedores que escrevem software empresarial.

8.2.4. Jogos

Os jogos populares escritos em linguagem de programação Java incluem o Minecraft original e o *RuneScape*.

8.2.5. Aplicativos de IoT

Os aplicativos de IoT estão em todos os lugares — TVs inteligentes, carros, máquinas pesadas, instalações de trabalho e muito mais — e o Java é usado para programar muitos deles. O Java é uma opção popular para desenvolvedores de IoT devido à facilidade com que seu código pode ser transferido entre plataformas.

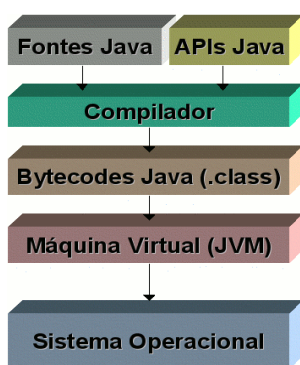
8.3. Como funciona o Java?

Conforme explicado anteriormente, o Java é uma linguagem de programação multiplataforma. Isso significa que ele pode ser escrito para um sistema operacional e executado em outro. Como isso é possível?

O código Java é escrito primeiro em um Kit de Desenvolvimento Java, que está disponível para Windows, Linux e macOS. Os programadores escrevem na linguagem de programação Java, que o kit traduz em código de computador que pode ser lido por qualquer dispositivo com o software certo. Isso é feito com um software chamado compilador. Um compilador usa um código de computador de alto nível como Java e o converte em uma linguagem que os sistemas operacionais entendem chamado código de bytes.

O código de bytes é então processado por um interpretador chamado Máquina Virtual Java (JVM). As JVMs estão disponíveis para a maioria das plataformas de software e hardware, e isso é o que permite que o código Java seja transferido de um dispositivo para outro. Para executar o Java, os JVMs carregam o código, verificam-no e fornecem um ambiente de *runtime*.

Dada a alta portabilidade do Java, não é de admirar que muitas pessoas queiram aprender como escrevê-lo. Felizmente, há muitos recursos disponíveis para começar a aprender Java.



Funcionamento do Java

9. Classes em Java

O conceito de classe apresentado anteriormente é genérico e pode ser aplicado em diversas linguagens de programação. Mostraremos como a classe *Conta* poderia ser escrita utilizando a linguagem Java. Inicialmente, discutiremos apenas sobre os atributos. Os métodos serão abordados posteriormente.

A classe Java *Conta* é declarada utilizando a palavra reservada **class**. No corpo dessa classe, são declaradas três variáveis que são os atributos que os objetos possuirão. Como a linguagem Java é estaticamente tipada, os tipos dos atributos são definidos no código. Os atributos *saldo* e *limite* são do tipo *double*, que permite armazenar números com casas decimais, e o atributo *numero* é do tipo *int*, que permite armazenar números inteiros.

Por convenção, os nomes das classes na linguagem Java devem seguir o padrão “Pascal Case” e “Camel Case” como vimos:

```
Conta.java x
Source History
1 package com.mycompany.exemplosparaaulas;
2
3 public class Conta {
4     double saldo;
5     double limite;
6     int numero;
7 }
```

10. Criando objetos em Java

Após definir a classe `Conta`, podemos criar objetos a partir dela. Esses objetos devem ser alocados na memória RAM do computador. Felizmente, todo o processo de alocação do objeto na memória é gerenciado pela máquina virtual. O gerenciamento da memória é um dos recursos mais importantes oferecidos pela máquina virtual.

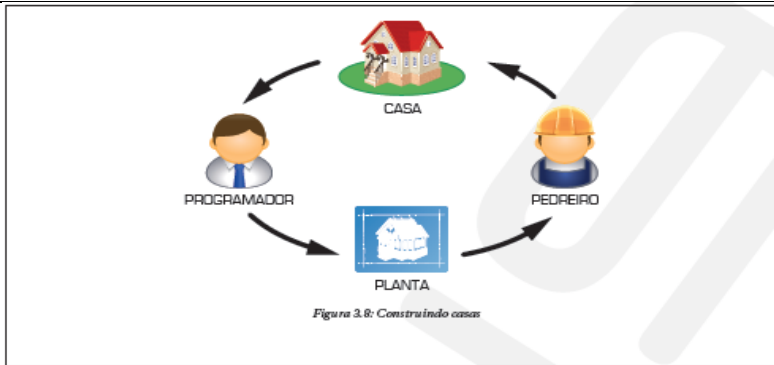
Do ponto de vista da aplicação, basta utilizar um comando especial para criar objetos e a máquina virtual se encarrega do resto. O comando para criar objetos é o **new**.

```
Conta.java x TestaConta.java x
Source History
1 package com.mycompany.exemplosparaaulas;
2
3 public class TestaConta {
4     public static void main(String[] args) {
5         // TODO code application logic here
6         new Conta();
7     }
8 }
```

A linha com o comando `new` poderia ser repetida cada vez que desejássemos criar (instanciar) um objeto da classe `Conta`. A classe `TestaConta` serve apenas para colocarmos o método `main`, que é o ponto de partida da aplicação.

```
Conta.java x TestaConta.java x
Source History
1 package com.mycompany.exemplosparaaulas;
2
3 public class TestaConta {
4     public static void main(String[] args) {
5         // TODO code application logic here
6         new Conta();
7         new Conta();
8         new Conta();
9     }
10 }
```

Chamar o comando **new** passando uma classe Java é como se estivéssemos contratando uma construtora passando a planta da casa que queremos construir. A construtora se encarrega de construir a casa para nós de acordo com a planta. Assim como a máquina virtual se encarrega de construir o objeto na memória do computador.



11. Referências

Todo objeto possui uma referência. A referência de um objeto é a única maneira de acessar os seus atributos e métodos. Dessa forma, devemos guardar as referências dos objetos que desejamos utilizar.

A princípio, podemos comparar a referência de um objeto com o endereço de memória desse objeto. De fato, essa comparação simplifica o aprendizado. Contudo, o conceito de referência é mais amplo. Uma referência é o elemento que permite que um determinado objeto seja acessado. Uma referência está para um objeto assim como um controle remoto está para um aparelho de TV. Através do controle remoto de uma TV você pode aumentar o volume ou trocar de canal. Analogamente, podemos controlar um objeto através da referência do mesmo.



12. Referências em Java

Ao utilizar o comando `new`, um objeto é alocado em algum lugar da memória. Para que possamos acessar esse objeto, precisamos de sua referência. O comando `new` devolve a referência do objeto que foi criado.

Para guardar as referências devolvidas pelo comando `new`, devemos utilizar variáveis não primitivas.

```

1 package com.mycompany.exemplosparaaulas;
2
3 public class TestaConta {
4     public static void main(String[] args) {
5         // TODO code application logic here
6         Conta referencia = new Conta();
7     }
8 }
    
```

No código Java acima, a variável **referencia** receberá a referência do objeto criado pelo comando `new`. Essa variável é do tipo `Conta`. Isso significa que ela só pode armazenar referências de objetos do tipo `Conta`.

13. Manipulando Atributos

Podemos alterar ou acessar os valores guardados nos atributos de um objeto se tivermos a referência a esse objeto. Os atributos são acessados pelo nome. No caso específico da linguagem Java, a sintaxe para acessar um atributo utiliza o operador `"."`.

```

1 package com.mycompany.exemplosparaaulas;
2
3 public class TestaConta {
4     public static void main(String[] args) {
5         //Instância do objeto conta em variáveis não
6         //primitivas
7         Conta referencia = new Conta();
8
9         //Adicionar os valores aos atributos
10        referencia.saldo = 1000;
11        referencia.limite = 500;
12        referencia.numero = 1;
13
14        //Mostrar no terminal os valores
15        System.out.println(referencia.saldo);
16        System.out.println(referencia.limite);
17        System.out.println(referencia.numero);
18    }
19 }

```

Output - Run (TestaConta) x

```

Recompiling the module because of changed source code.
Compiling 3 source files with javac [debug release 22] to target\classes
--- EXERCÍCIO 1.01: Exec (default-CL) de ExemplosParaAulas ---
1000.0
500.0
1
BUILD SUCCESS

```

No código acima, o atributo saldo recebe o valor 1000. O atributo limite recebe o valor 500 e o numero recebe o valor 1. Depois, os valores são impressos na tela através da função **System.out.println()**.

14. Valores Padrão

Poderíamos instanciar um objeto e utilizar seus atributos sem inicializá-los explicitamente, pois os atributos são inicializados com valores padrão. Os atributos de tipos numéricos são inicializados com 0, os atributos do tipo *boolean* são inicializados com false e os demais atributos com *null* (referência vazia).

```

1 package com.mycompany.exemplosparaaulas;
2
3 public class Conta {
4     double saldo;
5     double limite;
6     int numero;
7 }

```

```

1 package com.mycompany.exemplosparaaulas;
2
3 public class TestaConta {
4     public static void main(String[] args) {
5         //Instância do objeto conta em variáveis não
6         //primitivas
7         Conta referencia = new Conta();
8
9         //Mostrar no terminal os valores
10        System.out.println(referencia.saldo);
11        System.out.println(referencia.limite);
12        System.out.println(referencia.numero);
13    }
14 }
  
```

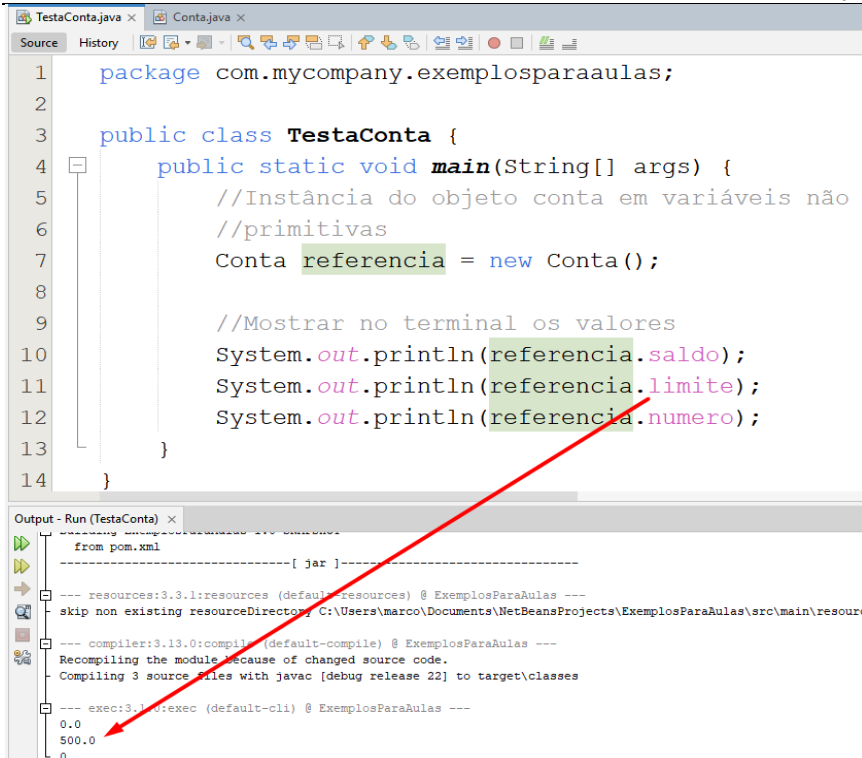
```

Output - Run (TestaConta) x
-----[ jar ]-----
--- resources:3.3.1:resources (default-resources) @ ExemplosParaAulas ---
skip non existing resourceDirectory C:\Users\marco\Documents\NetBeansProjects\ExemplosParaAulas\src\main\resou
--- compiler:3.13.0:compile (default-compile) @ ExemplosParaAulas ---
Recompiling the module because of changed source code.
Compiling 3 source files with javac [debug release 22] to target\classes
--- exec:3.1.0:exec (default-cli) @ ExemplosParaAulas ---
0.0
0.0
0
  
```

A inicialização dos atributos com os valores padrão ocorre na instanciação, ou seja, quando o comando `new` é utilizado. Dessa forma, todo objeto “nasce” com os valores padrão. Em alguns casos, é necessário trocar esses valores. Para trocar o valor padrão de um atributo, devemos inicializá-lo na declaração. Por exemplo, suponha que o limite padrão das contas de um banco seja R\$ 500. Nesse caso, seria interessante definir esse valor como padrão para o atributo `limite`.

```

1 package com.mycompany.exemplosparaaulas;
2
3 public class Conta {
4     double saldo;
5     double limite = 500;
6     int numero;
7 }
  
```

```
1 package com.mycompany.exemplosparaaulas;
2
3 public class TestaConta {
4     public static void main(String[] args) {
5         //Instância do objeto conta em variáveis não
6         //primitivas
7         Conta referencia = new Conta();
8
9         //Mostrar no terminal os valores
10        System.out.println(referencia.saldo);
11        System.out.println(referencia.limite);
12        System.out.println(referencia.numero);
13    }
14 }
```

Output - Run (TestaConta) x

```
from pom.xml
----- [ jar ] -----
--- resources:3.3.1:resources (default-resources) @ ExemplosParaAulas ---
skip non existing resourceDirectory C:\Users\marco\Documents\NetBeansProjects\ExemplosParaAulas\src\main\resourc
--- compiler:3.13.0:compile (default-compile) @ ExemplosParaAulas ---
Recompiling the module because of changed source code.
Compiling 3 source files with javac [debug release 22] to target\classes
--- exec:3.13.0:exec (default-cli) @ ExemplosParaAulas ---
0.0
500.0
0
```

15. Algumas categorias de projetos que podemos trabalhar em Java

Podemos categorizar nossos projetos em Java e podemos falar sobre Java com Maven, Java com Gradle e Java com Ant, como algumas categorias que aparecem quando criamos um projeto novo no Apache Netbeans por exemplo, nestes casos, estamos tratando de ferramentas de *build automation* (automação de compilação) que são usadas para compilar, empacotar, testar, e implantar aplicativos Java de forma eficiente. Cada uma dessas ferramentas tem uma abordagem distinta, mas todas têm o objetivo de facilitar o processo de desenvolvimento e tornar a construção de projetos mais ágil e menos propensa a erros, para nossas aulas e práticas, a ideia é utilizarmos a Maven ou Ant. Vamos ver cada uma delas em detalhes:

15.1. Java com Maven

Maven é uma das ferramentas de build mais populares no ecossistema Java. Ele usa um modelo baseado em XML para descrever o ciclo de vida de construção do projeto.

15.1.1. Principais Características:

- **Dependências:** Maven facilita o gerenciamento de dependências. Ele se conecta a repositórios (como o Maven Central) e baixa automaticamente as bibliotecas necessárias para o seu projeto. Isso significa que você não precisa gerenciar manualmente os arquivos JAR e suas versões.
- **Arquitetura baseada em Plugins:** O Maven executa tarefas por meio de plugins (como compilação, testes e empacotamento) que são configurados no arquivo pom.xml (*Project Object Model*).
- **Ciclo de Vida:** O Maven segue um ciclo de vida definido, que inclui fases como *compile*, *test*, *package*, *install*, *deploy*. Cada uma dessas fases pode ser configurada para realizar tarefas específicas.

15.1.2. Vantagens do Maven:

- Gerenciamento de dependências automático.
- Repositórios centralizados (Maven Central).
- Integração fácil com servidores de build e CI/CD.
- Documentação e convenção forte (se você seguir as convenções do Maven, o projeto é fácil de entender).

15.1.3. Desvantagens:

- Complexidade em grandes projetos.
- XML pode ser verboso.

15.2. Java com Gradle

Gradle é uma ferramenta de automação de build moderna que tem ganhado popularidade nos últimos anos, principalmente por ser mais flexível e rápida que o Maven.

15.2.1. Principais Características:

- **Baseado em Groovy ou Kotlin:** Gradle usa uma linguagem de script (Groovy ou Kotlin DSL) para descrever o build, o que torna a configuração mais flexível e programática.
- **Desempenho:** Gradle é conhecido por ser mais rápido que o Maven e o Ant devido ao seu build incremental (onde ele apenas recompila partes do código que foram alteradas).
- **Gerenciamento de Dependências:** Assim como o Maven, Gradle também oferece fácil integração com repositórios (como Maven Central), mas com maior flexibilidade.
- **Configuração:** Gradle permite configurações mais complexas e personalizadas, sendo ideal para projetos mais complexos.

15.2.2. Vantagens do Gradle:

- Desempenho superior (builds incrementais e paralelismo).
- Flexibilidade: Configurações personalizadas e lógica programática.
- Suporte a múltiplas linguagens: Além de Java, Gradle pode ser usado com outros tipos de projeto (Android, Groovy, Scala, etc.).
- DSL mais conciso: Arquivos de configuração mais simples e expressivos que o Maven (usando Groovy ou Kotlin).

15.2.3. Desvantagens:

- Curva de aprendizado: A flexibilidade de Gradle pode tornar o processo de configuração um pouco mais complexo para iniciantes.
- Menos documentação tradicional (se comparado ao Maven, por exemplo).

15.3. Java com Ant

Ant é uma das ferramentas de build mais antigas para Java. Ao contrário do Maven e do Gradle, o Ant não segue uma convenção padrão para construir o projeto e não tem uma definição de ciclo de vida, o que significa que você tem que definir manualmente como o projeto deve ser construído.

15.3.1. Principais Características:

- **Arquivo build.xml:** O Ant usa um arquivo XML chamado build.xml para definir as tarefas de construção, como compilar código, empacotar JARs, executar testes e mais.
- **Sem Convenção:** Diferente do Maven, o Ant não segue convenções. O desenvolvedor tem total liberdade para definir como o build deve ocorrer.
- **Extensível:** Ant permite adicionar novos tipos de tarefas personalizadas com facilidade.

15.3.2. Vantagens do Ant:

- Extremamente flexível: Você tem controle total sobre o processo de build.
- Pode ser usado em projetos não-Java: Ant pode ser usado para outras linguagens também.
- Facilidade em adicionar tarefas personalizadas.

15.3.3. Desvantagens:

- Não tem ciclo de vida predefinido: O desenvolvedor deve definir manualmente como o processo de construção será.

- Mais difícil de gerenciar dependências: Ao contrário do Maven e Gradle, você precisa gerenciar dependências manualmente ou com plugins adicionais.
- Configuração verbosa: Arquivos build.xml podem ser grandes e difíceis de manter à medida que o projeto cresce.

15.4. Comparação resumida

Característica	Maven	Gradle	Ant
Configuração	Baseado em XML (pom.xml)	Baseado em Groovy ou Kotlin (build.gradle)	Baseado em XML (build.xml)
Gerenciamento de dependências	Sim, integrado com repositórios Maven	Sim, mas mais flexível que o Maven	Necessário configuração manual
Desempenho	Médio (builds não incrementais)	Rápido (builds incrementais e paralelismo)	Médio, depende de como é configurado
Facilidade de uso	Fácil para iniciantes, com convenções fortes	Maior curva de aprendizado, mas flexível	Flexível, mas sem ciclo de vida predefinido
Popularidade	Muito popular, especialmente em grandes empresas	Crescendo rapidamente, especialmente para Android	Menos usado atualmente, mas ainda popular em projetos antigos

Tabela de comparação entre categorias

16. Definição e Instalação do JDK e NetBeans

16.1. O que é o JDK (Java Development Kit)?

O JDK (Java Development Kit) é o kit de desenvolvimento oficial para a linguagem de programação Java, fornecido pela Oracle Corporation e outras distribuidoras como OpenJDK e Amazon Corretto. Ele contém todas as ferramentas necessárias para escrever, compilar e executar programas em Java.

16.1.1. Componentes do JDK

O JDK é composto por vários elementos essenciais para o desenvolvimento Java. Abaixo estão os principais:

1. JRE (Java Runtime Environment)

O JRE é um subconjunto do JDK responsável apenas pela execução de programas Java. Ele inclui:

- Java Virtual Machine (JVM) → Executa bytecode Java.
- Bibliotecas Padrão (API Java) → Conjunto de classes e métodos prontos para uso.
- Ferramentas de suporte → Como carregamento de classes e gerenciamento de memória.

Importante: Se você apenas deseja executar programas Java (e não desenvolver), basta instalar o JRE em vez do JDK.

2. JVM (Java Virtual Machine)

A JVM é o ambiente que permite que programas Java sejam executados em diferentes sistemas operacionais. Ela funciona como um interpretador do bytecode (código intermediário gerado pelo compilador Java), tornando o Java uma linguagem portátil e independente de plataforma.

Princípio do Java: “Write Once, Run Anywhere” (Escreva uma vez, execute em qualquer lugar).

3. Ferramentas do JDK

O JDK também inclui um conjunto de ferramentas para facilitar o desenvolvimento:

Ferramenta	Função
javac	Compilador Java (transforma código-fonte <code>.java</code> em bytecode <code>.class</code>)
java	Interpretador que executa programas Java usando a JVM
jar	Empacotador de arquivos <code>.class</code> em um único <code>.jar</code>

16.2. Baixar e Instalar o JDK

Passo 1: Baixar o JDK

- Acesse o site oficial da Oracle: <https://www.oracle.com/java/technologies/javase-downloads.html>
- Escolha a versão mais recente do JDK.
- Na seção Windows, clique em Installer (.exe) para baixar o arquivo de instalação.

Passo 2: Instalar o JDK

- Execute o arquivo .exe baixado.
- Clique em Next para iniciar a instalação.
- Escolha um diretório de instalação (ou deixe o padrão).
- Clique em Next e aguarde a instalação.
- Ao finalizar, clique em Close.

Passo 3: Testar a instalação do Java

- Abra o Prompt de Comando (Win + R → digite cmd e pressione Enter).
- Digite o comando: `java -version`
- Se aparecer a versão do Java, significa que a instalação foi bem-sucedida.

16.3. Baixar e Instalar o NetBeans

Passo 1: Baixar o NetBeans

- Acesse o site oficial: <https://netbeans.apache.org/download/index.html>
- Escolha a versão mais recente e clique em **Download**.
- Baixe o instalador para Windows (**EXE Installer**).

Passo 2: Instalar o NetBeans

- Execute o arquivo .exe baixado.
- Clique em **Next**.
- Aceite os termos da licença e clique em **Next**.
- Escolha o diretório de instalação (ou deixe o padrão).
- Certifique-se de que o caminho do JDK foi detectado corretamente.
- Clique em **Install** e aguarde a instalação.
- Após a conclusão, clique em **Finish**.

Passo 3: Testar o NetBeans

- Abra o NetBeans pelo menu Iniciar.
- Clique em **File** → **New Project**.
- Escolha **Java with Maven** → **Java Application** e clique em **Next**.
- Dê um nome ao projeto e clique em **Finish**.
- No editor, escreva um código de teste:

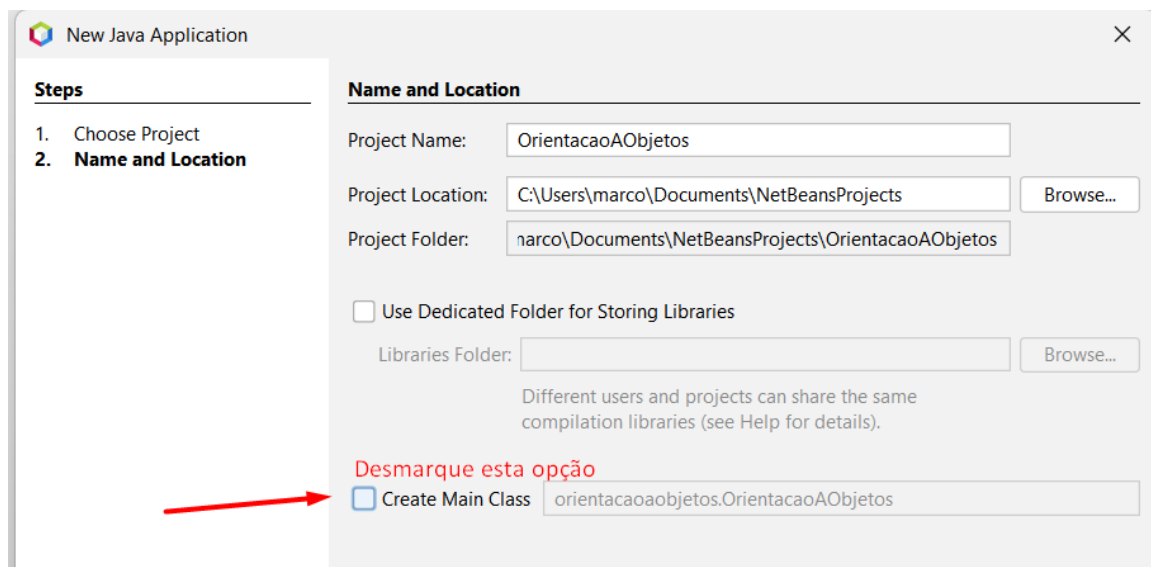
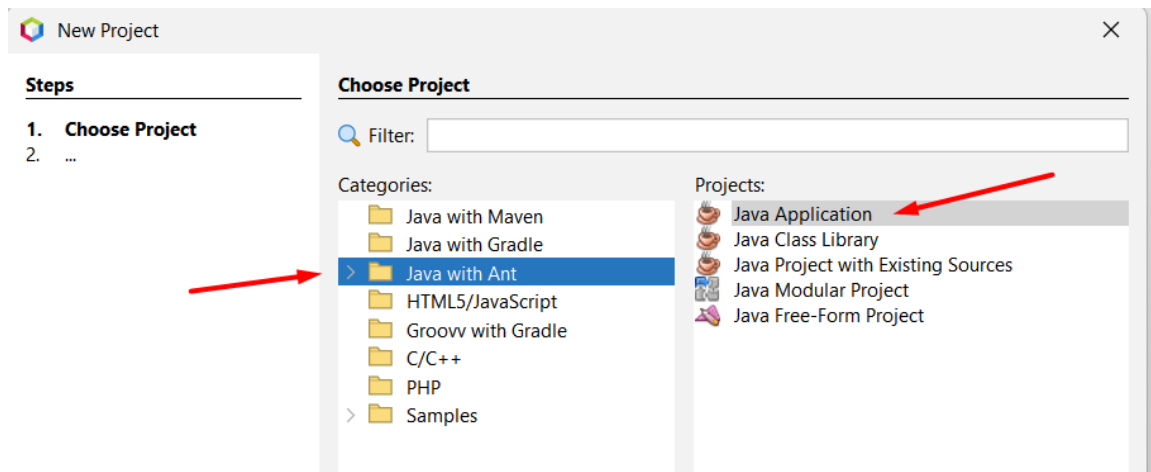
```
public class Main {
    public static void main(String[] args) {
        System.out.println("Olá, NetBeans!");
    }
}
```

- Pressione **F6** para executar. Se aparecer "Olá, NetBeans!" no console, está tudo certo!

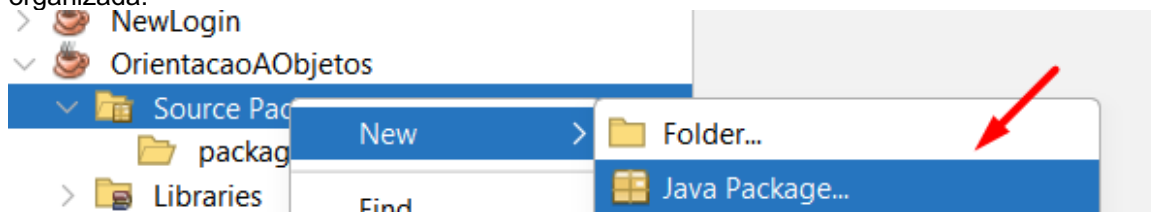
17. Exercícios de Fixação

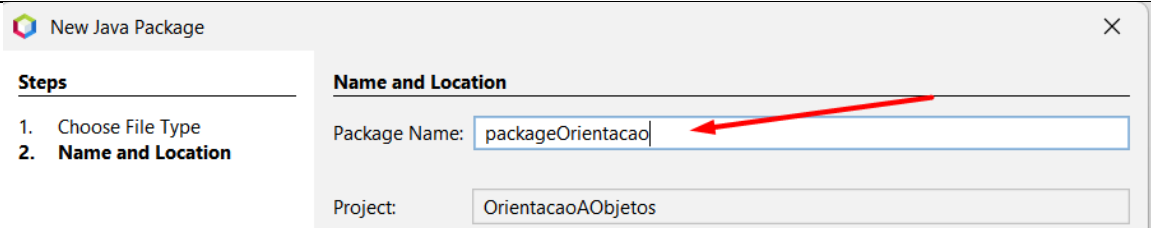
A seguir, iremos realizar este exercício para fixação de conteúdo, portanto, vocês devem codificar o passo a passo a seguir e verificar os resultados obtidos, como sempre falo, vocês podem alterar ou melhorar o código, isso é importante para que consigam ratificar o conteúdo visto até o momento, como informei anteriormente vocês podem utilizar o NetBeans ou a ferramenta que acharem melhor, porém, nas aulas aqui, abordarei o NetBeans.

1 – Para isso crie um novo projeto chamado **OrientacaoAObjetos** para os arquivos desenvolvidos neste exercício.

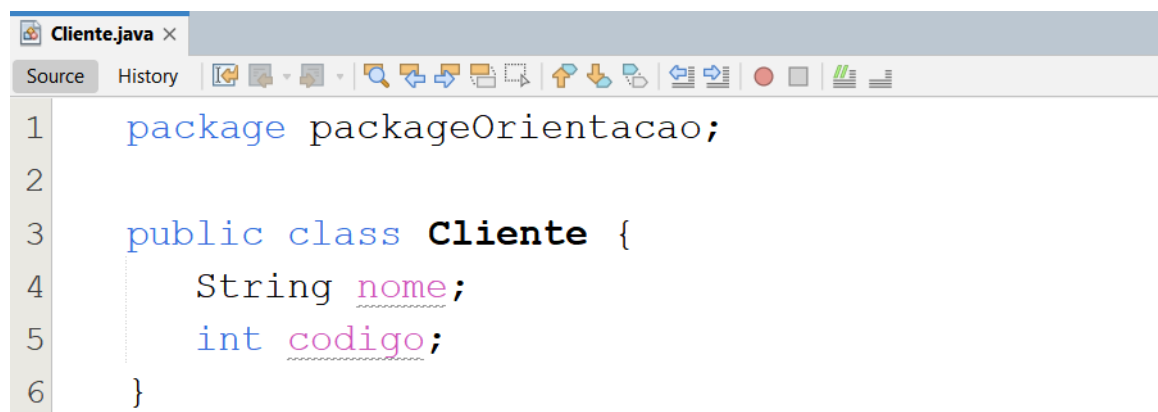
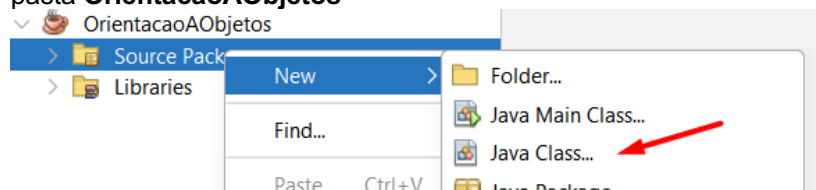


2 – Vamos criar um pacote para armazenar as classes de nosso projeto de forma organizada.

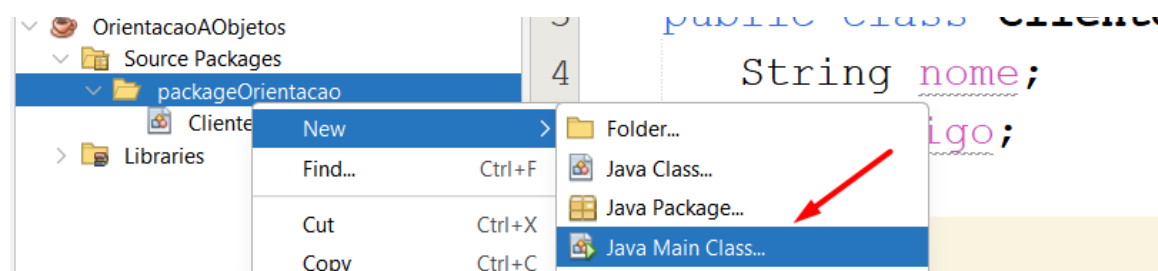




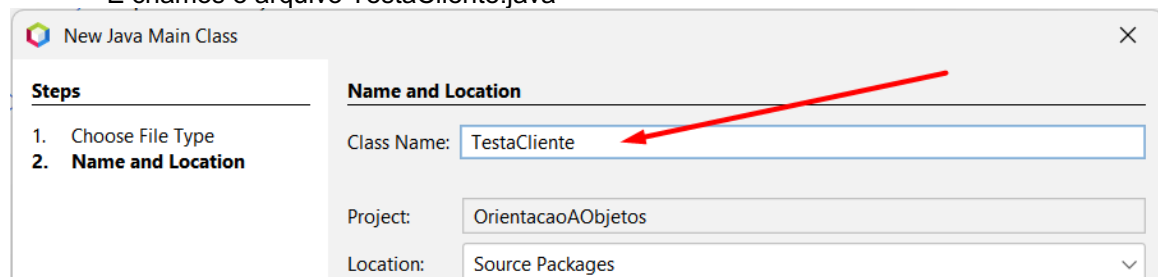
3 - Implemente uma classe para definir os objetos que representarão os clientes de um banco. Mas antes disso, vamos implementar o pacote da classe. Essa classe deve declarar dois atributos: um para os nomes e outro para os códigos dos clientes. Adicione o seguinte arquivo na pasta **OrientacaoAObjetos**

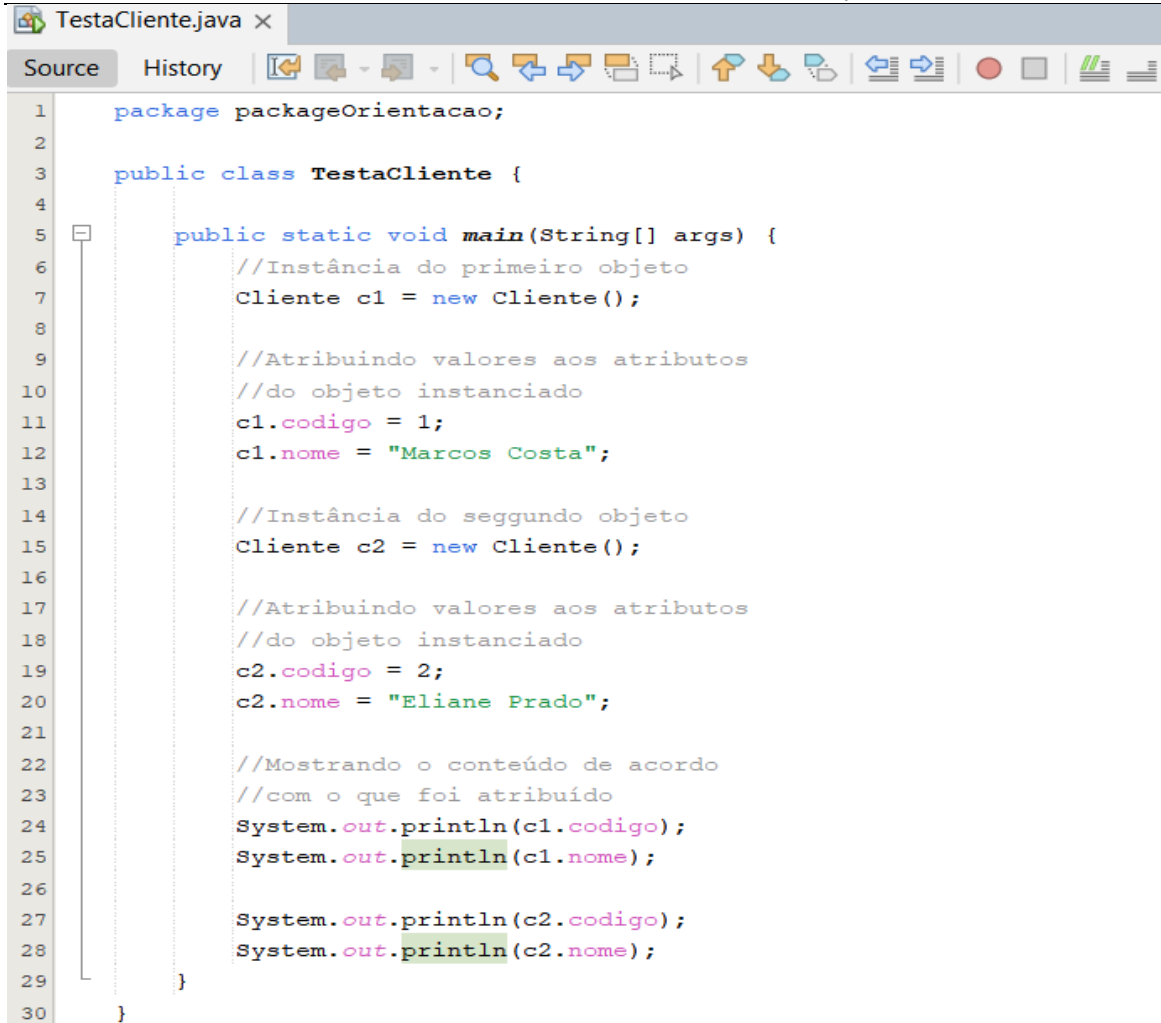


4 - Faça um teste criando dois objetos da classe Cliente. Vamos adicionar agora uma Java Main Class, adicione o seguinte arquivo na pasta **OrientacaoAObjetos**.



E criamos o arquivo TestaCliente.java



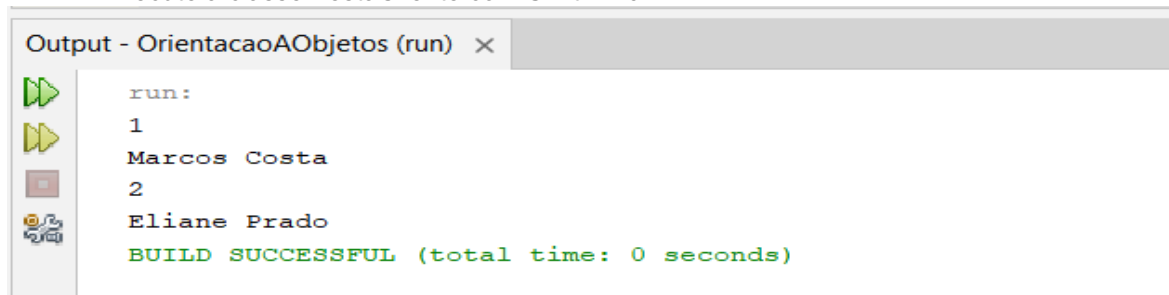


```

1  package packageOrientacao;
2
3  public class TestaCliente {
4
5      public static void main(String[] args) {
6          //Instância do primeiro objeto
7          Cliente c1 = new Cliente();
8
9          //Atribuindo valores aos atributos
10         //do objeto instanciado
11         c1.codigo = 1;
12         c1.nome = "Marcos Costa";
13
14         //Instância do segundo objeto
15         Cliente c2 = new Cliente();
16
17         //Atribuindo valores aos atributos
18         //do objeto instanciado
19         c2.codigo = 2;
20         c2.nome = "Eliane Prado";
21
22         //Mostrando o conteúdo de acordo
23         //com o que foi atribuído
24         System.out.println(c1.codigo);
25         System.out.println(c1.nome);
26
27         System.out.println(c2.codigo);
28         System.out.println(c2.nome);
29     }
30 }

```

Execute a classe TestaCliente com Shift + F6.

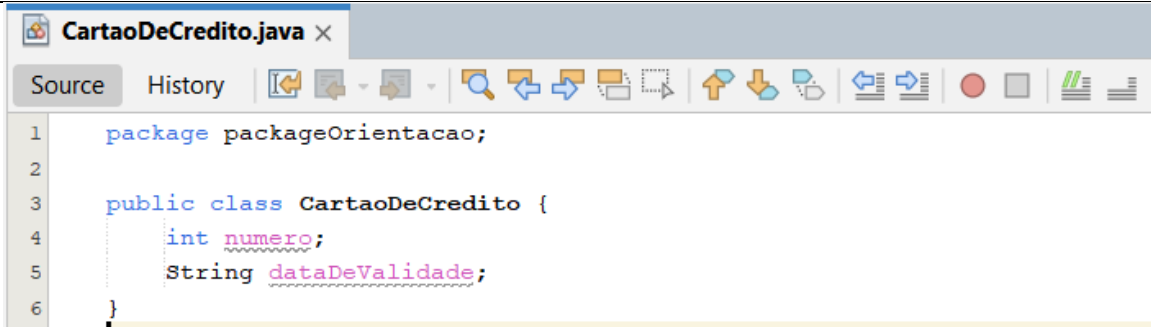


```

Output - OrientacaoAObjetos (run) x
run:
1
Marcos Costa
2
Eliane Prado
BUILD SUCCESSFUL (total time: 0 seconds)

```

5 - Os bancos oferecem aos clientes a possibilidade de obter um cartão de crédito que pode ser utilizados para fazer compras. Um cartão de crédito possui um número e uma data de validade. Crie uma classe (Java Class) para modelar os objetos que representarão os cartões de crédito. Adicione o seguinte arquivo na pasta **OrientacaoAObjetos**.

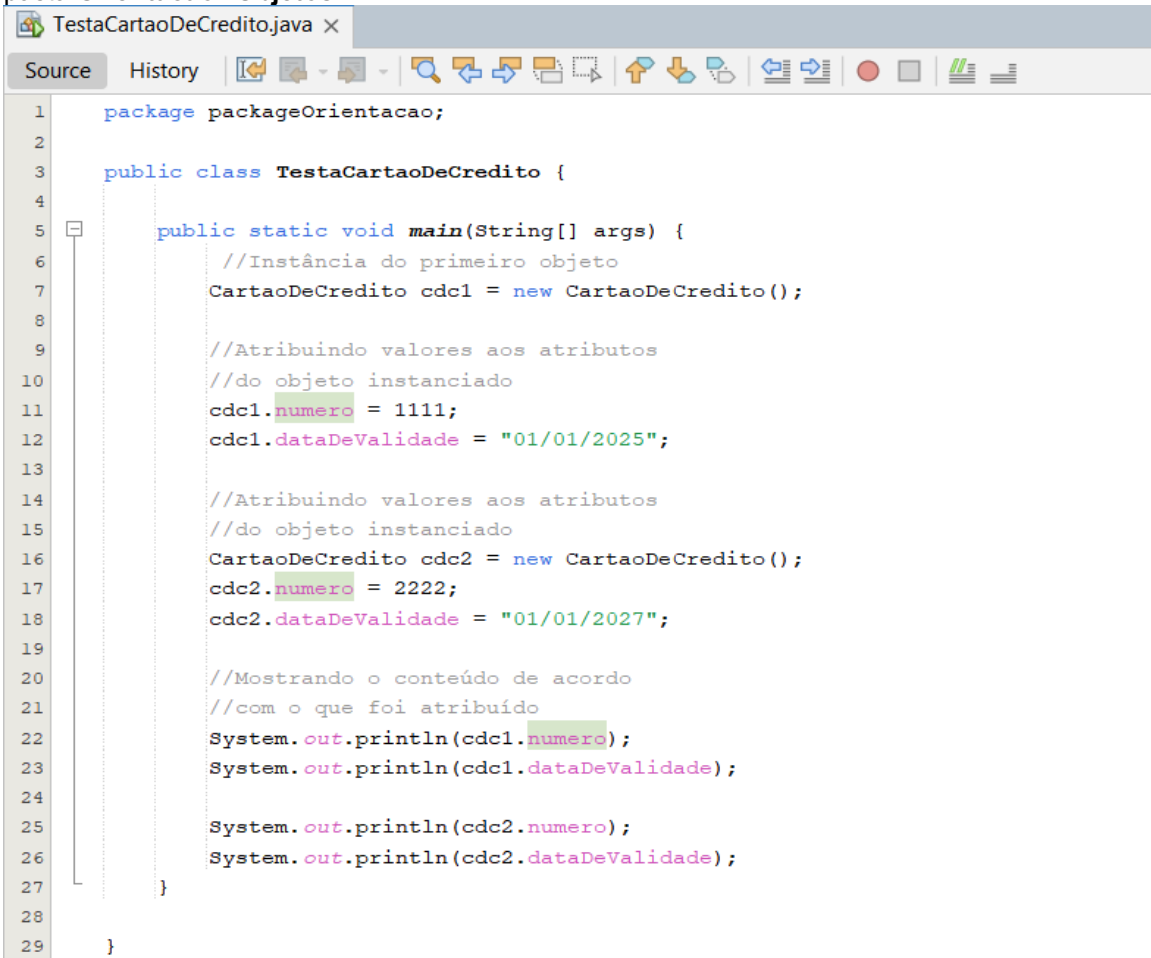


```

1 package packageOrientacao;
2
3 public class CartaoDeCredito {
4     int numero;
5     String dataDeValidade;
6 }

```

6 - Faça um teste criando dois objetos da classe CartaoDeCredito. Altere e imprima os atributos desses objetos. Adicione o seguinte arquivo TestaCartaoDeCredito (Java Main Class) na pasta **OrientacaoAObjetos**.

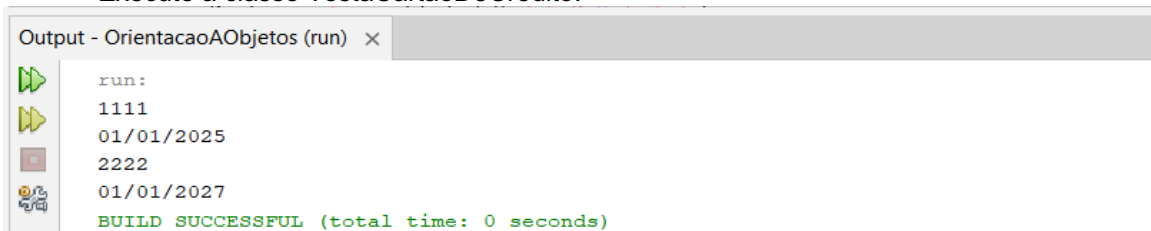


```

1 package packageOrientacao;
2
3 public class TestaCartaoDeCredito {
4
5     public static void main(String[] args) {
6         //Instância do primeiro objeto
7         CartaoDeCredito cdc1 = new CartaoDeCredito();
8
9         //Atribuindo valores aos atributos
10        //do objeto instanciado
11        cdc1.numero = 1111;
12        cdc1.dataDeValidade = "01/01/2025";
13
14        //Atribuindo valores aos atributos
15        //do objeto instanciado
16        CartaoDeCredito cdc2 = new CartaoDeCredito();
17        cdc2.numero = 2222;
18        cdc2.dataDeValidade = "01/01/2027";
19
20        //Mostrando o conteúdo de acordo
21        //com o que foi atribuído
22        System.out.println(cdc1.numero);
23        System.out.println(cdc1.dataDeValidade);
24
25        System.out.println(cdc2.numero);
26        System.out.println(cdc2.dataDeValidade);
27    }
28
29 }

```

Execute a classe TestaCartaoDeCredito.

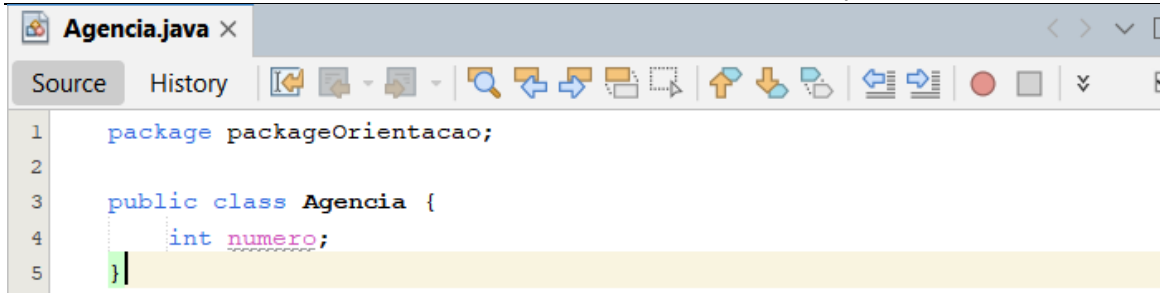


```

Output - OrientacaoAObjetos (run) x
run:
1111
01/01/2025
2222
01/01/2027
BUILD SUCCESSFUL (total time: 0 seconds)

```

7 - As agências do banco possuem número. Crie uma classe(Java Class) para definir os objetos que representarão as agências.

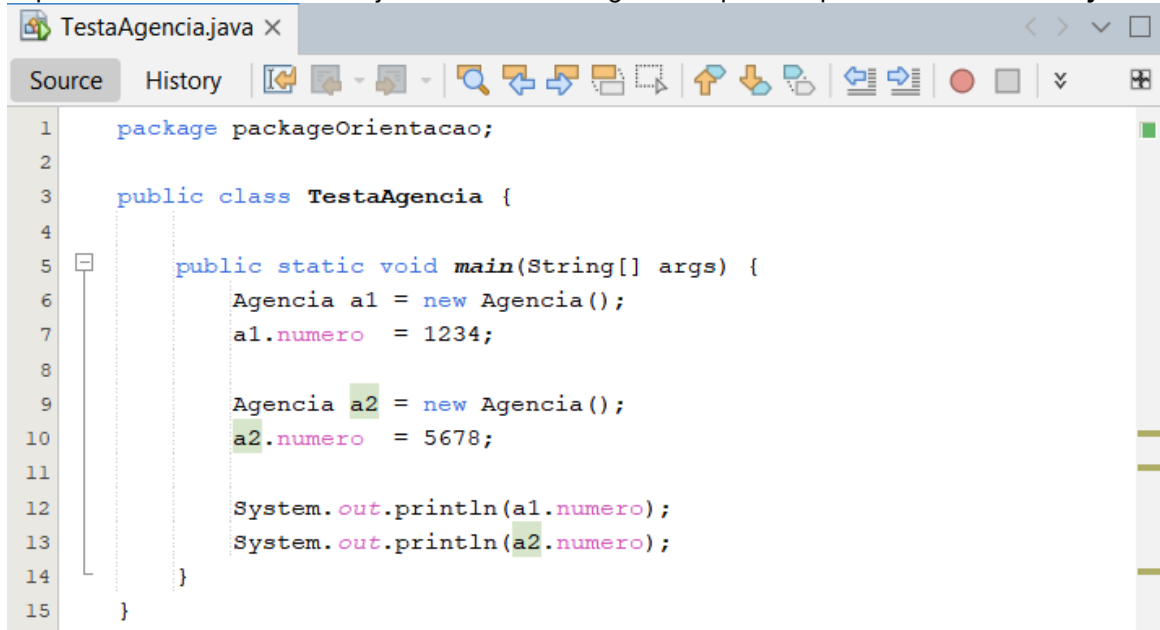


```

1 package packageOrientacao;
2
3 public class Agencia {
4     int numero;
5 }

```

8 - Faça um teste(Java Main Class) criando dois objetos da classe Agencia. Altere e imprima os atributos desses objetos. Adicione o seguinte arquivo na pasta **OrientacaoAObjetos**.

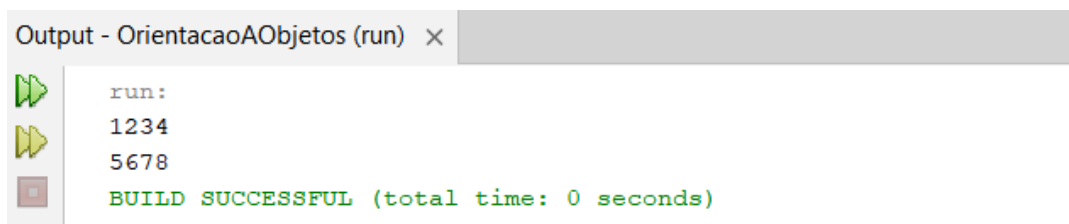


```

1 package packageOrientacao;
2
3 public class TestaAgencia {
4
5     public static void main(String[] args) {
6         Agencia a1 = new Agencia();
7         a1.numero = 1234;
8
9         Agencia a2 = new Agencia();
10        a2.numero = 5678;
11
12        System.out.println(a1.numero);
13        System.out.println(a2.numero);
14    }
15 }

```

Execute a classe TestaAgencia.

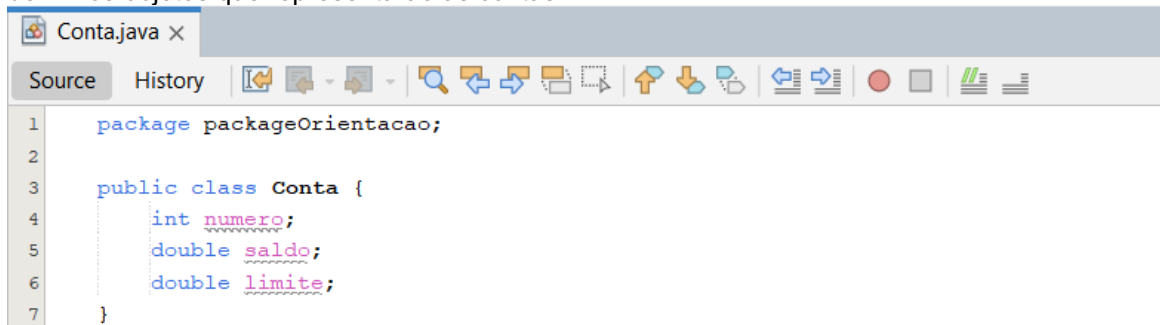


```

Output - OrientacaoAObjetos (run) x
run:
1234
5678
BUILD SUCCESSFUL (total time: 0 seconds)

```

9 - As contas do banco possuem número, saldo e limite. Crie uma classe(Java Class) para definir os objetos que representarão as contas.

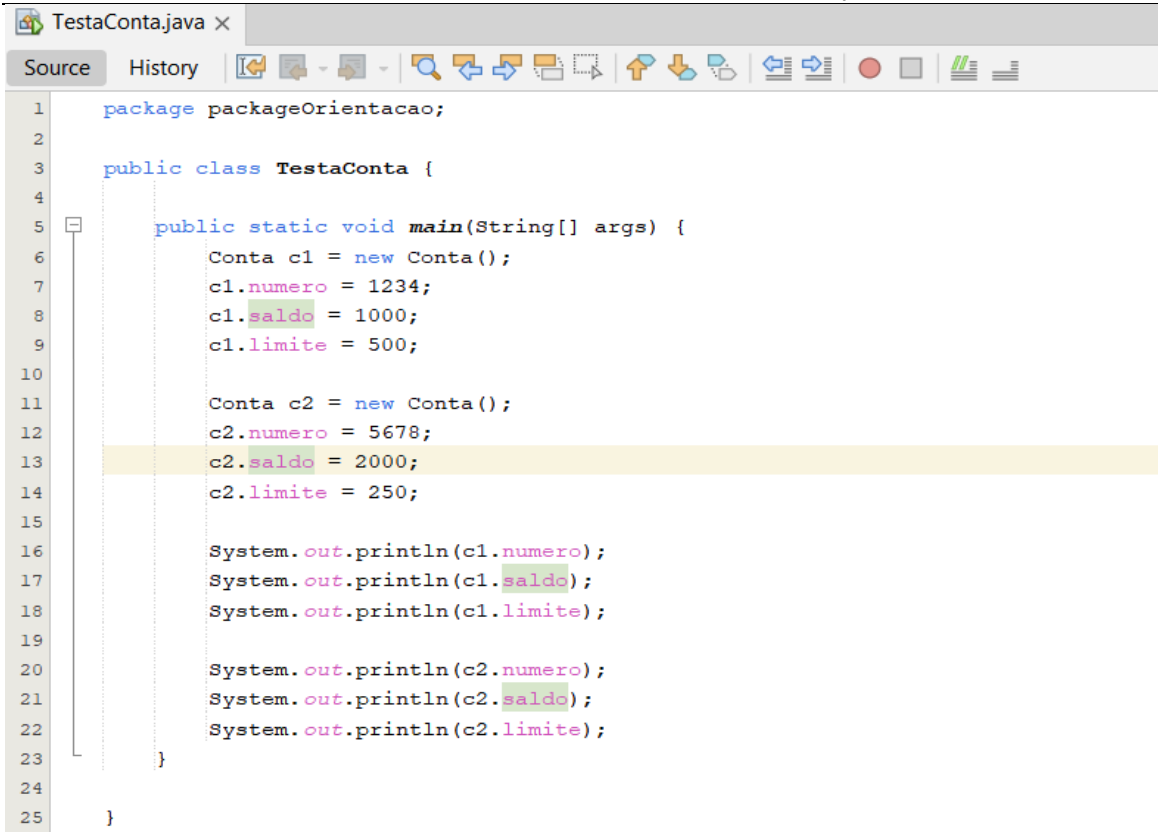


```

1 package packageOrientacao;
2
3 public class Conta {
4     int numero;
5     double saldo;
6     double limite;
7 }

```

10 - Faça um teste criando dois objetos da classe Conta(Java Main Class). Altere e imprima os atributos desses objetos. Adicione o seguinte arquivo na pasta **OrientacaoAObjetos**.

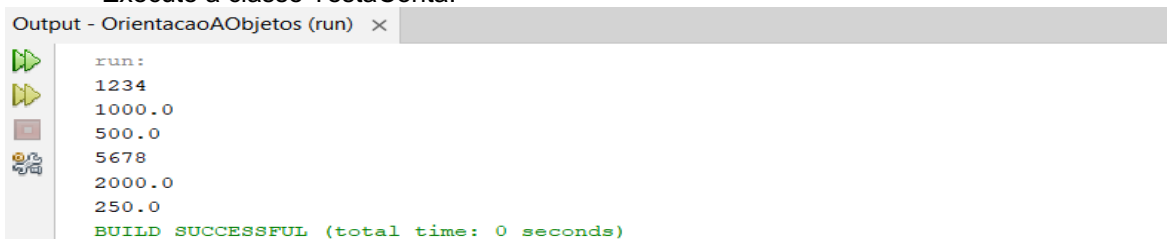


```

1  package packageOrientacao;
2
3  public class TestaConta {
4
5      public static void main(String[] args) {
6          Conta c1 = new Conta();
7          c1.numero = 1234;
8          c1.saldo = 1000;
9          c1.limite = 500;
10
11         Conta c2 = new Conta();
12         c2.numero = 5678;
13         c2.saldo = 2000;
14         c2.limite = 250;
15
16         System.out.println(c1.numero);
17         System.out.println(c1.saldo);
18         System.out.println(c1.limite);
19
20         System.out.println(c2.numero);
21         System.out.println(c2.saldo);
22         System.out.println(c2.limite);
23     }
24
25 }

```

Execute a classe TestaConta.

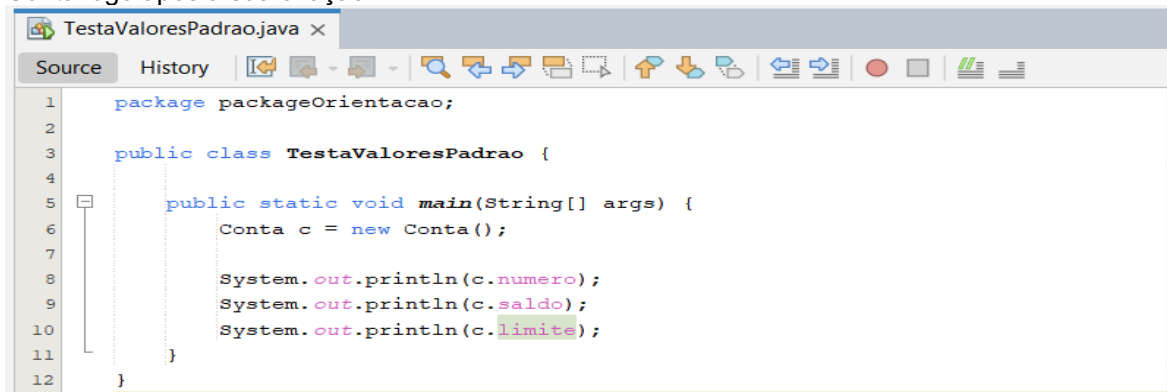


```

run:
1234
1000.0
500.0
5678
2000.0
250.0
BUILD SUCCESSFUL (total time: 0 seconds)

```

11 - Faça um teste(Java Main Class) que imprima os atributos de um objeto da classe Conta logo após a sua criação.

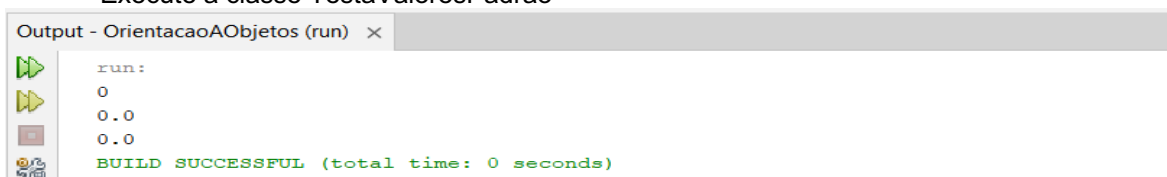


```

1  package packageOrientacao;
2
3  public class TestaValoresPadrao {
4
5      public static void main(String[] args) {
6          Conta c = new Conta();
7
8          System.out.println(c.numero);
9          System.out.println(c.saldo);
10         System.out.println(c.limite);
11     }
12 }

```

Execute a classe TestaValoresPadrao

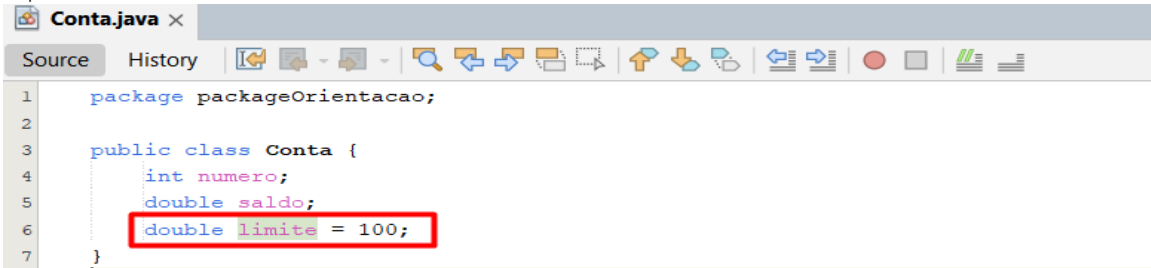


```

run:
0
0.0
0.0
BUILD SUCCESSFUL (total time: 0 seconds)

```

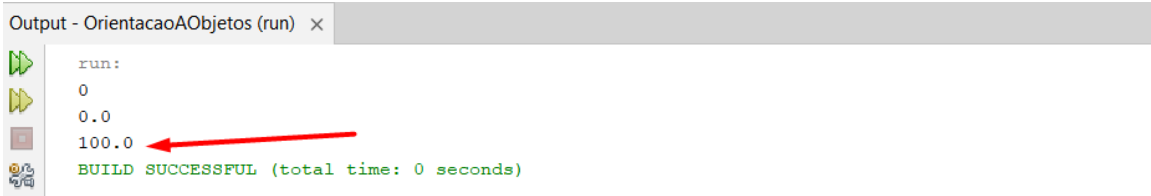
12. Altere a classe Conta para que todos os objetos criados a partir dessa classe possuam R\$ 100 de limite inicial.



```

1 package packageOrientacao;
2
3 public class Conta {
4     int numero;
5     double saldo;
6     double limite = 100;
7 }
    
```

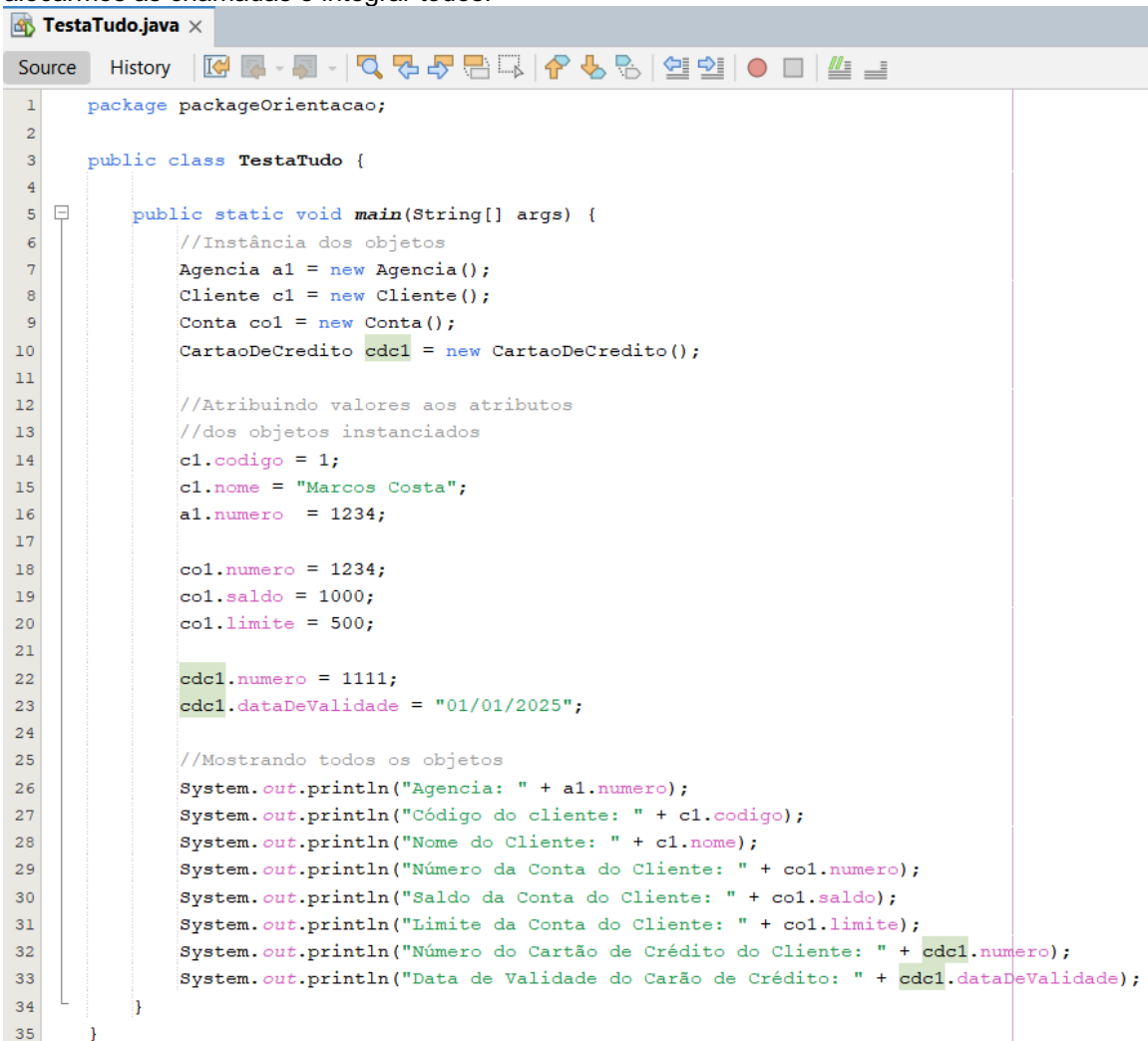
Execute a classe TestavaloresPadrao.



```

run:
0
0.0
100.0
BUILD SUCCESSFUL (total time: 0 seconds)
    
```

13. Criaremos uma classe Testatudo(Java Main Class) para criarmos todos os objetos, e alocarmos as chamadas e integrar todos.



```

1 package packageOrientacao;
2
3 public class Testatudo {
4
5     public static void main(String[] args) {
6         //Instância dos objetos
7         Agencia a1 = new Agencia();
8         Cliente c1 = new Cliente();
9         Conta co1 = new Conta();
10        CartaoDeCredito cdc1 = new CartaoDeCredito();
11
12        //Atribuindo valores aos atributos
13        //dos objetos instanciados
14        c1.codigo = 1;
15        c1.nome = "Marcos Costa";
16        a1.numero = 1234;
17
18        co1.numero = 1234;
19        co1.saldo = 1000;
20        co1.limite = 500;
21
22        cdc1.numero = 1111;
23        cdc1.dataDeValidade = "01/01/2025";
24
25        //Mostrando todos os objetos
26        System.out.println("Agencia: " + a1.numero);
27        System.out.println("Código do cliente: " + c1.codigo);
28        System.out.println("Nome do Cliente: " + c1.nome);
29        System.out.println("Número da Conta do Cliente: " + co1.numero);
30        System.out.println("Saldo da Conta do Cliente: " + co1.saldo);
31        System.out.println("Limite da Conta do Cliente: " + co1.limite);
32        System.out.println("Número do Cartão de Crédito do Cliente: " + cdc1.numero);
33        System.out.println("Data de Validade do Carão de Crédito: " + cdc1.dataDeValidade);
34    }
35 }
    
```

Executando o resultado fica assim:

```
Output - OrientacaoAObjetos (run) X
run:
Agencia: 1234
Codigo do cliente: 1
Nome do Cliente: Marcos Costa
Numero da Conta do Cliente: 1234
Saldo da Conta do Cliente: 1000.0
Limite da Conta do Cliente: 500.0
Numero do Cartao de Credito do Cliente: 1111
Data de Validade do Cartao de Credito: 01/01/2025
BUILD SUCCESSFUL (total time: 0 seconds)
```

Agora é com vocês, para praticar, instalem e desenvolvam este exercício.

18. Métodos

No banco, é possível realizar diversas operações em uma conta: depósito, saque, transferência, consultas e etc. Essas operações podem modificar ou apenas acessar os valores dos atributos dos objetos que representam as contas.

Essas operações são realizadas em métodos definidos na própria classe Conta. Por exemplo, para realizar a operação de depósito, podemos acrescentar o seguinte método na classe Conta.

```
1 void deposita(double valor) {
2     // implementação
3 }
```

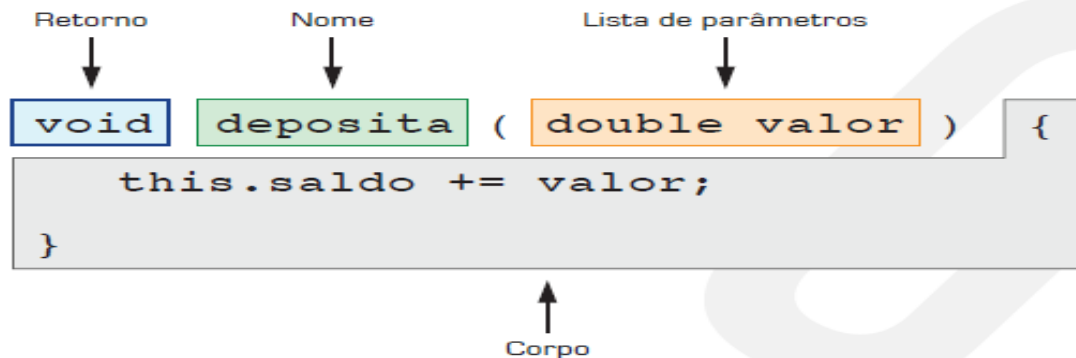
Podemos dividir um método em quatro partes:

Nome: É utilizado para chamar o método. Na linguagem Java, é uma **boa prática definir os nomes dos métodos utilizando a convenção "Camel Case"** com a primeira letra minúscula.

Lista de Parâmetros: Define os valores que o método deve receber. Métodos que não devem receber nenhum valor possuem a lista de parâmetros vazia.

Corpo: Define o que acontecerá quando o método for chamado.

Retorno: A resposta que será devolvida ao final do processamento do método. Quando um método **não devolve** nenhuma resposta, ele deve ser marcado coma palavra reservada void.



Para realizar um depósito, devemos chamar o método `deposita()` através da referência do objeto que representa a conta que terá o dinheiro creditado.

```
//Referencia de um objeto
```

```
Conta c = new Conta();
```

```
//Chamando o método deposita()
```

```
c.deposita(1000);
```

Normalmente, os métodos acessam ou alteram os valores armazenados nos atributos dos objetos.

Por exemplo, na execução do método `deposita()`, é necessário alterar o valor do atributo `saldo` do objeto que foi escolhido para realizar a operação.

Dentro de um método, para acessar os atributos do objeto que está processando o método, devemos utilizar a palavra reservada `this`.

```
void deposita (double valor){
    this.saldo += valor;
}
```

O método `deposita()` não possui nenhum retorno lógico. Por isso, foi marcado com `void`. Mas, para outros métodos, pode ser necessário definir um tipo de retorno específico.

Considere, por exemplo, um método para realizar a operação que consulta o saldo disponível das contas. Suponha também que o saldo disponível é igual a soma do saldo e do limite. Então, esse método deve somar os atributos `saldo` e `limite` e devolver o resultado. Por outro lado, esse método não deve receber nenhum valor, pois todas as informações necessárias para realizar a operação estão nos atributos dos objetos que representam as contas.

```
double consultaSaldoDisponivel () {
    return this.saldo + this.limite;
}
```

Ao chamar o método `consultaSaldoDisponivel()` a resposta pode ser armazenada em uma variável do tipo `double`.

```
//Referencia de um objeto
```

```
Conta c = new Conta();
```

```
//Armazenando a resposta de um método em variável
```

```
double saldoDisponivel = c.consultaSaldoDisponivel();
```

```
System.out.println("Saldo disponível: " + saldoDisponivel);
```

19. Exercícios de Fixação

1-) Acrescente alguns métodos na classe `Conta` (Java Class) para realizar as operações de depósito, saque, impressão de extrato e consulta do saldo disponível.

```
1 package packageOrientacao;
2
3 public class Conta {
4     int numero ;
5     double saldo ;
6     double limite = 100;
7
8     void deposita (double valor){
9         this.saldo += valor;
10    }
11
12    void saca (double valor){
13        this.saldo -= valor;
14    }
15
16    void imprimeExtrato () {
17        System.out.println("SALDO: " + this.saldo);
18    }
19
20    double consultaSaldoDisponivel () {
21        return this.saldo + this.limite;
22    }
23 }
```

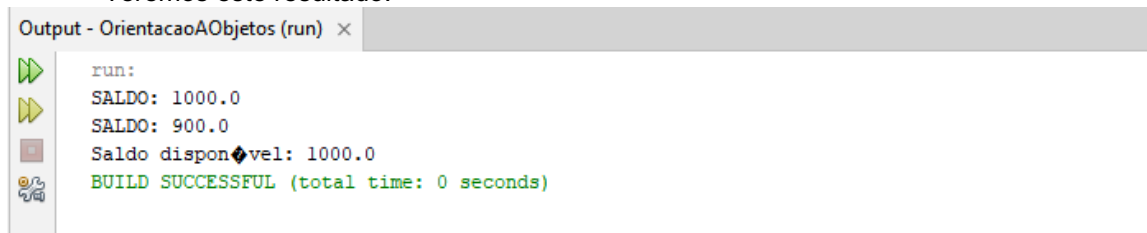

2-) Teste os métodos da classe Conta (Java Main Class), para isso crie:

```

1 package packageOrientacao;
2
3 public class TestaMetodosConta {
4
5     public static void main(String[] args) {
6         //Referencia de um objeto
7         Conta c = new Conta();
8
9         //Chamando o método deposita()
10        c.deposita(1000);
11        c.imprimeExtrato();
12
13        c.saca(100);
14        c.imprimeExtrato();
15
16        //Armazenando a resposta de um método em variável
17        double saldoDisponivel = c.consultaSaldoDisponivel();
18        System.out.println("Saldo disponível: " + saldoDisponivel);
19    }
20 }

```

Teremos este resultado:



```

Output - OrientacaoAObjetos (run) x
run:
SALDO: 1000.0
SALDO: 900.0
Saldo disponível: 1000.0
BUILD SUCCESSFUL (total time: 0 seconds)

```

20. Exercícios Complementares

Para esta atividade, crie um novo projeto que pode ser chamado de ExercicioMetodos, execute os passos seguintes, após isso, submeta o projeto ao GitHub, e nas tarefas no Microsoft Teams, informem o link do repositório. Prazo, até o dia 05/03/2026, esta atividade servirá para caso no final do semestre, tenhamos alguém precisando de nota e contará como participação e ajudará na mesma. Na aula seguinte, faremos a implementação do mesmo e continuidade com a matéria.

1-) Em uma empresa, temos os funcionários, que precisam ser representados em nossa aplicação. Então implemente uma classe chamada Funcionario que contenha dois atributos: o primeiro para o nome e o segundo para o salário dos funcionários.

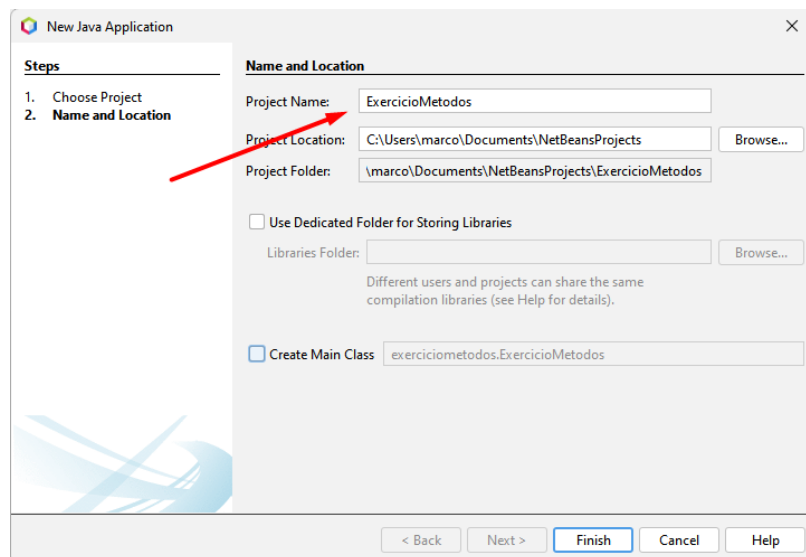
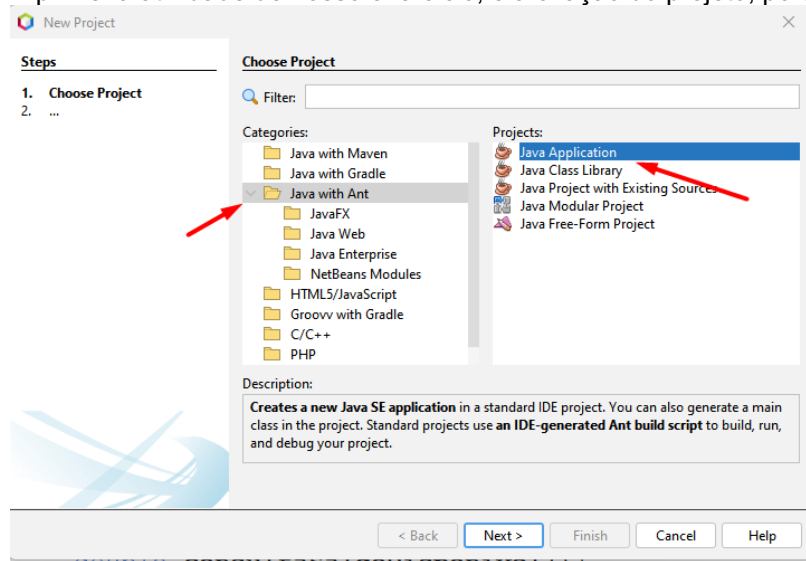
2-) Faça uma classe chamada TestaFuncionario e crie dois objetos da classe Funcionario atribuindo valores a eles. Mostre na tela as informações desses objetos.

3-) Adicione na classe Funcionario dois métodos: um para aumentar o salário e outro para consultar os dados dos funcionários.

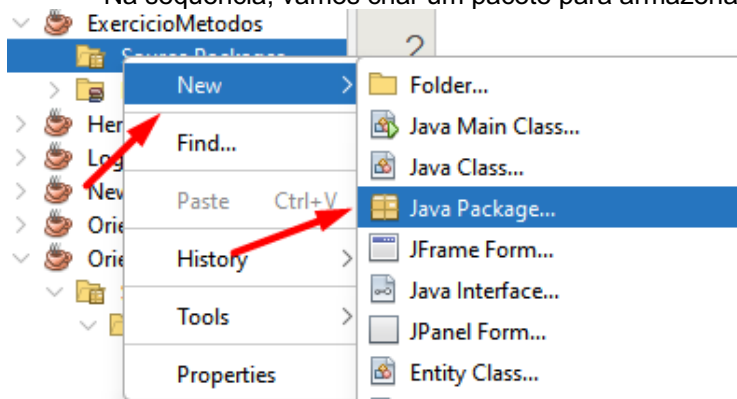
4-) Na classe TestaFuncionario teste novamente os métodos de um objeto da classe Funcionario.

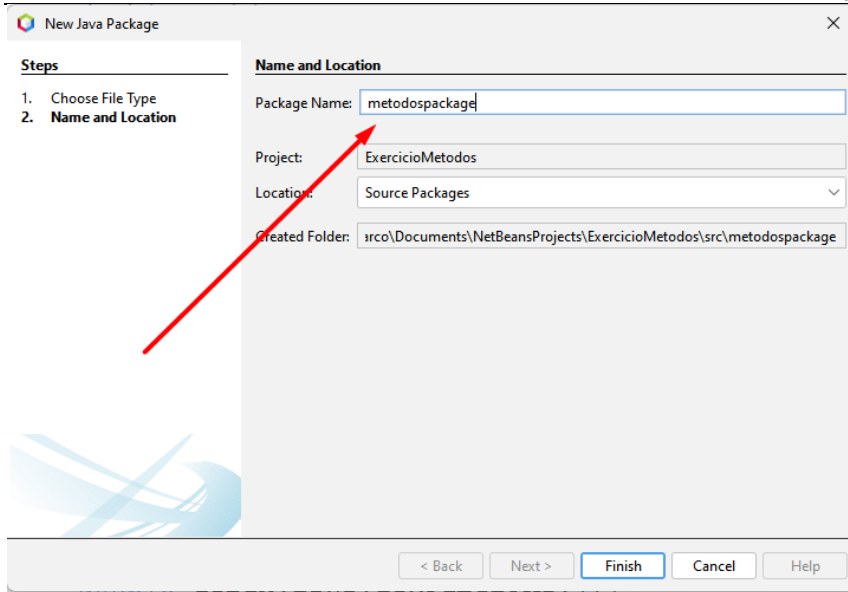
20.1. Correção dos Exercícios Complementares

A primeira atividade de nosso exercício, é a criação do projeto, portanto, devemos:

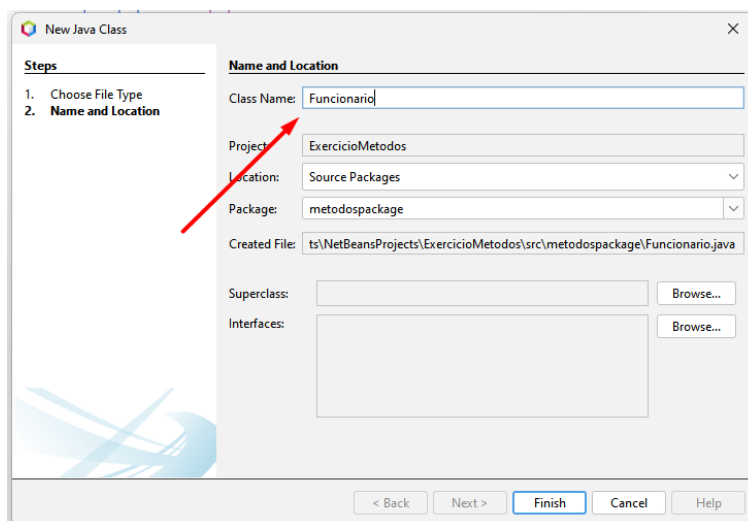
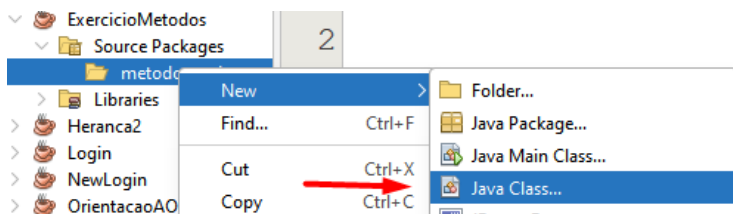


Na sequência, vamos criar um pacote para armazenar nossos arquivos.

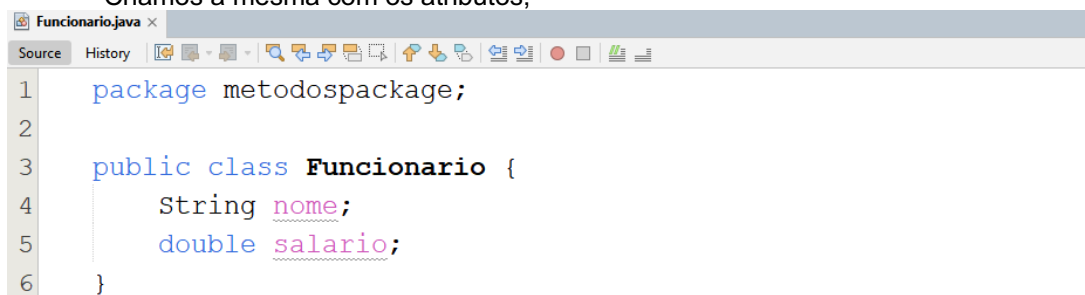




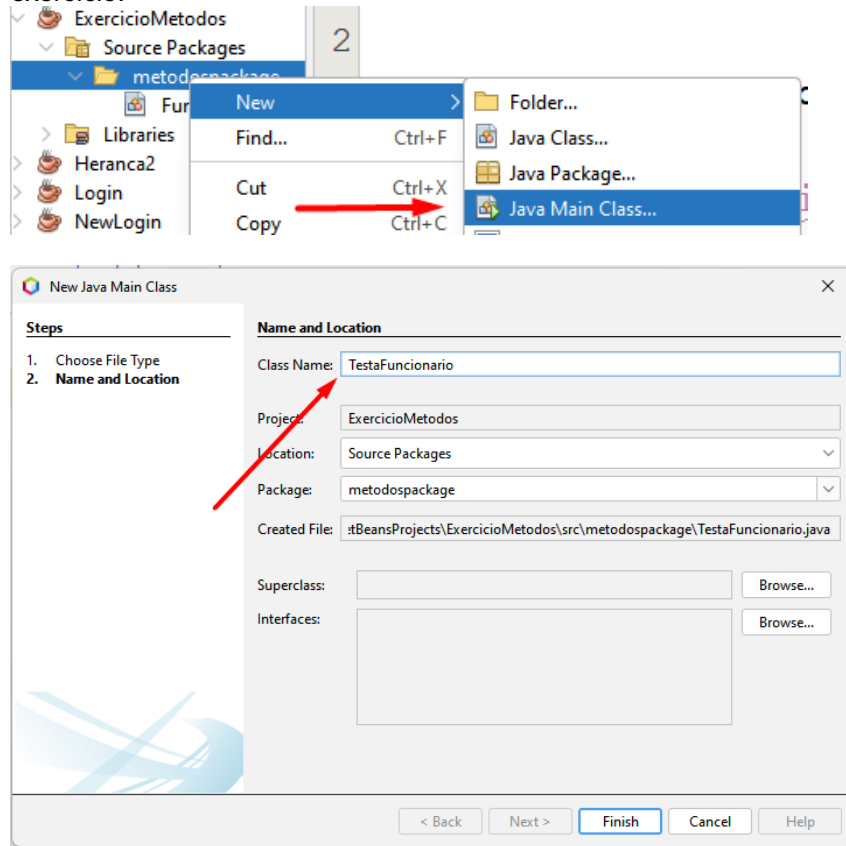
No primeiro exercício, vamos implementar a classe Funcionario (Java Class) e os atributos solicitados.



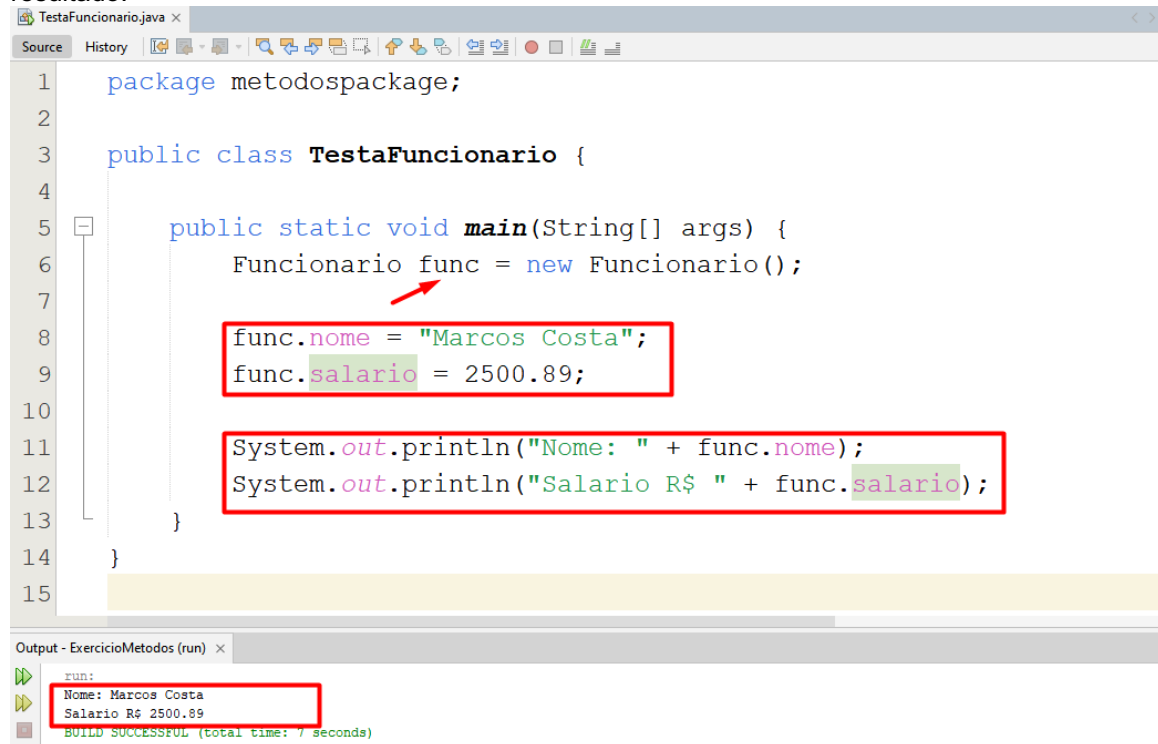
Criamos a mesma com os atributos;



Agora, dando continuidade, vamos criar uma *TestaFuncionario (Java Main Class)* para instanciar o objeto e fazer a inclusão dos dados nos atributos e os mostrar, conforme pedido no exercício:



Agora, codificamos conforme pedido no item 2 de nosso exercício e mostramos o resultado:



Continuando nosso exercício, agora é pedido a criação de dois métodos na classe Funcionario, um para aumento de salário, neste caso, vou fazer um cálculo com um percentual que será passado, e outro método que mostrará os dados do funcionário, ou seja, a própria classe Funcionario, irá ter um método para mostrar, ao invés de fazermos isso na *Java Main Class*.

```

Funcionario.java x
Source History
1 package metodospackage;
2
3 public class Funcionario {
4     String nome;
5     double salario;
6
7     void aumentaSalario(double percAumento) {
8         this.salario = this.salario + (this.salario * percAumento);
9     }
10
11     void mostraDadosFuncionario() {
12         System.out.println("Nome: " + this.nome);
13         System.out.println("Salario R$ " + this.salario);
14     }
15 }

```

Agora, a ideia é alterarmos a *Java Main Class* TestaFuncionario, atribuindo as execuções, conforme solicitado no item 4 de nossa atividade.

```

TestaFuncionario.java x
Source History
1 package metodospackage;
2
3 public class TestaFuncionario {
4
5     public static void main(String[] args) {
6         Funcionario func = new Funcionario();
7
8         func.nome = "Marcos Costa";
9         func.salario = 2500.89;
10
11         func.aumentaSalario(10);
12         func.mostraDadosFuncionario();
13     }
14 }

```

Output - ExercícioMetodos (run) x

```

run:
Nome: Marcos Costa
Salario R$ 27509.789999999997
BUILD SUCCESSFUL (total time: 0 seconds)

```

Com isso, finalizamos esta correção, sempre lembro que temos outras formas de resolver este mesmo exercício, o que importa é a implementação do que é solicitado no enunciado do mesmo, portanto, se fez de uma forma diferente, mas atingiu o mesmo objetivo, parabéns, este é o caminho.

21. Sobrecarga (Overloading)

Voltando ao nosso conceito, os clientes dos bancos costumam consultar periodicamente informações relativas às suas contas.

Geralmente, essas informações são obtidas através de extratos. No sistema do banco, os extratos podem ser gerados por métodos da classe Conta.

```

Conta.java x
Source History
1 package metodospackage;
2
3 public class Conta {
4     int numero ;
5     double saldo ;
6     double limite = 100;
7
8     void imprimeExtrato (int dias){
9         //Extrato
10    }
11 }

```

O método `imprimeExtrato()` recebe a quantidade de dias que deve ser considerada para gerar o extrato da conta. Por exemplo, se esse método receber o valor 30 então ele deve gerar um extrato com as movimentações dos últimos 30 dias.

Em geral, extratos dos últimos 15 dias atendem as necessidades dos clientes. Dessa forma, poderíamos acrescentar um método na classe Conta para gerar extratos com essa quantidade fixa de dias.

```

Conta.java x
Source History
1 package metodospackage;
2
3 public class Conta {
4     int numero ;
5     double saldo ;
6     double limite = 100;
7
8     void imprimeExtrato () {
9         //Extrato dos ultimos 15 dias
10    }
11
12    void imprimeExtrato (int dias){
13        //Extrato
14    }
15 }

```

O primeiro método não recebe parâmetros pois ele utilizará uma quantidade de dias padrão definida pelo banco para gerar os extratos (15 dias).

O segundo recebe um valor inteiro como parâmetro e deve considerar essa quantidade de dias para gerar os extratos.

Os dois métodos possuem o mesmo nome e lista de parâmetros diferentes. Quando dois ou mais métodos são definidos na mesma classe com o mesmo nome, dizemos que houve uma **sobrecarga** de métodos. Uma sobrecarga de métodos só é válida se as listas de parâmetros dos métodos são diferentes entre si.

No caso dos dois métodos que geram extratos, poderíamos evitar repetição de código fazendo um método chamar o outro.

```
Conta.java x
Source History
1 package metodospackage;
2
3 public class Conta {
4     int numero ;
5     double saldo ;
6     double limite = 100;
7
8     void imprimeExtrato () {
9         this.imprimeExtrato(15);
10    }
11
12    void imprimeExtrato (int dias){
13        //Extrato
14    }
15 }
```

21.1 Exercícios de Fixação

1-) Crie uma classe (Java Class) chamada **Gerente** para definir os objetos que representarão os gerentes do banco. Defina dois métodos de aumento salarial nessa classe. O primeiro deve aumentar o salário com uma taxa fixa de 10%. O segundo deve aumentar o salário com uma taxa variável.

```
Gerente.java x
Source History
1 package packageOrientacao;
2
3 public class Gerente {
4     String nome;
5     double salario;
6
7     void aumentaSalario() {
8         this.aumentaSalario(0.1);
9     }
10
11    void aumentaSalario(double taxa) {
12        this.salario += this.salario * taxa;
13    }
14 }
```

Teste os métodos de aumento salarial definidos na classe Gerente.


```
1 package packageOrientacao;
2
3 public class TestaGerente {
4
5     public static void main(String[] args) {
6         Gerente g = new Gerente();
7         g.salario = 1000;
8
9         System.out.println("Salario R$ " + g.salario);
10
11         System.out.println("Aumentando o salário em 10%");
12         g.aumentaSalario();
13
14         System.out.println("Salario com 10% R$ " + g.salario);
15
16         System.out.println("Aumentando o salário em 30%");
17         g.aumentaSalario(0.3);
18
19         System.out.println("Salario com 30% R$ " + g.salario);
20     }
21 }
```

Output - OrientacaoAObjetos (run) ×

```
run:
Salario R$ 1000.0
Aumentando o salário em 10%
Salario com 10% R$ 1100.0
Aumentando o salário em 30%
Salario com 30% R$ 1430.0
```

Compile e execute a classe TestaGerente.

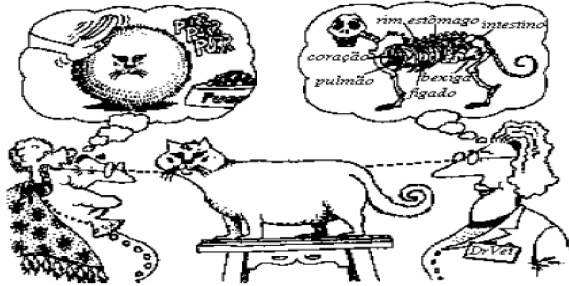
22. Abstração

Uma das principais formas do ser humano lidar com a **complexidade é através do uso de abstrações**. As pessoas tipicamente tentam compreender o mundo, construindo modelos mentais de partes dele. Tais modelos são uma visão simplificada de algo, onde apenas elementos relevantes são considerados. Modelos mentais, portanto, são mais simples do que os complexos sistemas que eles modelam.

Consideremos, por exemplo, um mapa como um modelo do território que ele representa. Um mapa é útil porque abstrai apenas aquelas características do território que se deseja modelar. Se um mapa incluísse todos os detalhes do território, provavelmente teria o mesmo tamanho do território e, portanto, não serviria a seu propósito.

Da mesma forma que um mapa precisa ser significativamente menor que o território que mapeia, incluindo apenas informações cuidadosamente selecionadas, um modelo mental abstrai apenas as características relevantes de um sistema para seu entendimento. Assim, podemos definir abstração como sendo o princípio de ignorar aspectos não relevantes de um assunto, segundo a perspectiva de um observador, tornando possível uma concentração maior nos aspectos principais do mesmo. De fato, a abstração consiste na seleção que um observador faz de alguns aspectos de um assunto, em detrimento de outros que não demonstram ser relevantes para o propósito em questão, isto é, a abstração é aplicada de acordo com o interesse do observador, e por isso de um mesmo objeto pode-se ter diferentes visões, como demonstra a figura 1 abaixo.

Só devem ser mapeados os objetos que são relevantes ao problema, bem como as características (propriedades e comportamento) desses objetos que forem necessários.



No que tange ao desenvolvimento de software, duas formas adicionais de abstração têm grande importância: a abstração de dados e a abstração de procedimentos.

23. Atributos Privados

No sistema do banco, cada objeto da classe `Funcionario` possui um atributo para guardar o salário do funcionário que ele representa.

```
Funcionario.java x
Source History
1 package packageOrientacao;
2
3 public class Funcionario {
4     double salario;
5 }
```

O atributo `salario` pode ser acessado ou modificado por código escrito em qualquer classe que esteja no mesmo diretório que a classe `Funcionario`. Portanto, o controle desse atributo é descentralizado.

Para identificar algum erro relacionado a manipulação dos salários dos funcionários, é necessário verificar o código de todos os arquivos da pasta onde a classe `Funcionario` está definida. Quanto maior o número de arquivos, menos eficiente será a manutenção da aplicação.

Podemos obter um controle centralizado tornando o atributo `salario` **privado** e definindo métodos para implementar todas as lógicas que utilizam ou modificam o valor desse atributo.

```
Funcionario.java x
Source History
1 package packageOrientacao;
2
3 public class Funcionario {
4     private double salario;
5
6     void aumentaSalario(double aumento) {
7         //Lógica para aumentar o salário
8     }
9 }
```

Um atributo privado só pode ser acessado ou alterado por código escrito dentro da classe na qual ele foi definido. Se algum código fora da classe `Funcionario` tentar acessar ou alterar o valor do atributo privado `salario`, um erro de compilação será gerado.

Definir todos os atributos como privado e métodos para implementar as lógicas de acesso e alteração é quase uma regra da orientação a objetos. O intuito é ter sempre um controle centralizado dos dados dos objetos para facilitar a manutenção do sistema e a detecção de erros.

24. Métodos Privados

O papel de alguns métodos pode ser o de auxiliar outros métodos da mesma classe. E muitas vezes, não é correto chamar esses métodos auxiliares de fora da sua classe diretamente.

No exemplo abaixo, o método `desconta Tarifa()` é um método auxiliar dos métodos `deposita()` e `saca()`. Além disso, ele não deve ser chamado diretamente, pois a tarifa só deve ser descontada quando ocorre um depósito ou um saque.

```
Conta.java x
Source History
1 package packageOrientacao;
2
3 public class Conta {
4     private double saldo;
5
6     void deposita(double valor){
7         this.saldo += valor;
8         this.descontaTarifa();
9     }
10
11    void saca(double valor){
12        this.saldo -= valor;
13        this.descontaTarifa();
14    }
15
16    void descontaTarifa(){
17        this.saldo -= 0.1;
18    }
19 }
```

Para garantir que métodos auxiliares não sejam chamados por código escrito fora da classe na qual eles foram definidos, podemos torná-los privados, acrescentando o modificador `private`.

```
16 private void descontaTarifa(){
17     this.saldo -= 0.1;
18 }
```

Qualquer chamada ao método `descontaTarifa()` realizada fora da classe `Conta` gera um erro de compilação.

25. Métodos Públicos

Os métodos que devem ser chamados a partir de qualquer parte do sistema devem possuir o modificador de visibilidade `public`.

```
Conta.java x
Source History
1 package packageOrientacao;
2
3 public class Conta {
4     private double saldo;
5
6     public void deposita(double valor){
7         this.saldo += valor;
8         this.descontaTarifa();
9     }
10
11    public void saca(double valor){
12        this.saldo -= valor;
13        this.descontaTarifa();
14    }
15
16    private void descontaTarifa(){
17        this.saldo -= 0.1;
18    }
19 }
```

26. Por que encapsular?

Uma das ideias mais importantes da orientação a objetos é o encapsulamento. Encapsular significa esconder a implementação dos objetos. O encapsulamento favorece principalmente dois aspectos de um sistema: a manutenção e o desenvolvimento.

A manutenção é favorecida pois, uma vez aplicado o encapsulamento, quando o funcionamento de um objeto deve ser alterado, em geral, basta modificar a classe do mesmo.

O desenvolvimento é favorecido pois, uma vez aplicado o encapsulamento, conseguimos determinar precisamente as responsabilidades de cada classe da aplicação.

O conceito de encapsulamento pode ser identificado em diversos exemplos do cotidiano. Mostraremos alguns desses exemplos para esclarecer melhor a ideia.

26.1.1.1. Celular - Escondendo a complexidade

Hoje em dia, as pessoas estão acostumadas com os celulares. Os botões, a tela e os menus de um celular formam a **interface de uso** do mesmo. Em outras palavras, o usuário interage com esses aparelhos através dos botões, da tela e dos menus. Os dispositivos internos de um celular e os processos que transformam o som capturado pelo microfone em ondas que podem ser transmitidas para uma antena da operadora de telefonia móvel constituem a **implementação** do celular.

Do ponto de vista do usuário de um celular, para fazer uma ligação, basta digitar o número do telefone desejado e clicar no botão que efetua a ligação. Porém, diversos processos complexos são realizados pelo aparelho para que as pessoas possam conversar através dele. Se os usuários tivessem que possuir conhecimento de todo o funcionamento interno dos celulares, certamente a maioria das pessoas não os utilizariam.

No contexto da orientação a objetos, aplicamos o encapsulamento para criar objetos mais simples de serem utilizados em qualquer parte do sistema.



26.1.2. Carro - Evitando efeitos colaterais

A interface de uso de um carro é composta pelos dispositivos que permitem que o motorista conduza o veículo (volante, pedais, alavanca do câmbio, etc).

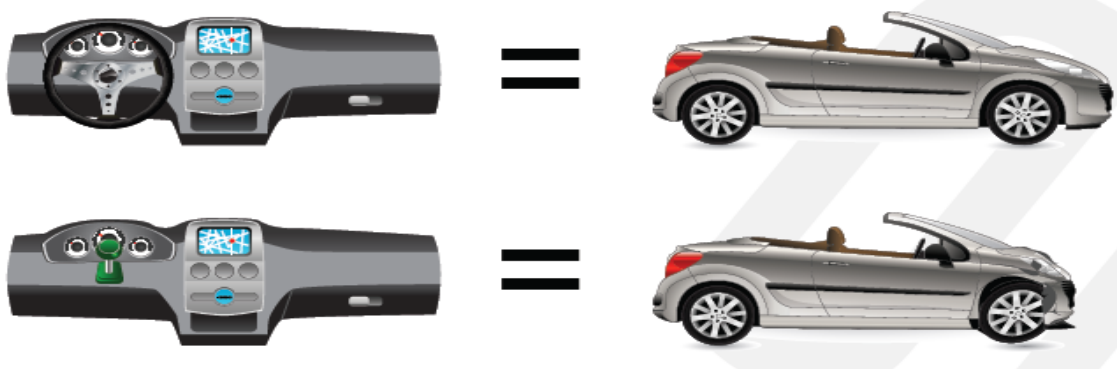
A implementação do carro é composta pelos dispositivos internos (motor, caixa de câmbio, radiador, sistema de injeção eletrônica ou carburador, etc) e pelos processos realizados internamente por esses dispositivos.

Nos carros mais antigos, o dispositivo interno que leva o combustível para o motor é o carburador. Nos carros mais novos, o carburador foi substituído pelo sistema de injeção eletrônica. Inclusive, algumas oficinas especializadas substituem o carburador pelo sistema de injeção eletrônica.

Essa alteração na implementação do carro não afeta a maneira que o motorista dirige. Todo mundo que sabe dirigir um carro com carburador também sabe dirigir um carro com injeção eletrônica. Hoje em dia, as montadoras fabricam veículos com câmbio mecânico ou automático. O motorista acostumado a dirigir carros com câmbio mecânico pode ter dificuldade para dirigir carros com câmbio automático e vice-versa. Quando a interface de uso do carro é alterada, a maneira de dirigir é afetada, fazendo com que as pessoas que sabem dirigir tenham que se adaptar.

No contexto da orientação a objetos, aplicando o conceito do encapsulamento, as implementações dos objetos ficam “escondidas”. Dessa forma, podemos modificá-las sem afetar a maneira de utilizar esses objetos. Por outro lado, se alterarmos a interface de uso que está exposta, afetaremos a maneira de usar os objetos.

Considere, por exemplo, a mudança do nome de um método público. Todas as chamadas a esse método devem ser alteradas, o que pode causar diversos efeitos colaterais nas classes da aplicação.



26.1.3. Máquinas de Porcarias - Aumentando o controle

Estamos acostumados a utilizar máquinas de refrigerantes, de salgadinhos, de doces, de café, etc. Em geral, essas máquinas oferecem uma interface de uso composta por:

- Entradas para moedas ou cédulas.
- Botões para escolher o produto desejado.
- Saída do produto.
- Saída para o troco.

Normalmente, essas máquinas são extremamente protegidas. Elas garantem que nenhum usuário mal intencionado (ou não) tente alterar a implementação da máquina, ou seja, tente alterar como a máquina funciona por dentro.

Levando essa ideia para um sistema orientado a objetos, um objeto deve ser bem protegido para que outros objetos não prejudiquem o seu funcionamento interno.



27. Acessando ou modificando atributos

Aplicando a ideia do encapsulamento, os atributos deveriam ser todos privados. Consequentemente, os atributos não podem ser acessados ou modificados por código escrito fora da classe na qual eles foram definidos.

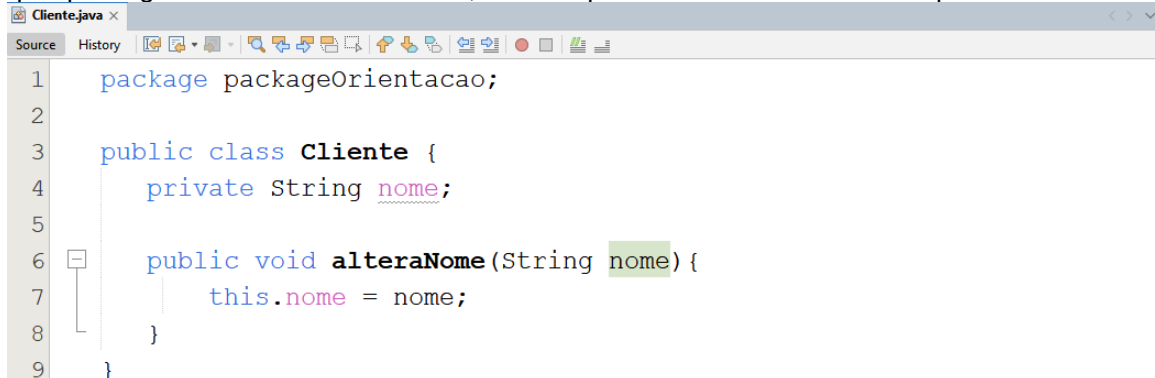
Porém, muitas vezes, as informações armazenadas nos atributos precisam ser consultadas de qualquer lugar do sistema. Nesse caso, **podemos disponibilizar métodos para consultar os valores dos atributos.**

```

Cliente.java x
Source History
1 package packageOrientacao;
2
3 public class Cliente {
4     private String nome;
5
6     public String consultaNome() {
7         return this.nome;
8     }
9 }

```

Da mesma forma, eventualmente, é necessário modificar o valor de um atributo a partir de qualquer lugar do sistema. Nesse caso, também poderíamos criar um método para essa tarefa.



```
1 package packageOrientacao;
2
3 public class Cliente {
4     private String nome;
5
6     public void alteraNome(String nome) {
7         this.nome = nome;
8     }
9 }
```

Muitas vezes, é necessário consultar e alterar o valor de um atributo a partir de qualquer lugar do sistema. Nessa situação, podemos definir os dois métodos discutidos anteriormente. Mas, o que é melhor? Criar os dois métodos (um de leitura e outro de escrita) ou deixar o atributo público?

Quando queremos consultar a quantidade de combustível de um automóvel, olhamos o painel ou abrimos o tanque de combustível?

Quando queremos alterar o toque da campainha de um celular, utilizamos os menus do celular ou desmontamos o aparelho?

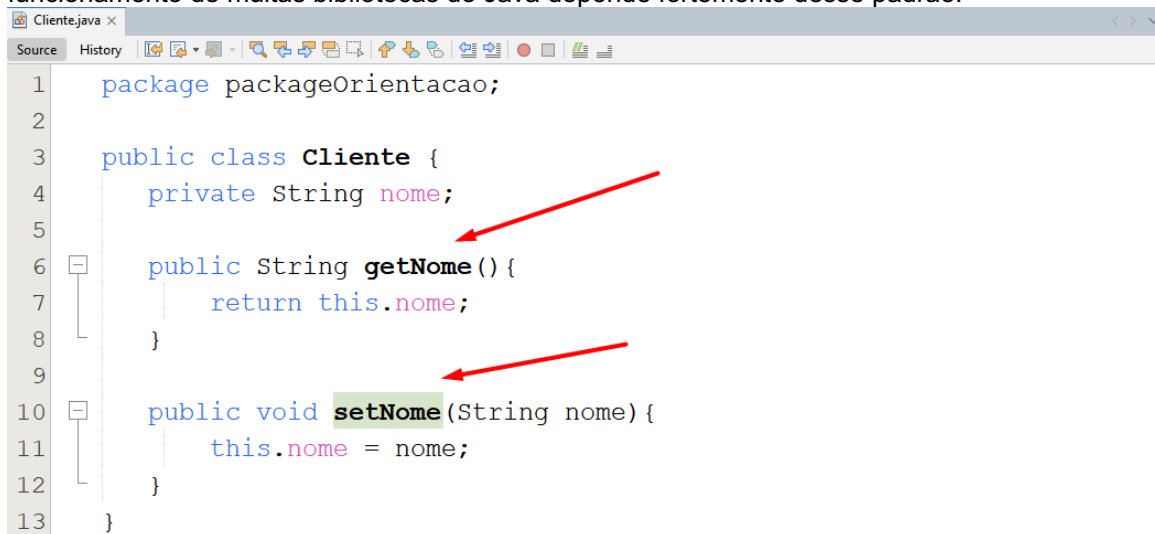
Acessar ou modificar as propriedades de um objeto manipulando diretamente os seus atributos é uma abordagem que normalmente gera problemas. Por isso, é mais seguro para a integridade dos objetos e, conseqüentemente, para a integridade da aplicação, que esse acesso ou essa modificação sejam realizados através de métodos do objeto. Utilizando métodos, podemos controlar como as alterações e as consultas são realizadas. Ou seja, temos um controle maior.

28. Getters e Setters

Na linguagem Java, há uma convenção de nomenclatura para os métodos que têm como finalidade acessar ou alterar as propriedades de um objeto.

Segundo essa convenção, os nomes dos métodos que permitem a consulta das propriedades de um objeto devem possuir o prefixo **get**. Analogamente, os nomes dos métodos que permitem a alteração das propriedades de um objeto devem possuir o prefixo **set**.

Na maioria dos casos, é muito conveniente seguir essa convenção, pois os desenvolvedores Java já estão acostumados com essas regras de nomenclatura e o funcionamento de muitas bibliotecas do Java depende fortemente desse padrão.

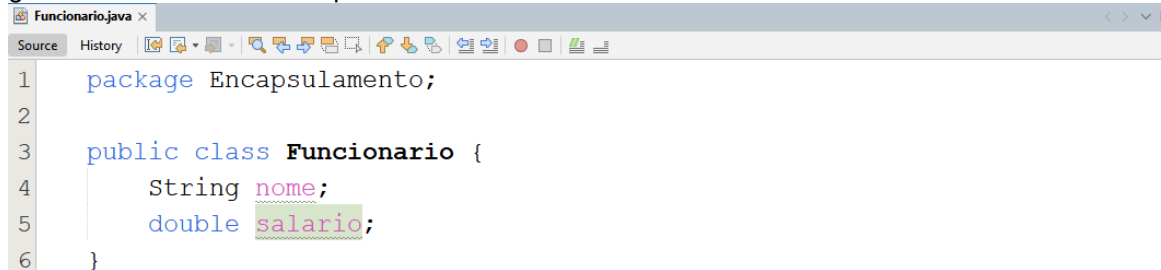


```
1 package packageOrientacao;
2
3 public class Cliente {
4     private String nome;
5
6     public String getNome() {
7         return this.nome;
8     }
9
10    public void setNome(String nome) {
11        this.nome = nome;
12    }
13 }
```

29. Exercícios de Fixação

1-) Crie um projeto no NetBeans chamado **Encapsulamento**.

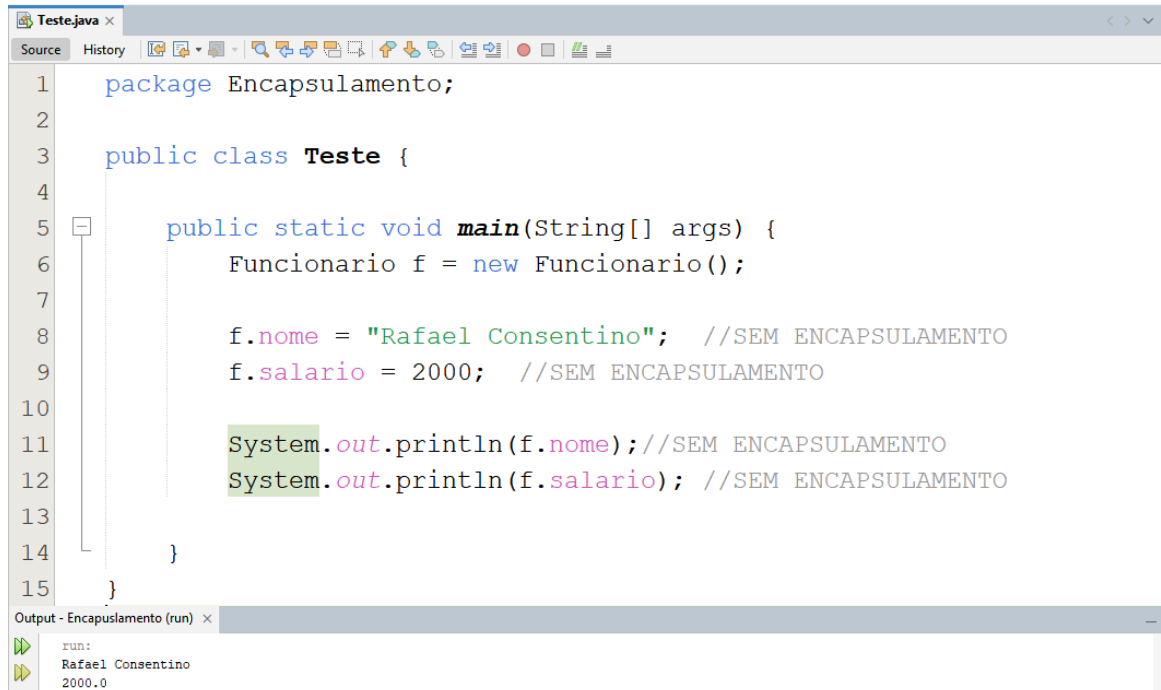
2-) Defina uma classe para representar os funcionários do banco com um atributo para guardar os salários e outro para os nomes.



```

1 package Encapsulamento;
2
3 public class Funcionario {
4     String nome;
5     double salario;
6 }
    
```

3-) Teste a classe Funcionario criando um objeto e manipulando diretamente os seus atributos.



```

1 package Encapsulamento;
2
3 public class Teste {
4
5     public static void main(String[] args) {
6         Funcionario f = new Funcionario();
7
8         f.nome = "Rafael Consentino"; //SEM ENCAPSULAMENTO
9         f.salario = 2000; //SEM ENCAPSULAMENTO
10
11         System.out.println(f.nome); //SEM ENCAPSULAMENTO
12         System.out.println(f.salario); //SEM ENCAPSULAMENTO
13
14     }
15 }
    
```

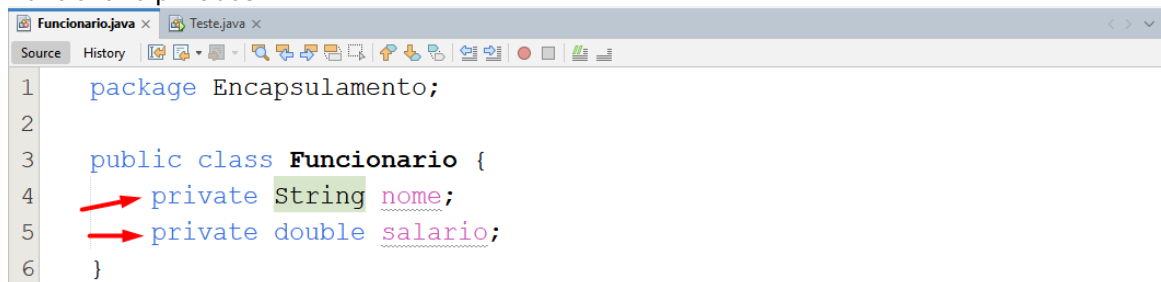
Output - Encapsulamento (run) x

```

run:
Rafael Consentino
2000.0
    
```

4-) Compile a classe Teste e perceba que ela pode acessar ou modificar os valores dos atributos de um objeto da classe Funcionario. Execute o teste e observe o console.

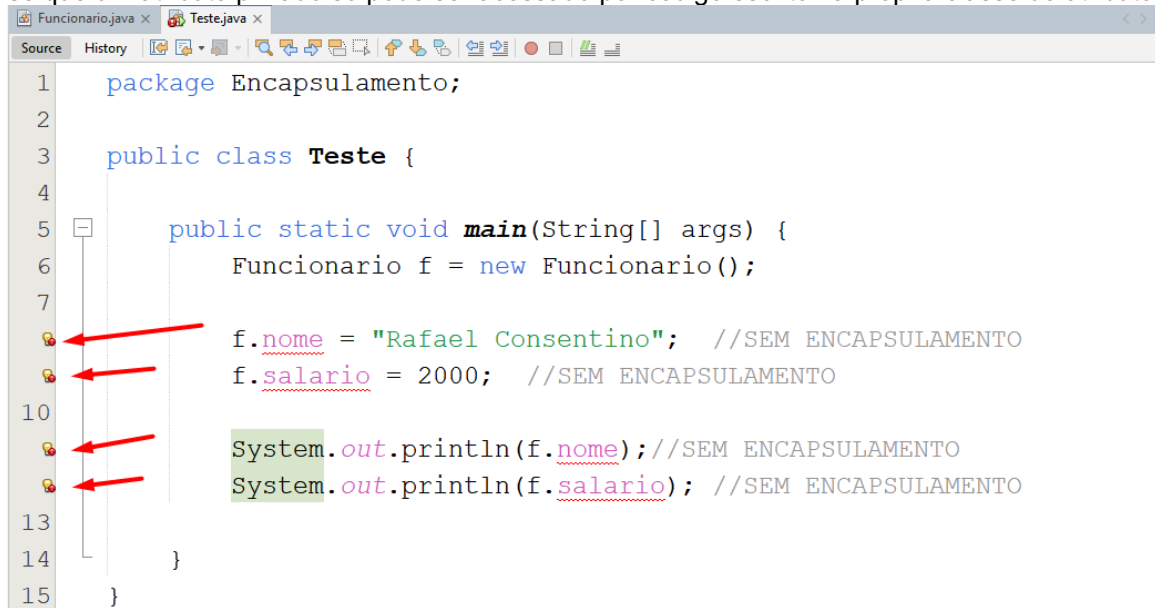
5-) Aplique a ideia do encapsulamento tornando os atributos definidos na classe Funcionario privados.



```

1 package Encapsulamento;
2
3 public class Funcionario {
4     private String nome;
5     private double salario;
6 }
    
```


6-) Tente compilar novamente a classe Teste. Observe os erros de compilação. Lembre-se que um atributo privado só pode ser acessado por código escrito na própria classe do atributo.

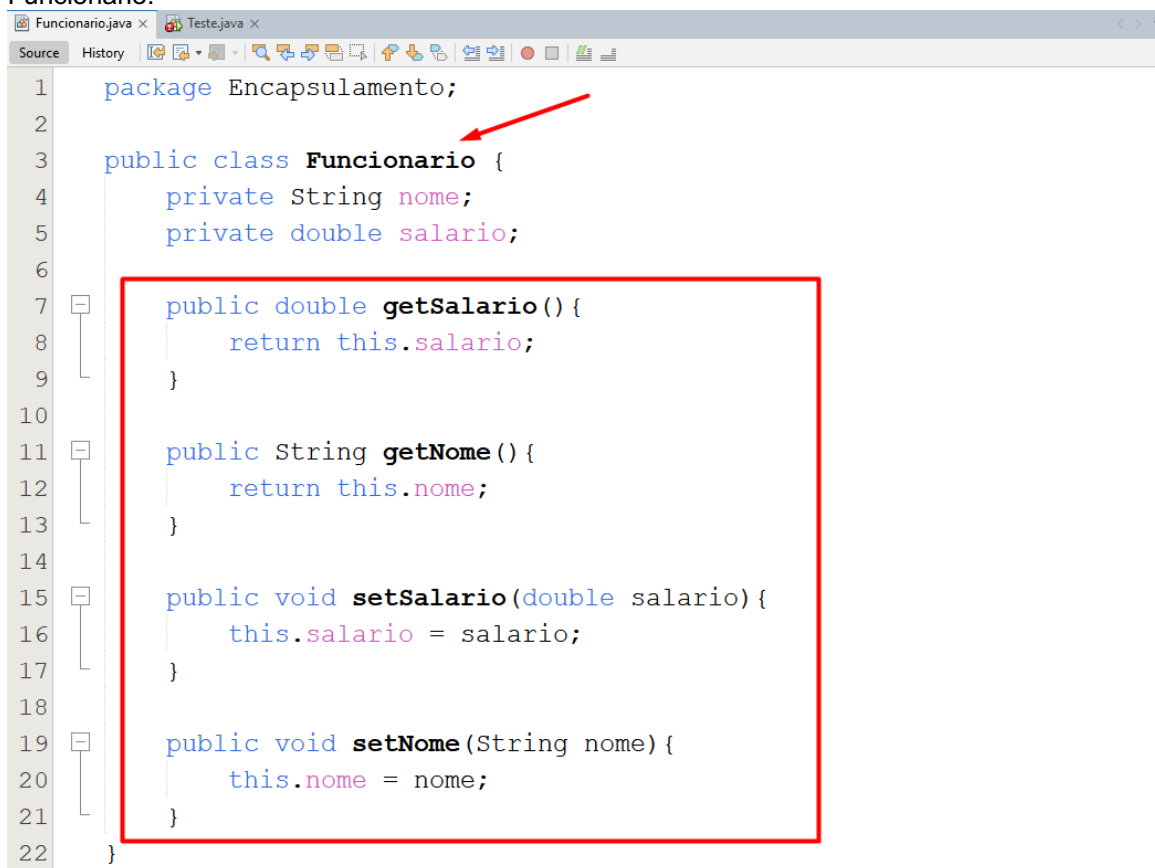


```

1 package Encapsulamento;
2
3 public class Teste {
4
5     public static void main(String[] args) {
6         Funcionario f = new Funcionario();
7
8         f.nome = "Rafael Consentino"; //SEM ENCAPSULAMENTO
9         f.salario = 2000; //SEM ENCAPSULAMENTO
10
11         System.out.println(f.nome); //SEM ENCAPSULAMENTO
12         System.out.println(f.salario); //SEM ENCAPSULAMENTO
13
14     }
15 }

```

7-) Crie métodos de acesso com nomes padronizados para os atributos definidos na classe Funcionario.



```

1 package Encapsulamento;
2
3 public class Funcionario {
4     private String nome;
5     private double salario;
6
7     public double getSalario() {
8         return this.salario;
9     }
10
11     public String getNome() {
12         return this.nome;
13     }
14
15     public void setSalario(double salario) {
16         this.salario = salario;
17     }
18
19     public void setNome(String nome) {
20         this.nome = nome;
21     }
22 }

```

8-) Altere a classe Teste para que ela utilize os métodos de acesso ao invés de manipular os atributos do objeto da classe Funcionario diretamente.

```

1 package Encapsulamento;
2
3 public class Teste {
4
5     public static void main(String[] args) {
6         Funcionario f = new Funcionario();
7
8         //f.nome = "Rafael Consentino"; //SEM ENCAPSULAMENTO
9         f.setNome("Rafael Consentino"); //COM ENCAPSULAMENTO
10
11        //f.salario = 2000; //SEM ENCAPSULAMENTO
12        f.setSalario(20000); //COM ENCAPSULAMENTO
13
14        //System.out.println(f.nome); //SEM ENCAPSULAMENTO
15        System.out.println(f.getNome()); //COM ENCAPSULAMENTO
16
17        //System.out.println(f.salario); //SEM ENCAPSULAMENTO
18        System.out.println(f.getSalario()); //COM ENCAPSULAMENTO
19    }
20 }

```

Compile e execute o teste!

30. HERANÇA

30.1. Reutilização de Código

Um banco oferece diversos serviços que podem ser contratados individualmente pelos clientes. Quando um serviço é contratado, o sistema do banco deve registrar quem foi o cliente que contratou o serviço, quem foi o funcionário responsável pelo atendimento ao cliente e a data de contratação.

Com o intuito de ser produtivo, a modelagem dos serviços do banco deve diminuir a repetição de código. A ideia é reaproveitar o máximo do código já criado. Essa ideia está diretamente relacionada ao conceito *Don't Repeat Yourself*. Em outras palavras, devemos minimizar ao máximo a utilização do "copiar e colar". O aumento da produtividade e a diminuição do custo de manutenção são as principais motivações do DRY.

Em seguida, vamos discutir algumas modelagens possíveis para os serviços do banco.

Buscaremos seguir a ideia do DRY na criação dessas modelagens.

30.2. Uma classe para todos os serviços

Poderíamos definir apenas uma classe para modelar todos os tipos de serviços que o banco oferece.

```

1 public class Servico {
2     private String nomeServico;
3     private int codigoServico;
4     private String dataDeContratacao;
5
6
7
8 }

```

30.3. Empréstimo

O empréstimo é um dos serviços que o banco oferece. Quando um cliente contrata esse serviço, são definidos o valor e a taxa de juros mensal do empréstimo. Devemos acrescentar dois atributos na classe `Servico`: um para o valor e outro para a taxa de juros do serviço de empréstimo.

```
3 public class Emprestimo extends Servico {  
4  
5     private double valor;  
6     private double taxa;  
7  
8 }
```

30.4. Seguro de veículos

Outro serviço oferecido pelo banco é o seguro de veículos. Para esse serviço devem ser definidas as seguintes informações: veículo segurado, valor do seguro e a franquia. Devemos adicionar três atributos na classe `Servico`.

```
3 public class SeguroDeVeiculo extends Servico {  
4  
5     private double valorDoSeguroDeVeiculo;  
6     private double franquia;  
7  
8 }
```

Apesar de seguir a ideia do DRY, modelar todos os serviços com apenas uma classe pode dificultar o desenvolvimento. Supondo que dois ou mais desenvolvedores são responsáveis pela implementação dos serviços, eles provavelmente modificariam a mesma classe concorrentemente. Além disso, os desenvolvedores, principalmente os recém chegados no projeto do banco, ficariam confusos com o código extenso da classe `Servico`.

Outro problema é que um objeto da classe `Servico` possui atributos para todos os serviços que o banco oferece. Na verdade, ele deveria possuir apenas os atributos relacionados a um serviço. Do ponto de vista de performance, essa abordagem causaria um consumo desnecessário de memória.

30.5. Uma classe para cada serviço

Para modelar melhor os serviços, evitando uma quantidade grande de atributos e métodos desnecessários, criaremos uma classe para cada serviço.

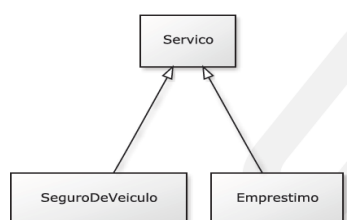
Criar uma classe para cada serviço torna o sistema mais flexível, pois qualquer alteração em um determinado serviço não causará efeitos colaterais nos outros. Mas, por outro lado, essas classes teriam bastante código repetido, contrariando a ideia do DRY. Além disso, qualquer alteração que deva ser realizada em todos os serviços precisa ser implementada em cada uma das classes.

30.6. Uma classe genérica e várias específicas

Na modelagem dos serviços do banco, podemos aplicar um conceito de orientação a objetos chamado Herança. A ideia é reutilizar o código de uma determinada classe em outras classes.

Aplicando herança, teríamos a classe `Servico` com os atributos e métodos que todos os serviços devem ter e uma classe para cada serviço com os atributos e métodos específicos do determinado serviço.

As classes específicas seriam “ligadas” de alguma forma à classe `Servico` para reaproveitar o código nela definido. Esse relacionamento entre as classes é representado em UML pelo diagrama a seguir.



Os objetos das classes específicas Emprestimo e SeguroDeVeiculo possuiriam tanto os atributos e métodos definidos nessas classes quanto os definidos na classe Servico.

As classes específicas são vinculadas a classe genérica utilizando o comando extends. Não é necessário redefinir o conteúdo já declarado na classe genérica.

```

32  public class Servico {
31      private String nomeServico;
30      private int codigoServico;
29      private String dataDeContratacao;
28
27      public String getNomeServico() {
26          return nomeServico;
25      }
24
23      public void setNomeServico(String nomeServico) {
22          this.nomeServico = nomeServico;
21      }
20
19      public int getCodigoServico() {
18          return codigoServico;
17      }
16
15      public void setCodigoServico(int codigoServico) {
14          this.codigoServico = codigoServico;
13      }
12
11      public String getDataDeContratacao() {
10          return dataDeContratacao;
9      }
8
7      public void setDataDeContratacao(String dataDeContratacao) {
6          this.dataDeContratacao = dataDeContratacao;
5      }
4
3  }

```

```

25
24      public class Emprestimo extends Servico {
23
22          private double valor;
21          private double taxa;
20
19          public double getValor() {
18              return valor;
17          }
16
15          public void setValor(double valor) {
14              this.valor = valor;
13          }
12
11          public double getTaxa() {
10              return taxa;
9          }
8
7          public void setTaxa(double taxa) {
6              this.taxa = taxa;
5          }
4
3  }

```



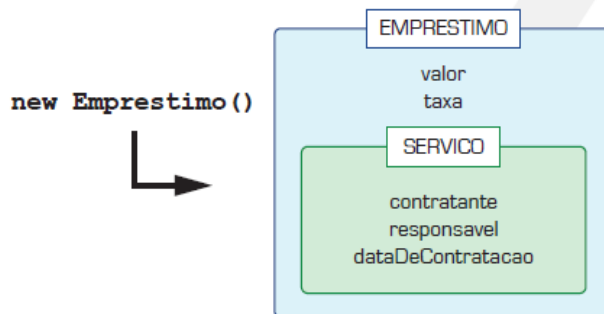
```

3 public class SeguroDeVeiculo extends Servico {
4
5     private double valorDoSeguroDeVeiculo;
6     private double franquia;
7
8     public double getValorDoSeguroDeVeiculo() {
9         return valorDoSeguroDeVeiculo;
10    }
11
12    public void setValorDoSeguroDeVeiculo(double valorDoSeguroDeVeiculo) {
13        this.valorDoSeguroDeVeiculo = valorDoSeguroDeVeiculo;
14    }
15
16    public double getFranquia() {
17        return franquia;
18    }
19
20    public void setFranquia(double franquia) {
21        this.franquia = franquia;
22    }
23
24 }

```

A classe genérica é denominada super classe, classe base ou classe mãe. As classes específicas são denominadas sub classes, classes derivadas ou classes filhas.

Quando o operador new é aplicado em uma sub classe, o objeto construído possuirá os atributos e métodos definidos na sub classe e na super classe.



```

3 public class testeHeranca {
4
5     public static void main(String[] args) {
6         SeguroDeVeiculo sv = new SeguroDeVeiculo();
7
8         sv.setNomeServico("Seguro de veículo");
9         sv.setValorDoSeguroDeVeiculo(125.99);
10
11        System.out.println(sv.getNomeServico());
12        System.out.println(sv.getValorDoSeguroDeVeiculo());
13
14        Emprestimo e = new Emprestimo();
15        e.setNomeServico("Empréstimo");
16        e.setValor(1250);
17        e.setTaxa(0.1);
18
19        System.out.println(e.getNomeServico());
20        System.out.println(e.getValor());
21        System.out.println(e.getTaxa());
22
23    }
24 }

```

```
Output x
Debugger Console x OrientacaoAObjetos (run) x
run:
Seguro de veículo
125.99
Empréstimo
1250.0
0.1
BUILD SUCCESSFUL (total time: 0 seconds)
```

Preço Fixo

Suponha que todo serviço do banco possui uma taxa administrativa que deve ser paga pelo cliente que contratar o serviço. Inicialmente, vamos considerar que o valor dessa taxa é igual para todos os serviços do banco.

Neste caso, poderíamos implementar um método na classe `Servico` para calcular o valor da taxa.

Este método será reaproveitado por todas as classes que herdam da classe `Servico`.

```
28 public void setDataDeContratacao(String dataDeContratacao) {
29     this.dataDeContratacao = dataDeContratacao;
30 }
31
32 public double calculaTaxa(){
33     return 10;
34 }
35
36 }
37
```

```
20
21 System.out.println(e.getNomeServico());
22 System.out.println(e.getValor());
23 System.out.println(e.getTaxa());
24
25 System.out.println("Taxa Seguro:" + sv.calculaTaxa());
26 System.out.println("Taxa Empréstimo:" + e.calculaTaxa());
27
28 }
29 }
```

```
Debugger Console x OrientacaoAObjetos (run) x
run:
Seguro de veículo
125.99
12/03/2026
Empréstimo
1250.0
0.1
Taxa Seguro:10.0
Taxa Empréstimo:10.0
BUILD SUCCESSFUL (total time: 0 seconds)
```

31. Reescrita de Método

Suponha que o valor da taxa administrativa do serviço de empréstimo é diferente dos outros serviços, pois ele é calculado a partir do valor emprestado ao cliente. Como esta lógica é específica para o serviço de empréstimo, devemos acrescentar um método para implementar esse cálculo na classe `Emprestimo`.

```

3 public class Emprestimo extends Servico {
4
5     private double valor;
6     private double taxa;
7
8     public double getValor() {
9         return valor;
10    }
11
12    public void setValor(double valor) {
13        this.valor = valor;
14    }
15
16    public double getTaxa() {
17        return taxa;
18    }
19
20    public void setTaxa(double taxa) {
21        this.taxa = taxa;
22    }
23
24    public double calculaTaxaDeEmprestimo(){
25        return this.valor * 0.1;
26    }
27 }

```

Os métodos das classes específicas têm prioridade sobre os métodos das classes genéricas. Em outras palavras, se o método chamado existe na classe filha ele será chamado, caso contrário o método será procurado na classe mãe.

Quando definimos um método com a mesma assinatura na classe base e em alguma classe derivada, estamos aplicando o conceito de Reescrita de Método.

Fixo + Específico

Suponha que o preço de um serviço é a soma de um valor fixo mais um valor que depende do tipo do serviço. Por exemplo, o preço do serviço de empréstimo é 5 reais mais uma porcentagem do valor emprestado ao cliente. O preço do serviço de seguro de veículo é 5 reais mais uma porcentagem do valor do veículo segurado. Em cada classe específica, podemos reescrever o método calculaTaxa().

```

3 public class Emprestimo extends Servico {
4
5     private double valor;
6     private double taxa;
7
8     public double getValor() {
9         return valor;
10    }
11
12    public void setValor(double valor) {
13        this.valor = valor;
14    }
15
16    public double getTaxa() {
17        return taxa;
18    }
19
20    public void setTaxa(double taxa) {
21        this.taxa = taxa;
22    }
23
24    @Override
25    public double calculaTaxa(){
26        return 5 + this.valor * 0.1;
27    }
28 }

```



```

3 public class SeguroDeVeiculo extends Servico {
4
5     private double valorDoSeguroDeVeiculo;
6     private double franquia;
7
8     public double getValorDoSeguroDeVeiculo() {
9         return valorDoSeguroDeVeiculo;
10    }
11
12    public void setValorDoSeguroDeVeiculo(double valorDoSeguroDeVeiculo) {
13        this.valorDoSeguroDeVeiculo = valorDoSeguroDeVeiculo;
14    }
15
16    public double getFranquia() {
17        return franquia;
18    }
19
20    public void setFranquia(double franquia) {
21        this.franquia = franquia;
22    }
23
24    @Override
25    public double calculaTaxa(){
26        return 5 + this.valorDoSeguroDeVeiculo * 0.1;
27    }
28 }

```

Se o valor fixo dos serviços for atualizado, todas as classes específicas devem ser modificadas. Outra alternativa seria criar um método na classe Servico para calcular o valor fixo de todos os serviços e chamá-lo dos métodos reescritos nas classes específicas.

```

3 public class Servico {
4     private String nomeServico;
5     private int codigoServico;
6     private String dataDeContratacao;
7
8     public String getNomeServico() {
9         return nomeServico;
10    }
11
12    public void setNomeServico(String nomeServico) {
13        this.nomeServico = nomeServico;
14    }
15
16    public int getCodigoServico() {
17        return codigoServico;
18    }
19
20    public void setCodigoServico(int codigoServico) {
21        this.codigoServico = codigoServico;
22    }
23
24    public String getDataDeContratacao() {
25        return dataDeContratacao;
26    }
27
28    public void setDataDeContratacao(String dataDeContratacao) {
29        this.dataDeContratacao = dataDeContratacao;
30    }
31
32    public double calculaTaxa(){
33        return 10;
34    }
35 }

```

```

3 public class Emprestimo extends Servico {
4
5     private double valor;
6     private double taxa;
7
8     public double getValor() {
9         return valor;
10    }
11
12    public void setValor(double valor) {
13        this.valor = valor;
14    }
15
16    public double getTaxa() {
17        return taxa;
18    }
19
20    public void setTaxa(double taxa) {
21        this.taxa = taxa;
22    }
23
24    @Override
25    public double calculaTaxa(){
26        return super.calculaTaxa() + this.valor * 0.1;
27    }
28 }

```

Dessa forma, quando o valor padrão do preço dos serviços é alterado, basta modificar o método na classe Servico.

31.1. Método Construtores

Um método construtor em Java é um método especial usado para inicializar um objeto quando ele é criado. Ele é executado automaticamente no momento em que utilizamos a palavra-chave new para criar um objeto de uma classe.

Em outras palavras, o construtor serve para definir os valores iniciais dos atributos de um objeto.

Características do Construtor:

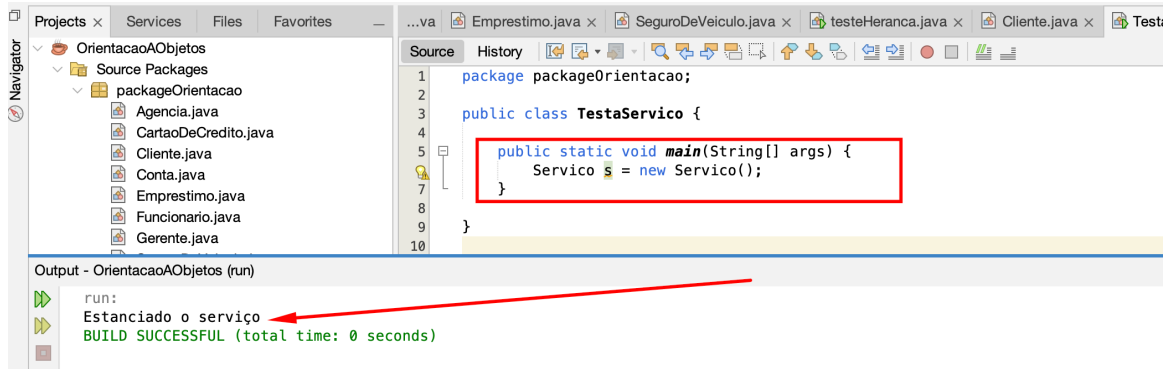
- Tem o mesmo nome da classe.
- Não possui tipo de retorno (nem void).
- É chamado automaticamente quando um objeto é criado.
- Pode receber parâmetros para inicializar os atributos.
- Uma classe pode ter vários construtores (sobrecarga).

```

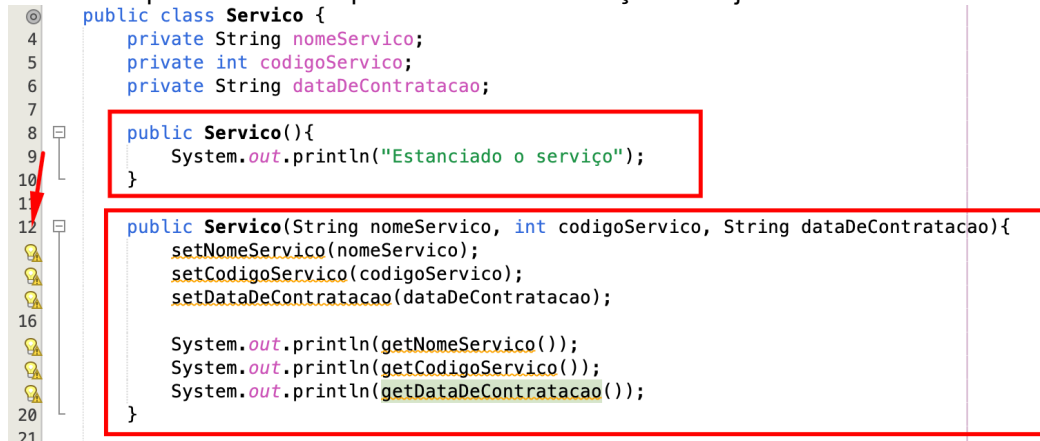
1 public class Servico {
2     private String nomeServico;
3     private int codigoServico;
4     private String dataDeContratacao;
5
6     public Servico(){
7         System.out.println("Estanciado o serviço");
8     }
9
10    public String getNomeServico() {
11        return nomeServico;
12    }
13
14    public void setNomeServico(String nomeServico) {
15        this.nomeServico = nomeServico;
16    }
17 }
18
19

```

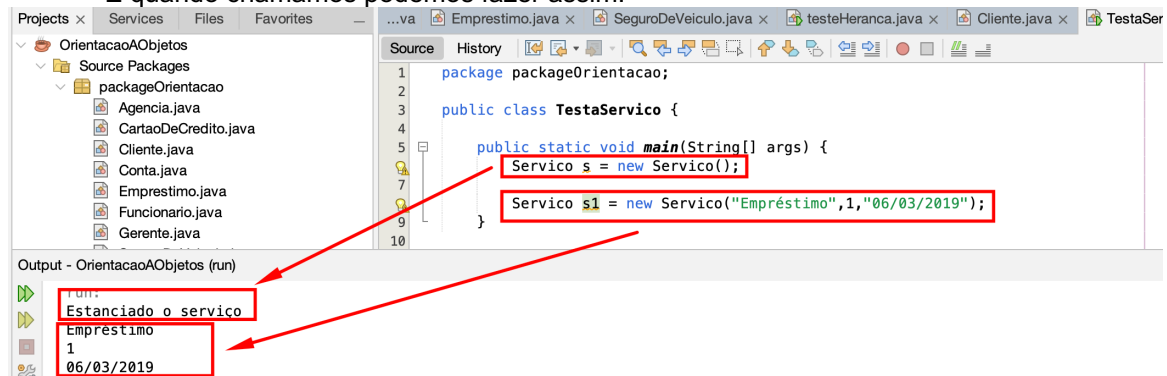
Para exemplo, se fizermos a instância do objeto do tipo Serviço veremos que executará o método construtor:



Outro exemplo, vai aparecer uma observação que é perigosa, rs! Mas como é um exemplo para vocês entenderem o construtor é válido, mas o aviso é: Aviso: "Overridable method call in constructor". Esse aviso significa que um construtor está chamando um método que pode ser sobrescrito (override) por uma subclasse. Isso é considerado uma prática perigosa, pois pode causar comportamentos inesperados durante a criação do objeto.



E quando chamamos podemos fazer assim:



31.2. Exercícios de Fixação

- 1) Crie um projeto chamado Heranca2.
- 2) Defina uma classe para modelar os funcionários do banco. Sabendo que todo funcionário possui nome e salário, inclua os *getters* e *setters* dos atributos.

```

1  package heranca3;
2
3  public class Funcionario {
4      private String nome;
5      private double salario;
6
7      public String getNome() {
8          return nome;
9      }
10
11     public void setNome(String nome) {
12         this.nome = nome;
13     }
14
15     public double getSalario() {
16         return salario;
17     }
18
19     public void setSalario(double salario) {
20         this.salario = salario;
21     }
22 }

```

3) Crie uma classe para cada tipo específico de funcionário herdando da classe Funcionario. Considere apenas três tipos específicos de funcionários: gerentes, telefonistas e secretarias. Os gerentes possuem um nome de usuário e uma senha para acessar o sistema do banco. As telefonistas possuem um código de estação de trabalho. As secretarias possuem um número de ramal.

```

1  package heranca3;
2
3  public class Gerente extends Funcionario{
4      private String usuario;
5      private String senha;
6
7      public String getUsuario() {
8          return usuario;
9      }
10
11     public void setUsuario(String usuario) {
12         this.usuario = usuario;
13     }
14
15     public String getSenha() {
16         return senha;
17     }
18
19     public void setSenha(String senha) {
20         this.senha = senha;
21     }
22
23 }

```

```

1 package heranca3;
2
3 public class Telefonista extends Funcionario{
4     private int estacaoDeTrabalho;
5
6     public int getEstacaoDeTrabalho() {
7         return estacaoDeTrabalho;
8     }
9
10    public void setEstacaoDeTrabalho(int estacaoDeTrabalho) {
11        this.estacaoDeTrabalho = estacaoDeTrabalho;
12    }
13 }

```

```

1 package heranca3;
2
3 public class Secretaria extends Funcionario{
4     private int ramal;
5
6     public int getRamal() {
7         return ramal;
8     }
9
10    public void setRamal(int ramal) {
11        this.ramal = ramal;
12    }
13 }

```

4) Teste o funcionamento dos três tipos de funcionários criando um objeto de cada uma das classes: Gerente, Telefonista e Secretaria.

```

1 package heranca3;
2
3 public class TestaFuncionarios {
4
5     public static void main(String[] args) {
6         Gerente g = new Gerente();
7         //Classe Funcionario (Mãe)
8         g.setNome("Marcos Costa");
9         g.setSalario(32450.22);
10
11         //Classe Filha (Funcionario)
12         g.setUsuario("marcão");
13         g.setSenha("batatinha");
14
15         Telefonista t = new Telefonista();
16         //Classe Telefonista (Mãe)
17         t.setNome("Eliane Prado");
18         t.setSalario(10780.23);
19
20         //Classe Filha (Telefonista)
21         t.setEstacaoDeTrabalho(12);
22
23         Secretaria s = new Secretaria();

```

```

24      //Classe Funcionario (Mãe)
25      s.setNome("Nilva Prado");
26      s.setSalario(6550.99);
27
28      //Classe Filha (Secretaria)
29      s.setRamal(6677);
30
31      //Mostrar os dados Gerente
32      System.out.println("=====");
33      System.out.println("GERENTE:");
34      System.out.println("Nome: " + g.getNome());
35      System.out.println("Salário: " + g.getSalario());
36      System.out.println("Usuario: " + g.getUsuario());
37      System.out.println("Senha: " + g.getSenha());
38
39      //Mostrar os dados Telefonista
40      System.out.println("=====");
41      System.out.println("TELEFONISTA:");
42      System.out.println("Nome: " + t.getNome());
43      System.out.println("Salário: " + t.getSalario());
44      System.out.println("Estação de Trabalho: " + t.getEstacaoDeTrabalho());
45
46      //Mostrar os dados Secretaria
47      System.out.println("=====");
48      System.out.println("SECRETARIA:");
49      System.out.println("Nome: " + s.getNome());
50      System.out.println("Salário: " + s.getSalario());
51      System.out.println("Ramal: " + s.getRamal());
52  }
53  }

```

Execute o teste!

5) Suponha que todos os funcionários recebam uma bonificação de 10% do salário. Acrescente um método na classe Funcionario para calcular essa bonificação.

```

1  package heranca3;
2
3  @
4  public class Funcionario {
5      private String nome;
6      private double salario;
7
8      public String getNome() {
9          return nome;
10     }
11
12     public void setNome(String nome) {
13         this.nome = nome;
14     }
15
16     public double getSalario() {
17         return salario;
18     }
19
20     public void setSalario(double salario) {
21         this.salario = salario;
22     }
23 }

```

```
24 public double calculaBonificacao() {  
25     return this.salario * 0.1;  
26 }
```

6) Altere a classe TestaFuncionarios para imprimir a bonificação de cada funcionário, além dos dados que já foram impressos. Depois, execute o teste novamente.

```
1 package heranca3;  
2  
3 public class TestaFuncionarios {  
4  
5     public static void main(String[] args) {  
6         Gerente g = new Gerente();  
7         //Classe Funcionario (Mãe)  
8         g.setNome("Marcos Costa");  
9         g.setSalario(32450.22);  
10  
11         //Classe Filha (Funcionario)  
12         g.setUsuario("marcão");  
13         g.setSenha("batatinha");  
14  
15         Telefonista t = new Telefonista();  
16         //Classe Telefonista (Mãe)  
17         t.setNome("Eliane Prado");  
18         t.setSalario(10780.23);  
19  
20         //Classe Filha (Telefonista)  
21         t.setEstacaoDeTrabalho(12);  
22  
23         Secretaria s = new Secretaria();  
24         //Classe Funcionario (Mãe)  
25         s.setNome("Nilva Prado");  
26         s.setSalario(6550.99);  
27  
28         //Classe Filha (Secretaria)  
29         s.setRamal(6677);  
30  
31         //Mostrar os dados Gerente  
32         System.out.println("=====");  
33         System.out.println("GERENTE:");  
34         System.out.println("Nome: " + g.getNome());  
35         System.out.println("Salário: " + g.getSalario());  
36         System.out.println("Usuario: " + g.getUsuario());  
37         System.out.println("Senha: " + g.getSenha());  
38         System.out.println("Bonificação: " + g.calculaBonificacao());  
39  
40         //Mostrar os dados Telefonista  
41         System.out.println("=====");  
42         System.out.println("TELEFONISTA:");  
43         System.out.println("Nome: " + t.getNome());  
44         System.out.println("Salário: " + t.getSalario());  
45         System.out.println("Estação de Trabalho: " + t.getEstacaoDeTrabalho());  
46         System.out.println("Bonificação: " + t.calculaBonificacao());  
47  
48         //Mostrar os dados Secretaria  
49         System.out.println("=====");  
50         System.out.println("SECRETARIA:");
```



```

51     System.out.println("Nome: " + s.getNome());
52     System.out.println("Salário: " + s.getSalario());
53     System.out.println("Ramal: " + s.getRamal());
54     System.out.println("Bonificação: " + s.calculaBonificacao());
55 }
56 }

```

7) Suponha que os gerentes recebam uma bonificação maior que os outros funcionários. Reescreva o método `calculaBonificacao()` na classe `Gerente`. Depois, compile e execute o teste novamente.

```

1  package heranca3;
2
3  public class Gerente extends Funcionario{
4      private String usuario;
5      private String senha;
6
7      public String getUsuario() {
8          return usuario;
9      }
10
11     public void setUsuario(String usuario) {
12         this.usuario = usuario;
13     }
14
15     public String getSenha() {
16         return senha;
17     }
18
19     public void setSenha(String senha) {
20         this.senha = senha;
21     }
22
23     @Override
24     public double calculaBonificacao() {
25         return this.getSalario() * 0.6 + 100;
26     }
27 }

```

31.3. Exercícios Complementares

1) Defina na classe `Funcionario` um método para imprimir na tela o nome, salário e bonificação dos funcionários.

2) Reescreva o método que imprime os dados dos funcionários nas classes `Gerente`, `Telefonista` e `Secretaria` para acrescentar a impressão dos dados específicos de cada tipo de funcionário.

3) Modifique a classe `TestaFuncionarios` para utilizar o método `mostraDados()`.

31.4. Correção dos Exercícios Complementares

Primeiramente vamos criar o método de impressão na classe de funcionário.

```

Funcionario.java x
Source History
16     return salario;
17 }
18
19 public void setSalario(double salario) {
20     this.salario = salario;
21 }

```

```

22 public double calculaBonificacao() {
23     return this.salario * 0.1;
24 }
25
26
27 public void mostrarDados() {
28     System.out.println("Nome: " + getNome());
29     System.out.println("Salário: " + getSalario());
30     System.out.println("Bonificação: " + calculaBonificacao());
31 }
32 }

```

Agora vamos trabalhar a classe Gerente.

```

Gerente.java x
Source History
23 @Override
24 public double calculaBonificacao() {
25     return this.getSalario() * 0.6 + 100;
26 }
27
28 @Override
29 public void mostrarDados() {
30     super.mostrarDados();
31     System.out.println("Usuario: " + getUsuario());
32     System.out.println("Senha: " + getSenha());
33     System.out.println("Bonificação: " + calculaBonificacao());
34 }
35 }

```

Agora vamos trabalhar a classe Secretaria

```

Secretaria.java x
Source History
8 }
9
10 public void setRamal(int ramal) {
11     this.ramal = ramal;
12 }
13
14 @Override
15 public void mostrarDados() {
16     super.mostrarDados();
17     System.out.println("Ramal: " + getRamal());
18     System.out.println("Bonificação: " + calculaBonificacao());
19 }
20 }

```

Agora vamos trabalhar a classe Telefonista

```

Telefonista.java x
Source History
8 }
9
10 public void setEstacaoDeTrabalho(int estacaoDeTrabalho) {
11     this.estacaoDeTrabalho = estacaoDeTrabalho;
12 }
13
14 @Override
15 public void mostrarDados() {
16     super.mostrarDados();
17     System.out.println("Estação de trabalho: " + getEstacaoDeTrabalho());
18     System.out.println("Bonificação: " + calculaBonificacao());
19 }
20 }

```

Modificando agora o TestaFuncionarios, que ficará menor.

```

31 //Mostrar os dados Gerente
32 System.out.println("=====");
33 System.out.println("GERENTE:");
34 g.mostrarDados();
35
36 //Mostrar os dados Telefonista
37 System.out.println("=====");
38 System.out.println("TELEFONISTA:");
39 t.mostrarDados();
40
41 //Mostrar os dados Secretaria
42 System.out.println("=====");
43 System.out.println("SECRETARIA:");
44 s.mostrarDados();
45
46
47 }
48 }

```

Teremos este resultado:

```

Output - Heranca3 (run) x
run:
=====
GERENTE:
Nome: Marcos Costa
Salário: 32450.22
Bonificação: 19570.132
Usuário: marc
Senha: batatinha
Bonificação: 19570.132
=====
TELEFONISTA:
Nome: Eliane Prado
Salário: 10780.23
Bonificação: 1078.023
Estado de trabalho: 12
Bonificação: 1078.023
=====
SECRETARIA:
Nome: Nilva Prado
Salário: 6550.99
Bonificação: 655.099
Ramal: 6677
Bonificação: 655.099
BUILD SUCCESSFUL (total time: 0 seconds)

```

31.5. Exercício final nossa P1.

Implementem um sistema (livre escolha) com os conceitos abordados em aula (classe, atributo – visibilidade, método, encapsulamento, herança, *getters* e *setters*, sobrecarga de método e reescrita de método), realizar de forma individual.

Vocês devem subir o projeto desenvolvido ao seu GitHub até o dia 02/04/2026, o restante da aula de hoje e a próxima será exclusivamente para esta entrega que será nossa P1.

Como estamos com problemas no Microsoft Teams, vocês me devem mandar no chat do Teams o link do GitHub com a solução de vocês implementada.

32. Prática do começo ao final

Nesse momento iremos implementar um projeto de login simples, onde abordaremos a Orientação a Objetos com banco de dados e tela, tudo isso confeccionado em Java, e fazendo um *CRUD* (*Create, Read, Update and Delete*) em uma aplicação.

32.1. Banco de dados

Como sempre disse, nosso sistema precisa começar pelo levantamento de requisitos e posteriormente, antes de programarmos qualquer coisa, precisamos implementar nosso banco de dados, iremos utilizar o MySQL, e iremos fazer somente uma tabela, que será o usuário de nosso sistema. Vejam abaixo:

```
1  #CRIAÇÃO DO BANCO DE DADOS BD_LOGIN
2  •  create database bd_login;
3
4  #SELECIONANDO O BANCO DE DADOS
5  •  use bd_login;
6
7  #CRIANDO A TABELA DE LOGIN
8  •  create table usuario(
9      usuario varchar(10) primary key,
10     nome    varchar(30) default '',
11     senha   char(32) default ''
12 );
```

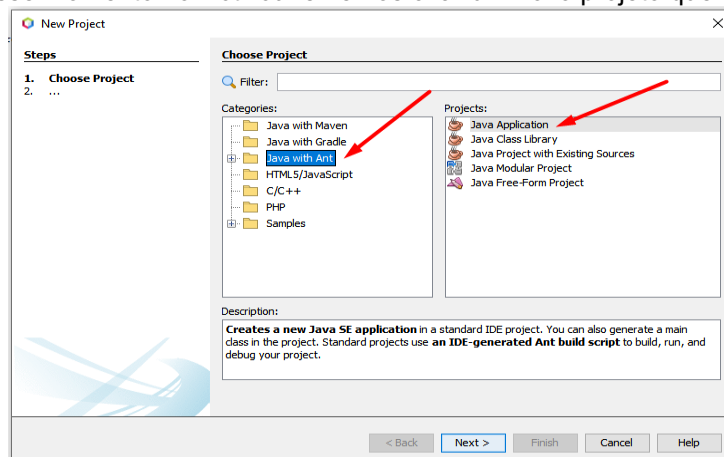
Vocês viram que usamos para inserir o usuário em nossa tabela uma função MD5, correto? Mas o que ela significa?

O MD5 é um dos algoritmos de criptografia mais conhecidos que funciona através de chamada de função como fizemos acima, e seu funcionamento consiste na geração de um valor hexadecimal de 32 dígitos a partir de uma *String*, por isso que criamos o campo senha com tamanho de 32. Esse processo, no entanto, é unidirecional, ou seja, uma vez aplicada, essa função criptográfica não pode ser revertida a fim de recuperar o valor original, ou seja, se o usuário esquecer essa senha, nem nós como administradores do banco de dados conseguiremos passar a senha para o usuário, o que força fazer o *reset* da mesma.

Isso nos ajuda nos quesitos de segurança (principalmente para salvar dados sensíveis como a senha do usuário), já que mesmo que os dados criptografados sejam hackeados, o invasor não terá como descriptografá-los. Para aplicar esse algoritmo no MySQL, dispomos da função MD5, que recebe como único parâmetro o texto a ser criptografado e retorna o valor mascarado. Assim, podemos utilizá-la diretamente ao inserir um registro na tabela, como fizemos anteriormente.

32.2. Projeto Login

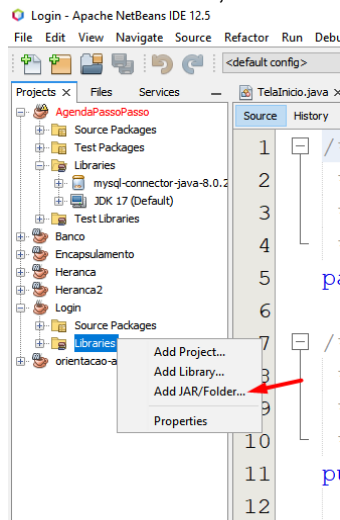
Nesse momento no *NetBeans* iremos criar um novo projeto que chamaremos de Login.



32.3. Conectando nossa aplicação ao banco de dados

Nesse momento precisamos anexar uma biblioteca do MySQL ao nosso projeto para que aconteça a integração entre banco de dados e projeto.

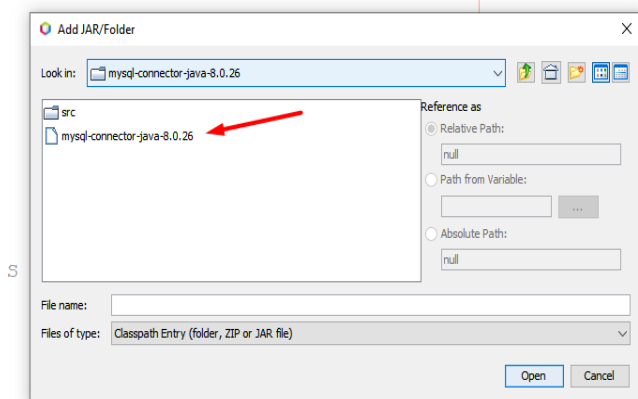
Para fazermos tal, vamos olhar a pasta *Libraries* (biblioteca) dentro de nosso projeto:



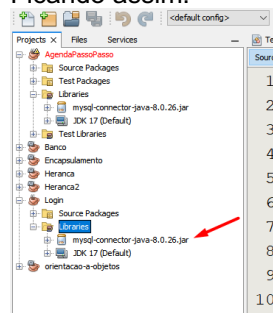
Selecionamos **mysql-connector-java**, se vocês não tiverem esse arquivo eu disponibilizo o mesmo, **e coloquem este arquivo na mesma pasta do seu projeto**.

Este arquivo é um JAR (*Java ARchive*) é um arquivo compactado no formato ZIP que contém um conjunto de classes (arquivos “.class”) e arquivos de configuração. O formato JAR é utilizado para permitir a distribuição de bibliotecas ou, até mesmo, sistemas inteiros em Java (uma vez que é possível montar arquivos JAR formados por centenas de classes).

As principais bibliotecas Java desenvolvidas por terceiros – ou seja, bibliotecas que não fazem parte da JDK – são estruturadas e distribuídas em arquivos JAR.



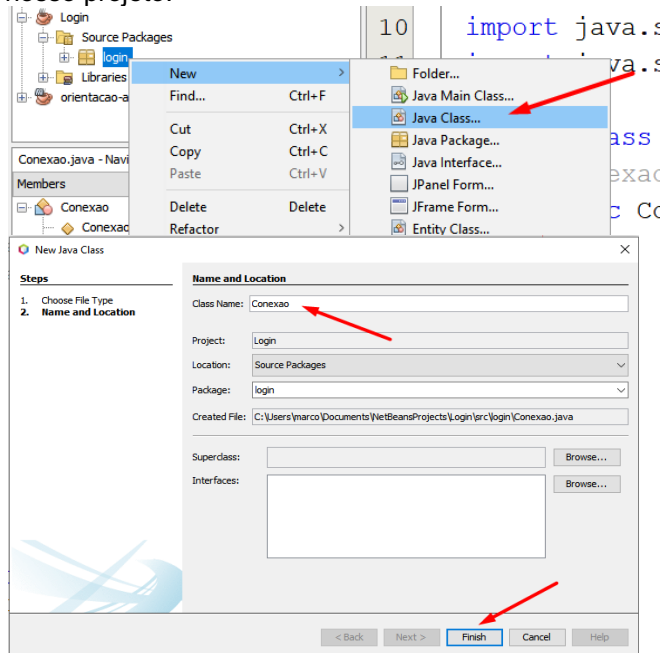
Ficando assim:



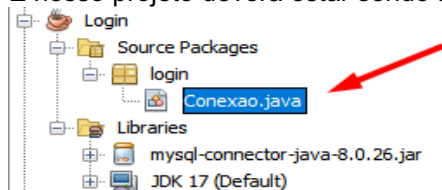
Pronto, nosso projeto já possui a biblioteca MySQL para fazermos a conexão com um banco de dados.

32.3.1. Classe Conexao

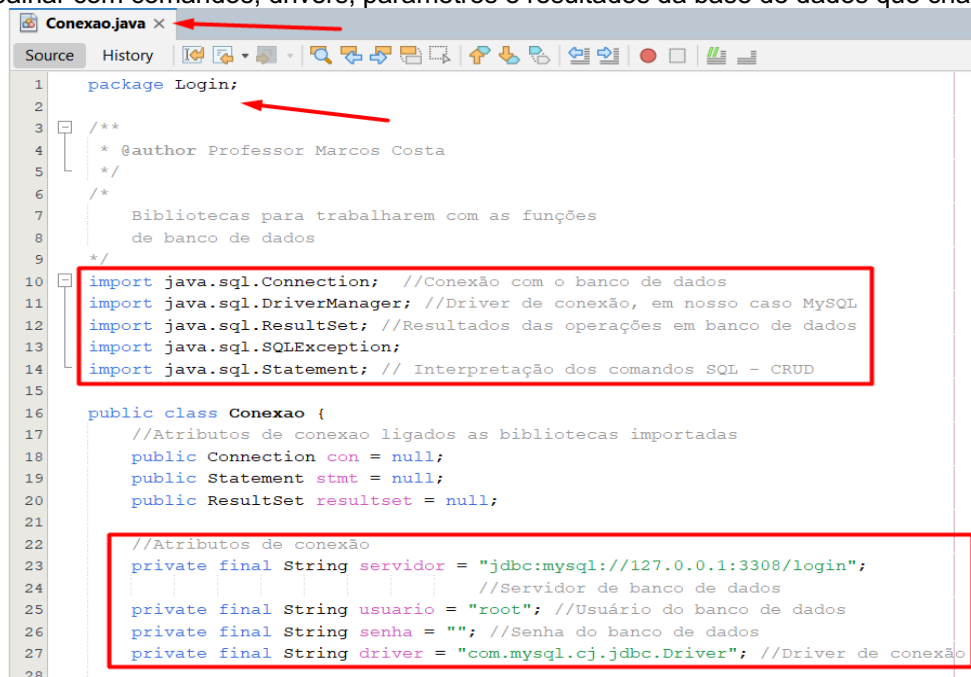
Agora iremos codificar nossa classe de conexão, para isso vamos adicionar uma nova classe em nosso projeto.



E nosso projeto deverá estar sendo apresentado assim:



Iremos agora codificar a classe Conexao.java, nela faremos a importação de bibliotecas (conceito que vimos em DS I) de SQL (Banco de Dados) essas bibliotecas nos auxiliarão a trabalhar com comandos, drivers, parâmetros e resultados da base de dados que criamos, vejamos:



```

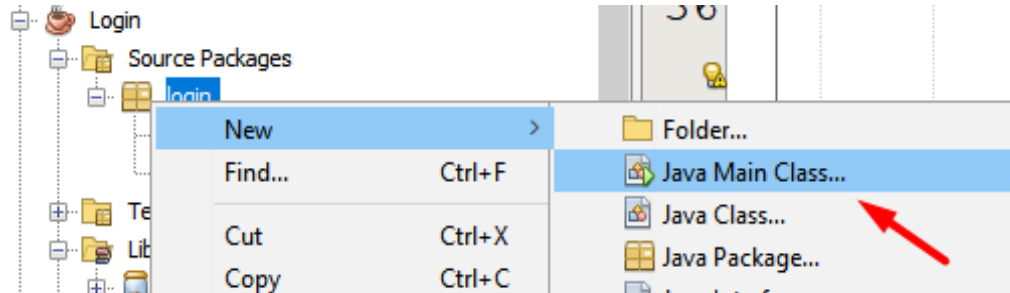
29 //Método de abertura da conexão com banco de dados
30 public Connection abrirConexao() {
31     try {
32         Class.forName(driver); //Driver de utilização
33         //Atributos de conexão
34         con = DriverManager.getConnection(servidor, usuario, senha);
35         stmt = con.createStatement(); //Fazendo conexão com o banco de dados
36         //Mensagem informando que a conexão foi realizada com sucesso
37         System.out.println("Conexão aberta com sucesso");
38     } catch ( ClassNotFoundException | SQLException e) {
39         //Mensagem de saída caso haja erro
40         System.out.println("Erro ao acessar banco de dados, verifique! " + e.getMessage());
41     }
42     return con; //Retorna a conexão
43 }
44
45 //Método de fechamento da conexão com o banco de dados
46 public void fecharConexao() {
47     try {
48         con.close(); //Fechamento da conexão com o banco de dados
49         //Mensagem que fechou a conexão
50         System.out.println("Conexão finalizada com sucesso");
51     } catch (SQLException e) {
52         //Mensagem de erro ao fechar conexão
53         System.out.println("Erro ao encerrar conexão " + e.getMessage());
54     }
55 }
56 }

```

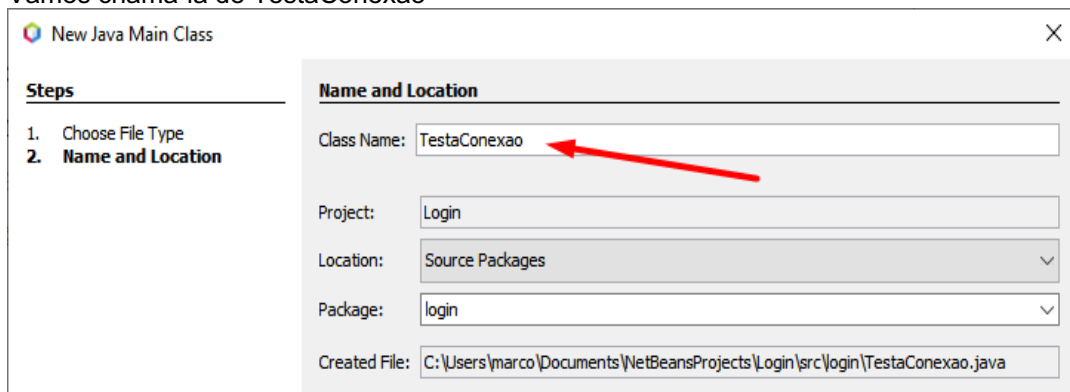
Um bloco **try** é chamado de bloco protegido porque, caso ocorra algum problema com os comandos dentro do bloco, a execução desviará para os blocos **catch** correspondentes, conforme fizemos acima, ou seja, se der algum erro nesse trecho de código, ele “escapa” para o **catch** e mostra o erro.

Muito bem, assim finalizamos nossa classe de conexão para banco de dados, vamos testar? Eu já respondo, é claro!!!

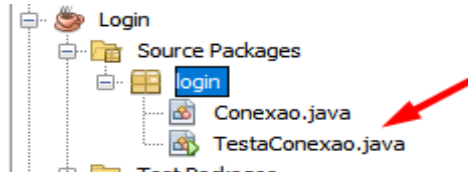
Para isso vamos criar uma classe *Main*:



Vamos chamá-la de TestaConexao



Ficando assim:



Agora vamos codificar a mesma fazendo o código abaixo:

```
TestaConexao.java x
Source History
4  /*
5  public class TestaConexao {
6      /*
7      Teste para verificarmos se o Banco conecta e desconecta
8      */
9      public static void main(String[] args) {
10         Conexao c = new Conexao(); // Instanciamos o objeto c de Conexao
11         c.abrirConexao(); //Chamamos o método de abertura da conexão do BD.
12
13         try {
14             Thread.sleep(4000); //Fazemos uma pausa de 4 segundos
15             c.fecharConexao(); //Fechamos a conexão
16         } catch (InterruptedException ex) {
17             //Mensagem de saída caso haja erro
18             System.out.println("Houve algum problema no teste de conexão. " + ex.getMessage());
19         }
20     }
21 }
```

Feito isso, podemos executar essa classe, a mesma fará a abertura da conexão com o banco de dados, aguardará 4 segundos e fechará a mesma, veja o resultado abaixo:

```
Output - Login (run) x
run:
Conexão aberta com sucesso
Conexão finalizada com sucesso
BUILD SUCCESSFUL (total time: 4 seconds)
```

32.3.2. Método construtor e Toolkits

Antes de continuarmos vamos ver algumas definições importantes no Java, vamos ver um conceito sobre Método Construtor.

Em Java, o método construtor é definido como um método cujo nome deve ser o mesmo nome da classe e sem indicação do tipo de retorno -- nem mesmo *void*. O construtor é unicamente invocado no momento da criação do objeto através do operador *new*. O retorno do operador *new* é uma referência para o objeto recém criado.

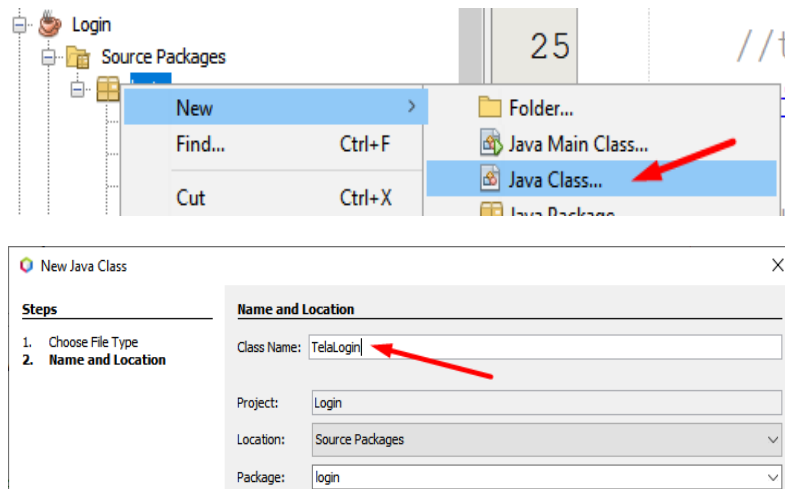
Outros dois elementos importantes a aprender é o AWT e o SWING, eles são *toolkits* (kits de ferramentas) que torna possível a criação de ambientes gráficos (janelas do Windows) no Java, quando ouvimos falar em GUI (*Graphical User Interface*).

AWT significa *Abstract Window Toolkit*. É uma **API** dependente de plataforma para desenvolver GUI ou aplicativos baseados em janela em Java. Foi desenvolvido pela *Sun Microsystems* em 1995. É de uso pesado porque é gerado pelo sistema operacional host do sistema. Ele contém um grande número de classes e métodos, que são usados para criar e gerenciar **GUI**.

Já o *Swing* é uma interface gráfica de usuário (GUI) Java leve que é usada para criar vários aplicativos. O *Swing* possui componentes que são independentes de plataforma. Ele permite ao usuário criar botões e barras de rolagem. O *Swing* inclui pacotes para a criação de aplicativos de desktop em Java. Os componentes do Swing são escritos na linguagem Java. É uma parte do *Java Foundation Classes* (JFC).

32.3.3. Classe TelaLogin

Agora iremos implementar uma classe que fará o desenho da tela de nosso projeto, vamos inserir uma nova classe, a TelaLogin.java.



Vamos codificar essa classe.

```

TelaLogin.java x
Source History
1 package Login;
2 /**
3  *
4  * @author Professor Marcos Costa
5  */
6 /**
7  Importações de todas as bibliotecas que estou utilizando na classe
8  */
9 import java.awt.Font; //Trabalhar com fontes
10 import java.awt.SystemColor; //Trabalhar com cores
11 import javax.swing.JButton; //Trabalhar com botões
12 import javax.swing.JFrame; //Trabalhar com frames
13 import javax.swing.JLabel; //Trabalhar com labels
14 import javax.swing.JPanel; //Trabalhar com painéis
15 import javax.swing.JPasswordField; //Trabalhar com campos de senha
16 import javax.swing.JTextField; //Trabalhar com campos de texto
17
18 public class TelaLogin extends JFrame{
19
20     //tela Objeto JPanel (tela em si)
21     private final JPanel panelTela;
22
23     //txtusuario Objeto JTextField (campo na tela)
24     private final JTextField txtUsuario;
25
26     //pswSenha Objeto PasswordField (campo na tela)
27     private final JPasswordField pswSenha;
28
29     //Método construtor
30     public TelaLogin() {
31         //coloca o objeto na referencia do centro da tela
32         setLocationRelativeTo(null);
33
34         //não permite que o objeto seja expandido
35         setResizable(false);

```

```
36
37 //coloca título na caixa JFrame
38 setTitle("Login - Senac");
39
40 //Quando clicado no X eu encerro todo o programa
41 setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
42
43 //define posionamento e tamanho
44 //      x      y width height
45 setBounds(500, 200, 426, 212);
46
47 //Crio um objeto JPanel e atribuo ele a variável tela
48 panelTela = new JPanel();
49
50 //Define a cor de fundo do JPanel (tela)
51 panelTela.setBackground(SystemColor.gray);
52 setContentPane(panelTela);
53
54 //Vou utilizar o meu panel sem utilizar o padrão
55 panelTela.setLayout(null);
56
57 //Adicionando elementos na tela:
58 // Criando um objeto do tipo JLabel e atribuindo o valor ao atributo
59 JLabel lblIdentificacao = new JLabel("IDENTIFICAÇÃO");
60
61 //Localização na tela
62 lblIdentificacao.setBounds(144, 0, 160, 39);
63
64 //Definindo a Fonte
65 lblIdentificacao.setFont(new Font("Arial", 3, 19));
66
67 //Adicionando o meu label ao meu Panel
68 panelTela.add(lblIdentificacao);
69
70 //Identificação e Posicionamento dos labels
71 JLabel lblUsuario = new JLabel("Usuario");
72 lblUsuario.setBounds(24, 65, 70, 15);
73 panelTela.add(lblUsuario);
74
75 JLabel lblSenha = new JLabel("Senha");
76 lblSenha.setBounds(24, 92, 70, 15);
77 panelTela.add(lblSenha);
78
79 //Identificação e Posicionamento dos texts fields
80 txtUsuario = new JTextField();
81 txtUsuario.setBounds(112, 63, 219, 19);
82 panelTela.add(txtUsuario);
83 txtUsuario.setColumns(10);
84
85 pswSenha = new JPasswordField();
86 pswSenha.setBounds(112, 90, 219, 19);
87 panelTela.add(pswSenha);
88
89 //Identificação e Posicionamento dos botões
90 JButton btnEntrar = new JButton("Entrar");
91 btnEntrar.setBounds(200, 136, 117, 25);
92 panelTela.add(btnEntrar);
93
94 JButton btnCadastrar = new JButton("Cadastrar");
95 btnCadastrar.setBounds(50, 136, 117, 25);
96 panelTela.add(btnCadastrar);
97 }
```

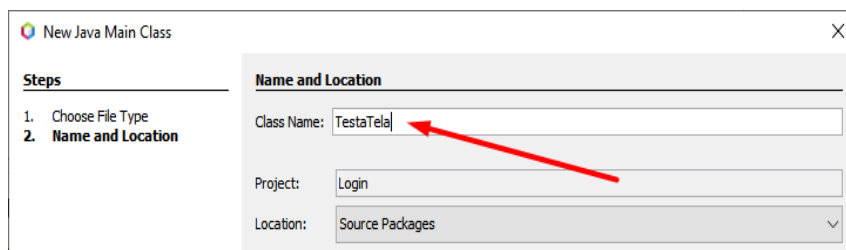
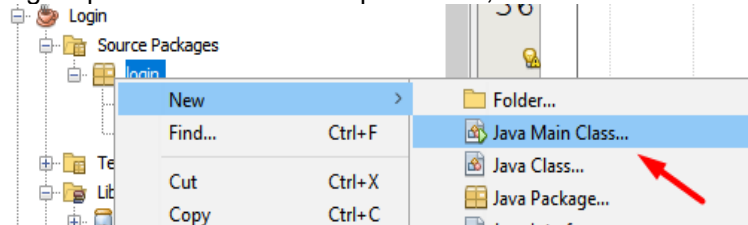
```

98
99     public void abreTela() {
100         TelaLogin tela = new TelaLogin();
101         tela.setVisible(true);
102     }

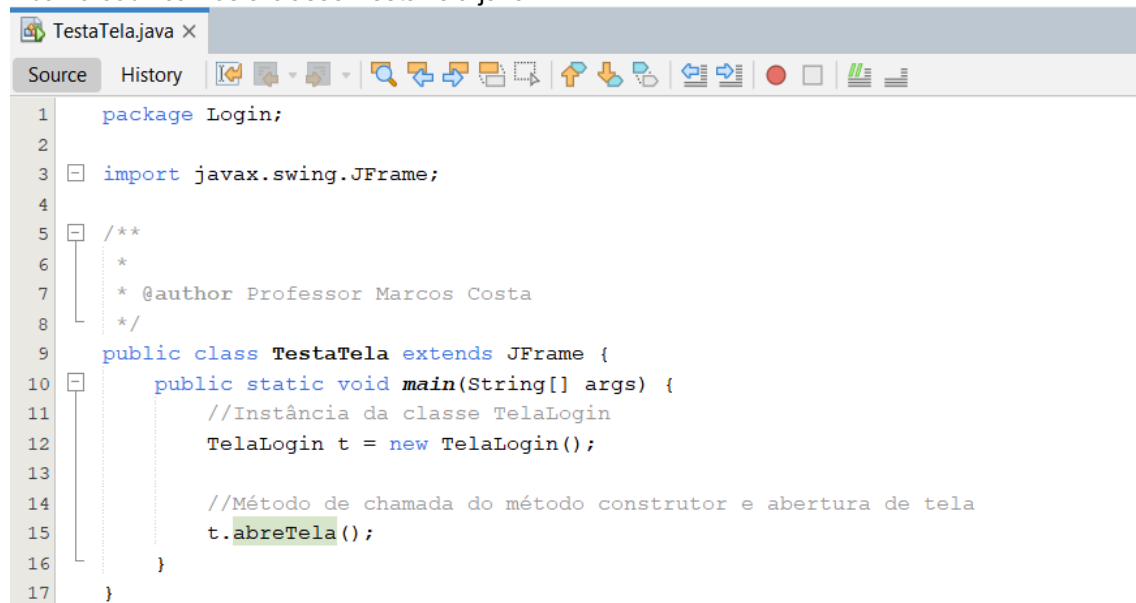
```

32.3.3.1. Classe TestaTela

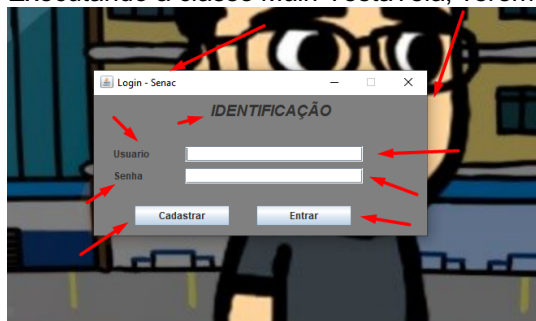
Agora para testarmos a tela aparecendo, vamos construir uma classe *Main*, assim:



Abaixo codificamos a classe TestaTela.java:

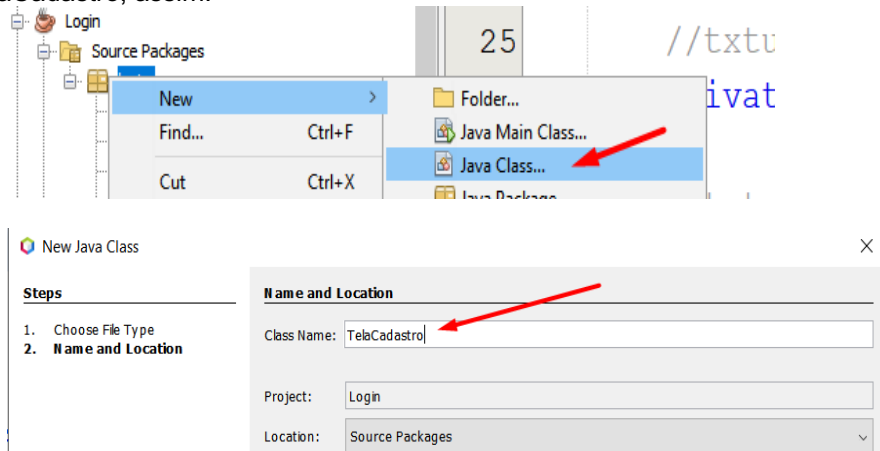


Executando a classe *Main* TestaTela, veremos:



32.3.2. Classe TelaCadastro

Nesse momento iremos criar a tela de cadastro, para isso vamos construir uma classe TelaCadastro, assim:



Abaixo codificamos a classe TelaCadastro.java:

```

1  package Login;
2
3  /**
4   *
5   * @author Professor Marcos Costa
6   */
7  /**
8   * Importações de todas as bibliotecas que estou utilizando na classe
9   */
10 import java.awt.Font;
11 import java.awt.SystemColor;
12 import javax.swing.JButton;
13 import javax.swing.JFrame;
14 import javax.swing.JPanel;
15 import javax.swing.JLabel;
16 import javax.swing.JPasswordField;
17 import javax.swing.JTextField;
18
19 public class TelaCadastro extends JFrame{
20     //Declaração dos atributos de tela
21     private final JPanel tela;
22     private final JTextField txtNome;
23     private final JTextField txtUsuario;
24     private final JPasswordField passSenha;
25     private final JPasswordField passConfSenha;
26
27     //Método construtor da classe
28     public TelaCadastro() {
29         setLocationRelativeTo(null);
30         setResizable(false);
31         setTitle("Cadastro");
32         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
33         setBounds(500, 200, 426, 230);
34
35         tela = new JPanel();
36         tela.setBackground(SystemColor.gray);

```

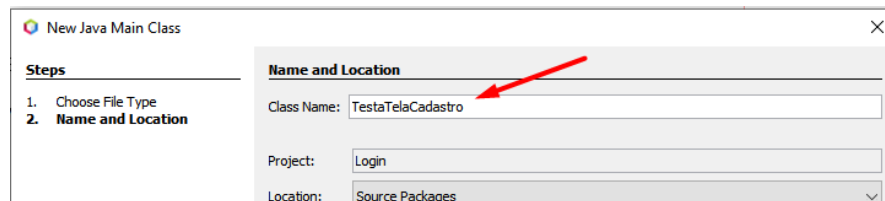
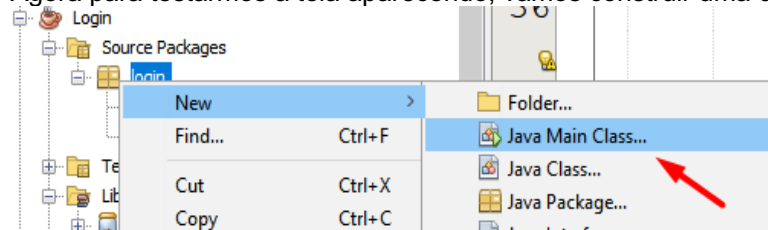
```

37         setContentPane(tela);
38         tela.setLayout(null);
39
40         //Adicionando elementos na tela:
41         JLabel lblIdentificacao = new JLabel("Informar campos para cadastro");
42         lblIdentificacao.setBounds(60, 0, 500, 39);
43         lblIdentificacao.setFont(new Font("Arial", 3, 19));
44         tela.add(lblIdentificacao); //adiciono o meu label ao meu Panel
45
46         JLabel lblNome = new JLabel("Nome");
47         lblNome.setBounds(24, 50, 70, 15);
48         tela.add(lblNome);
49
50         txtNome = new JTextField();
51         txtNome.setBounds(120, 50, 219, 19);
52         tela.add(txtNome);
53         txtNome.setColumns(10);
54
55         JLabel lblUsuario = new JLabel("Usuario");
56         lblUsuario.setBounds(24, 75, 70, 15);
57         tela.add(lblUsuario);
58
59         txtUsuario = new JTextField();
60         txtUsuario.setBounds(120, 75, 219, 19);
61         tela.add(txtUsuario);
62         txtUsuario.setColumns(10);
63
64         JLabel lblsenha = new JLabel("Senha");
65         lblsenha.setBounds(24, 100, 70, 15);
66         tela.add(lblsenha);
67
68         passSenha = new JPasswordField();
69         passSenha.setBounds(120, 100, 219, 19);
70         tela.add(passSenha);
71
72         JLabel lblconfsenha = new JLabel("Confirmar Senha");
73         lblconfsenha.setBounds(24, 125, 100, 15);
74         tela.add(lblconfsenha);
75
76         passConfSenha = new JPasswordField();
77         passConfSenha.setBounds(120, 125, 219, 19);
78         tela.add(passConfSenha);
79
80         JButton btnCadastrar = new JButton("Cadastrar");
81         btnCadastrar.setBounds(200, 156, 117, 25);
82         tela.add(btnCadastrar);
83
84         JButton btnCancelar = new JButton("Cancelar");
85         btnCancelar.setBounds(50, 156, 117, 25);
86         tela.add(btnCancelar);
87
88     }
89
90     public void abreTela() {
91         TelaCadastro panelCadastro = new TelaCadastro();
92         panelCadastro.setVisible(true);
93     }
94 }

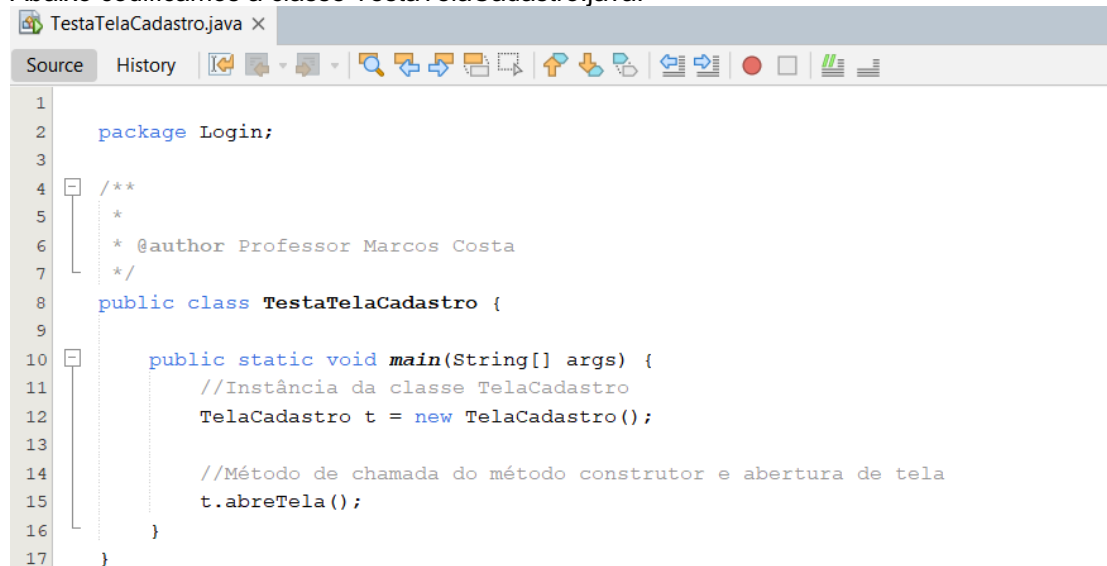
```

32.3.3. Classe TestaTelaCadastro

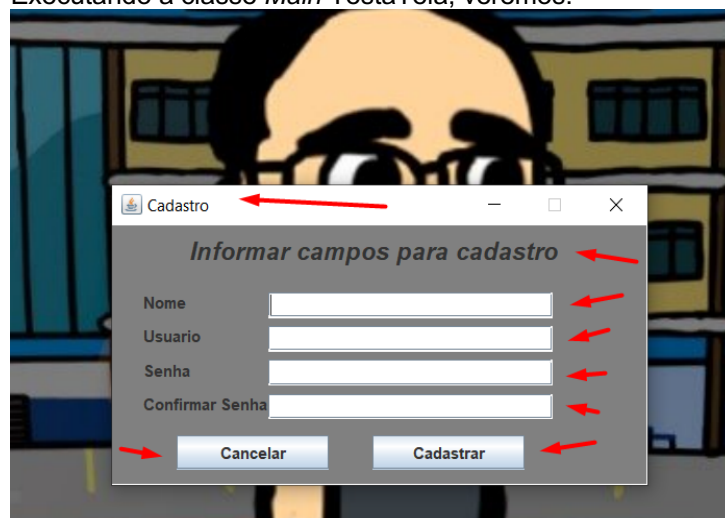
Agora para testarmos a tela aparecendo, vamos construir uma classe *Main*, assim:



Abaixo codificamos a classe TestaTelaCadastro.java:

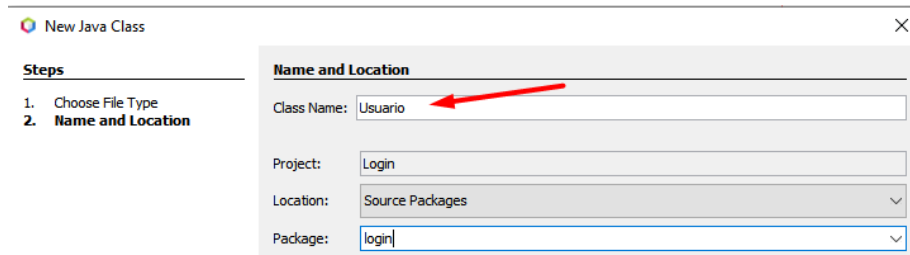
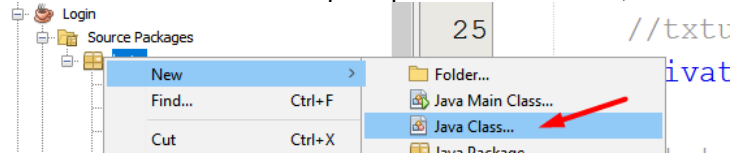


Executando a classe *Main* TestaTela, veremos:

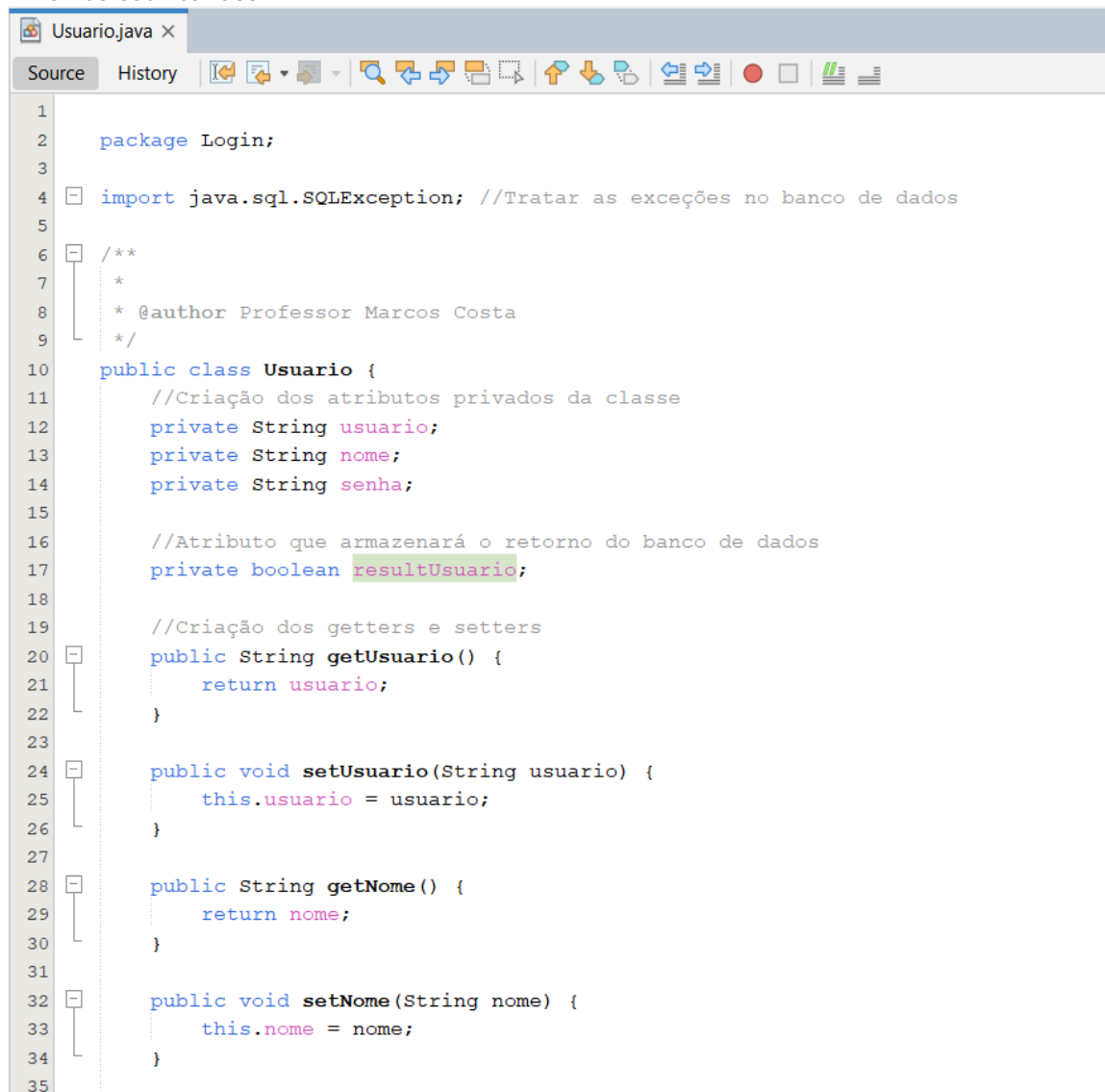


32.3.4. Classe de Usuário

Como o usuário é o nosso principal foco no sistema, vamos criar a classe Usuário, assim:



E iremos codificar assim:



```

36 public String getSenha() {
37     return senha;
38 }
39
40 public void setSenha(String senha) {
41     this.senha = senha;
42 }
43
44 //Método de verificação da autenticidade do usuario
45 public boolean verificaUsuario(String usuario, String senha){
46     //Fazer a instância da conexão com o banco de dados
47     Conexao banco = new Conexao();
48
49     try{
50         //Abro a conexão com o banco de dados
51         banco.abrirConexao();
52
53         //Criando parâmetro de retorno
54         banco.stmt= banco.con.createStatement();
55
56         //Executando a consulta no banco de dados
57         banco.resultset =
58             banco.stmt.executeQuery("SELECT * FROM usuario "
59                                     + "WHERE usuario = '" + usuario + "'"
60                                     + " AND senha = md5('" + senha + "')");
61
62         //Verificando se existe retorno de dados no banco
63         if (banco.resultset.next()) {
64             //Caso tenha
65             resultUsuario = true;
66         }else{
67             //Caso não tenha
68             resultUsuario = false;
69         }
70
71         banco.fecharConexao(); // fecha nossa conexão com o banco de dados
72
73     }catch (SQLException ec) {
74         System.out.println("Erro ao consultar usuário " + ec.getMessage());
75     }
76
77     return resultUsuario;
78 }
79 }

```

32.3.4.1. Colocando validações na tela de Login no botão Entrar

Para melhorarmos mais nosso projeto iremos programar o botão entrar, mas antes precisamos pensar em validar os campos Usuário e Senha, ou seja, só iremos validar a entrada se os campos estiverem preenchidos.

Iremos implementar mais algumas linhas de código na classe **TelaLogin**, iremos acrescentar a importação da classe **JOptionPane**, mas o que ela faz?

JOptionPane é uma classe que possibilita a criação de uma caixa de diálogo padrão que ou solicita um valor para o usuário ou retorna uma informação. Abaixo encontra-se alguns métodos e parâmetros mais utilizados quando se opta pelo **JOptionPane**.

Métodos

Método	Descrição
showConfirmDialog	Solicita uma confirmação como(YES, NO, CANCEL)
showInputDialog	Solicita algum valor
showMessageDialog	Informa ao usuário sobre algo

showOptionDialog	Unificação dos três acima
------------------	---------------------------

Parâmetros

Parâmetros	Descrição
parentComponent	Define a caixa de diálogo onde irá aparecer todo o conteúdo. Há duas maneiras de definir a caixa de diálogo a primeira você mesmo cria utilizando os conceitos da classe JFrame. A segunda, você define esse parâmetro como null e o Java irá gerar uma caixa de diálogo padrão.
message	É a mensagem que o usuário deve ler. Esta mensagem pode ser uma simples String ou um conjunto de objetos.
messageType	Define o estilo da mensagem. Ele pode expor a caixa de diálogo de formas diferentes, dependendo deste valor, pode fornecer um ícone padrão. Exemplos: ERROR_MESSAGE INFORMATION_MESSAGE WARNING_MESSAGE QUESTION_MESSAGE PLAIN_MESSAGE
optionType	Define o conjunto de botões que irá aparecer na parte inferior da caixa de diálogo. Exemplos: DEFAULT_OPTION YES_NO_OPTION YES_NO_CANCEL_OPTION OK_CANCEL_OPTION

Exemplos:

1. Mostra um diálogo de erro que exibe a mensagem “alerta”:

```

1
2 JOptionPane.showMessageDialog(null, “alerta”,
3     “alerta”, JOptionPane.ERROR_MESSAGE);
4
5

```



2. Mostra um painel de informação com as opções Sim/Não e exibe a mensagem: ‘Escolha um:’

```

1
2 JOptionPane.showConfirmDialog(null, “Escolha um:”,
3     “Escolha um”, JOptionPane.YES_NO_OPTION);
4

```



3. Mostrar uma janela de aviso com as opções OK, CANCELAR, o texto 'Aviso' no título e a mensagem 'Clique em OK para continuar':

```
1
2     Object[] options = { "OK", "CANCELAR" };
3     JOptionPane.showMessageDialog(null, "Clique OK para continuar", "Aviso",
4         JOptionPane.DEFAULT_OPTION, JOptionPane.WARNING_MESSAGE,
5         null, options, options[0]);
6
```



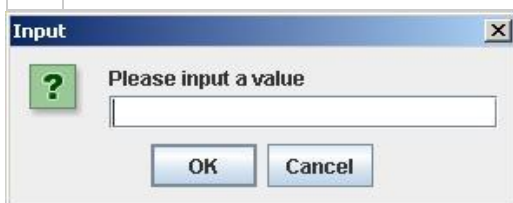
Note que se eu adicionar mais um item no vetor options, automaticamente adicionar mais um botão de opção com o nome que eu colocar.

```
1
2     Object[] options = { "OK", "CANCELAR", "VOLTAR" };
3     JOptionPane.showMessageDialog(null, "Clique OK para continuar", "Aviso",
4         JOptionPane.DEFAULT_OPTION, JOptionPane.WARNING_MESSAGE,
5         null, options, options[0]);
6
```



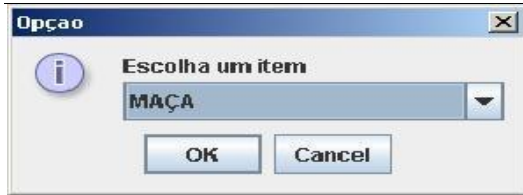
4. Mostra uma caixa de diálogo solicitando que o usuário digite uma string:

```
1
2     String inputValue = JOptionPane.showInputDialog("Please input a value");
3
4
```



5. Mostra uma caixa de diálogo solicitando que o usuário selecione uma item:

```
1
2     Object[] itens = { "MAÇA", "PERA", "BANANA" };
3     Object selectedValue = JOptionPane.showInputDialog(null,
4         "Escolha um item", "Opção",
5         JOptionPane.INFORMATION_MESSAGE, null,
6         itens, itens [0]); //
7
8
```



No código ficará assim, teremos que importar mais duas bibliotecas a **ActionEvent** para trabalharmos com eventos, em nosso caso, será o click do botão, e o **JOptionPane**, que tratamos anteriormente:

```

TelaLogin.java x
Source History
1 package Login;
2 /**
3  *
4  * @author Professor Marcos Costa
5  */
6 /**
7  Importações de todas as bibliotecas que estou utilizando na classe
8  */
9 import java.awt.Font; //Trabalhar com fontes
10 import java.awt.SystemColor; //Trabalhar com cores
11 import java.awt.event.ActionEvent; //Trabalhar com evento
12 import javax.swing.JButton; //Trabalhar com botões
13 import javax.swing.JFrame; //Trabalhar com frames
14 import javax.swing.JLabel; //Trabalhar com labels
15 import javax.swing.JOptionPane; //Trabalhar com mensagens
16 import javax.swing.JPanel; //Trabalhar com painéis
17 import javax.swing.JPasswordField; //Trabalhar com campos de senha
18 import javax.swing.JTextField; //Trabalhar com campos de texto
19

```

Acrescentem também um atributo para validar o retorno do usuário cadastrado no banco de dados.

```

31 //validar se o usuario é correto
32 private boolean usuarioValido;

```

Após isso iremos implementar a ação no botão Entrar, **btnEntrar**, conforme abaixo, se atentem a partir da linha 104, vejam:

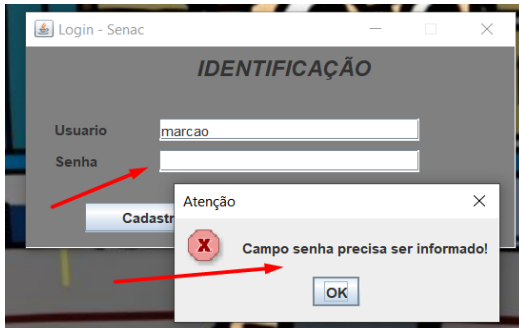
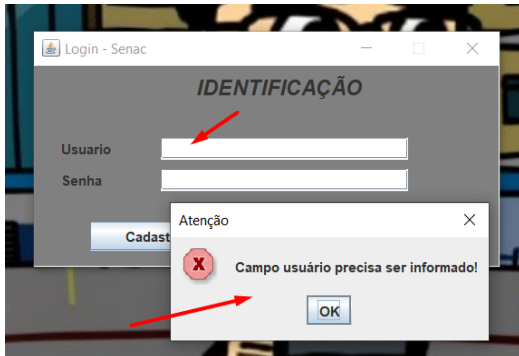
```

TelaLogin.java x
Source History
99      JButton btnCadastrar = new JButton("Cadastrar");
100     btnCadastrar.setBounds(50, 136, 117, 25);
101     panelTela.add(btnCadastrar);
102
103     //Ação no botão de entrar no sistema
104     btnEntrar.addActionListener((ActionEvent e) -> {
105         //Instancio a classe usuario
106         Usuario usu = new Usuario();
107
108         //Utilizo o setter de usuario e senha
109         usu.setUsuario(txtUsuario.getText());
110         usu.setSenha(pswSenha.getText());
111
112         if ("".equals(txtUsuario.getText())) {
113             //Vamos dar uma mensagem na tela
114             JOptionPane.showMessageDialog(null,
115                 "Campo usuário precisa ser informado!",
116                 "Atenção",
117                 JOptionPane.ERROR_MESSAGE);
118             //Voltar o cursor para o campo txtUsuario
119             txtUsuario.grabFocus();
120         } else if ("".equals(pswSenha.getText())) {
121
122             //Vamos dar uma mensagem na tela
123             JOptionPane.showMessageDialog(null,
124                 "Campo senha precisa ser informado!",
125                 "Atenção",
126                 JOptionPane.ERROR_MESSAGE);
127             //Voltar o cursor para o campo txtUsuario
128             pswSenha.grabFocus();
129         } else {
130             //Verifico se o usuário consta no banco de dados
131             usuarioValido = usu.verificaUsuario(usu.getUsuario(),
132                 usu.getSenha());
133
134             if (usuarioValido == true) {
135                 //Usuario e senha bateram no banco estão corretos
136                 JOptionPane.showMessageDialog(null,
137                     "Usuário válido em nossa base de dados",
138                     "Atenção",
139                     JOptionPane.INFORMATION_MESSAGE);
140
141             } else {
142                 //Usuario e senha bateram no banco estão incorretos
143                 JOptionPane.showMessageDialog(null,
144                     "Usuário inválido ou inexistente",
145                     "Atenção",
146                     JOptionPane.ERROR_MESSAGE);
147
148                 //Método para limpar os texto
149                 limpaText();
150
151                 //Mando o foco para o campo usuário
152                 txtUsuario.grabFocus();
153             }
154         }
    
```

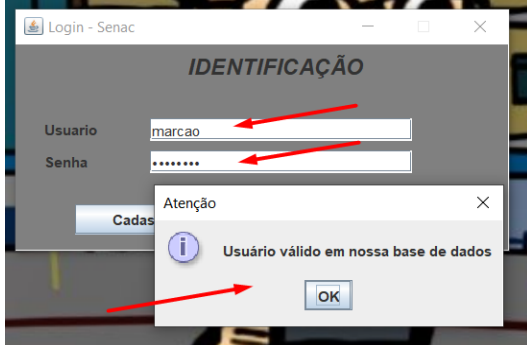
```

154         }
155     });
156 }
157
158 public void abreTela() {
159     TelaLogin tela = new TelaLogin();
160     tela.setVisible(true);
161 }
162
163 public void limpaText() {
164     txtUsuario.setText("");
165     pswSenha.setText("");
166 }
167 }
    
```

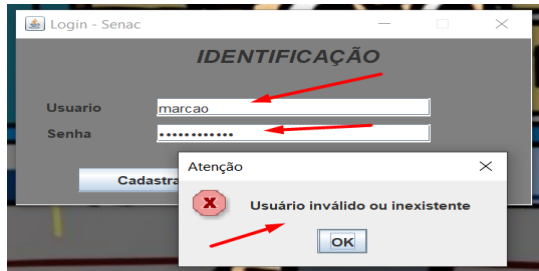
Percebam que os dois retângulos vermelhos representam respectivamente a chamada do método **limpaText**, que servirá para limpar os campos usuário e senha da tela. Feito isso vamos testar as funcionalidades de não passar usuário e senha, vejam:



E caso passemos um usuário e senha válidos:



E caso erremos ou usuário e senha:



22.1.1. Cadastrar o usuário

Agora iremos codificar o cadastro do usuário, ou seja, fazer o **insert** na base de dados, iremos programar o botão cadastrar, mas antes, precisamos pensar na validação dos campos **nome**, **usuário**, **senha** e **confirmar senha**, ou seja, só efetivaremos a entrada dos dados se os mesmos estiverem preenchidos, e se os campos **senha** e **confirmar senha** estiverem com o mesmo conteúdo.

Vamos implementar mais algumas linhas de código na classe TelaCadastro, com os **imports** das bibliotecas **HeadlessException**, **JOptionPane** e **ActionEvent**:

```

8      Importações de todas as bibliotecas que estou utilizando na classe
9      */
10     import java.awt.Font;
11     import java.awt.HeadlessException;
12     import java.awt.SystemColor;
13     import java.awt.event.ActionEvent;
14     import javax.swing.JButton;
15     import javax.swing.JFrame;
16     import javax.swing.JPanel;
17     import javax.swing.JLabel;
18     import javax.swing.JOptionPane;
19     import javax.swing.JPasswordField;
20     import javax.swing.JTextField;

```

Iremos acrescentar os atributos `usuarioValido` e `cadastroValido`, ambos como booleano para verificarmos se o usuário já existe na base de dados para não duplicarmos o mesmo, e para verificarmos a efetivação do cadastro, e também os atributos **mensagemJOption** e **mensagemTipo** para armazenarmos as mensagens e tipo de cada verificação que será realizada, no caso desta última, para uma melhor codificação, ao invés de colocarmos os **JOptionPane.ERROR_MESSAGE** ou **JOptionPane.INFORMATION_MESSAGE**, vamos colocar respectivamente 0 (ERROR) e 1 (INFORMATION).

```

29     //Validações de usuário e cadastro corretos
30     private boolean usuarioValido;
31     private boolean cadastroValido;
32
33     //String de mensagem
34     private String mensagemJOption;
35     private int mensagemTipo = 0;

```

Vamos antes de mais nada, fazer a implementação do botão Cancelar, que deverá retornar para a tela de Login.

```

167     btnCancelar.addActionListener((ActionEvent e) -> {
168         TelaLogin tLogin = new TelaLogin();
169         tLogin.abreTela();
170         dispose();
171     });

```

Vamos implementar mais uma verificação, neste exemplo a quantidade mínima de caracteres para nossa senha, assim modificaremos o `btnCadastrar` acrescentado mais uma verificação no `if else` conforme a seguir:

```

119 } else if ("".equals(usu.getSenha())) {
120     mensagemJOption = "Campo senha precisa ser informado!";
121     mensagemTipo = 0;
122 } else if (!usu.getSenha().equals(passConfSenha.getText())) {
123     mensagemJOption = "Campos senha e confirmação de senha não coincidem!";
124     mensagemTipo = 0;
125 } else if (usu.verificaPadraoSenha(usu.getSenha()) == false){
126     mensagemJOption = "O campo usuário é maior que 10 caracteres!";
127     mensagemTipo = 0;
128 } else {

```

E agora temos que implementar na classe de Usuario o método que traz esta resposta, mas antes faremos a declaração do atributo resultadoPadraoSenha:

```

12 // Atributo que armazenará o retorno do banco de dados
13 private boolean resultUsuario;
14 private boolean resultCadastro;
15
16 private boolean resultadoPadraoSenha;
17

```

E agora implementamos o método:

```

39 public void setSenha(String senha) {
40     this.senha = senha;
41 }
42
43 public boolean verificaPadraoSenha(String senha){
44     //Verificar o tamanho da senha
45     if (senha.length() < 10){
46         resultadoPadraoSenha = false;
47     }else{
48         resultadoPadraoSenha = true;
49     }
50
51     return resultadoPadraoSenha;
52 }
53

```

Agora faremos a implementação do botão cadastrar, onde faremos as validações de preenchimento dos campos e verificando se as senhas estão iguais.

```

100 //Ação do botão cadastrar usuário na base de dados
101 btnCadastrar.addActionListener((ActionEvent e) -> {
102     try{
103         //Instancio o objeto Usuario
104         Usuario usu = new Usuario();
105
106         //Realizando os setters dos dados de tela
107         usu.setNome(txtNome.getText());
108         usu.setUsuario(txtUsuario.getText());
109         usu.setSenha(passSenha.getText());
110
111         //Validações de preenchimento dos dados
112         if ("".equals(usu.getNome())) {
113             mensagemJOption = "Campo nome do usuário precisa ser informado!";
114             mensagemTipo = 0;
115         } else if ("".equals(usu.getUsuario())) {
116             mensagemJOption = "Campo usuário precisa ser informado!";
117         } else if ("".equals(usu.getSenha())) {
118             mensagemJOption = "Campo senha precisa ser informado!";
119             mensagemTipo = 0;
120         } else if (!usu.getSenha().equals(passConfSenha.getText())) {
121             mensagemJOption = "Campos senha e confirmação de senha não coincidem!";
122             mensagemTipo = 0;
123         } else {
124             //Verifico se somente o usuário consta no banco,
125             //neste caso, faremos uma sobrecarga de método
126             usuarioValido = usu.verificaUsuario(usu.getUsuario());
127
128             if (usuarioValido == true) {

```

```

129 //Caso exista, não pode ser colocado na base
130 mensagemJOption = "Usuário já existente na base da dados";
131 mensagemTipo = 0;
132 }else{
133     cadastroValido = usu.cadastraUsuario(usu.getNome(),
134                                         usu.getUsuario(),
135                                         usu.getSenha());
136
137     if (cadastroValido == true){
138         //Usuário cadastrado na base de dados
139         mensagemJOption = "Usuário cadastrado corretamente!";
140         mensagemTipo = 1;
141
142     }else{
143         //Algum erro aconteceu
144         mensagemJOption = "Problemas ao inserir o usuário!";
145         mensagemTipo = 0;
146     }
147 }
148
149
150 //Mostrar a mensagem referida
151 JOptionPane.showMessageDialog(null,
152                             mensagemJOption, "Atenção", mensagemTipo );
153 if (mensagemTipo == 1){
154     //Voltamos para a tela de login
155     TelaLogin tLogin = new TelaLogin();
156     tLogin.abreTela();
157
158     //Fecho a tela de cadastro
159     dispose();
160 }
161 }catch (HeadlessException ec) {
162     System.out.println("Erro no cadastro do usuário "
163                     + ec.getMessage());
164 }
165 });
166 }

```

Vamos nos atentar que na linha 126 fazemos a chamada do método **verificaUsuario()**, mas observe que só passamos um parâmetro, o usuário, diferente do que fizemos no Login, onde utilizamos esse mesmo método, mas passando o usuário e a senha, nesse esse caso, estamos fazendo uma **sobrecarga de método**, onde agora só nos interessa verificar o usuário, para que não haja duplicidade em nosso banco de dados, para isso precisaremos na classe **Usuário** acrescentar esse método, portanto, agora teremos dois métodos **verificaUsuario()**, só que as assinaturas são diferentes, veja:

```

51 //Método de verificação da autenticidade do usuario
52 public boolean verificaUsuario(String usuario, String senha){
53     //Fazer a instância da conexão com o banco de dados
54     Conexao banco = new Conexao();
55
56     try{
57         //Abro a conexão com o banco de dados
58         banco.abrirConexao();
59
60         //Criando parâmetro de retorno
61         banco.stmt = banco.con.createStatement();
62
63         //Executando a consulta no banco de dados
64         banco.resultset =
65             banco.stmt.executeQuery("SELECT * FROM usuario "
66                                   + "WHERE usuario = '" + usuario + "'"
67                                   + " AND senha = md5('" + senha + "')");
68
69         //Verificando se existe retorno de dados no banco
70         if (banco.resultset.next()){

```

Este método já existe em nossa classe. NÃO MEXER.

```

71 //Caso tenha
72 resultUsuario = true;
73
74 //Setters em Nome e Usuario
75 setNome (banco.resultset.getString(1));
76 setUsuario (banco.resultset.getString(2));
77
78 //Nos atributos estáticos, realizo as atribuições
79 nomeUsuario = this.getNome();
80 usuarioSistema = this.getUsuario();
81 }else{
82 //Caso não tenha
83 resultUsuario = false;
84 }
85
86 banco.fecharConexao(); // fecha nossa conexão com o banco de dados
87
88 }catch (SQLException ec) {
89 System.out.println("Erro ao consultar usuário " + ec.getMessage());
90 }
91
92 return resultUsuario;
93 }
94
95 //Método de verificação do usuário, estamos aqui fazendo uma
96 //sobrecarga de método
97 public boolean verificaUsuario(String usuario){
98 //Fazer a instância da conexão com o banco de dados
99 Conexao banco = new Conexao();
100
101 try{
102 //Criando parâmetro de retorno
103 banco.stmt= banco.con.createStatement();
104
105 //Executando a consulta no banco de dados
106 banco.resultset =
107 banco.stmt.executeQuery("SELECT * FROM usuario "
108 + "WHERE usuario = '" + usuario + "'");
109
110 //Verificando se existe retorno de dados no banco
111 if (banco.resultset.next()) {
112 //Caso tenha
113 resultUsuario = true;
114 }else{
115 //Caso não tenha
116 resultUsuario = false;
117 }
118
119 banco.fecharConexao(); // fecha nossa conexão com o banco de dados
120
121 }catch (SQLException ec) {
122 System.out.println("Erro ao consultar usuário " + ec.getMessage());
123 }
124
125 return resultUsuario;
126 }
127
128 }
129

```

Este método vamos acrescentar agora.

A diferença além da assinatura, é na *query*, pois não validamos a senha, somente o usuário.

Não havendo duplicidade de usuário na linha 124, conforme programamos há pouco, fazemos a chamada do método **cadastroUsuario()** que está na classe de **Usuário**, e implementamos um atributo para retornar o resultado do cadastro o **resultCadastro**, assim:

```

16 //Atributo que armazenará o retorno do banco de dados
17 private boolean resultUsuario;
18 private boolean resultCadastro;

```

Agora a implementação do método **cadastraUsuario()**:

```

131 //Método para cadastro de usuário em nosso sistema
132 public boolean cadastraUsuario(String nome, String usuario, String senha){
133     //Fazer a instância da conexão com o banco de dados
134     Conexao banco = new Conexao();
135
136     try{
137         //Abertura da conexão com o banco de dados
138         banco.abrirConexao();
139
140         //Criando parâmetro de retorno
141         banco.stmt = banco.con.createStatement();
142
143         //Executando a inserção dos dados
144         banco.stmt.execute("INSERT INTO usuario (nome, usuario, senha)"
145             + " VALUES ('" + nome + "','" + usuario + "','" + md5('"'
146             + senha + "')");
147
148         resultCadastro = true;
149
150     }catch(SQLException ec){
151         System.out.println("Erro ao inserir o usuário " + ec.getMessage());
152         resultCadastro = false;
153     }
154
155     return resultCadastro;
156 }

```

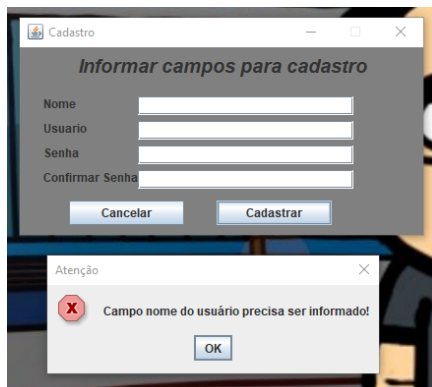
Também precisamos criar o método no botão Cadastrar para abrimos esta tela, para isso, precisamos ir no **TelaLogin.java** e acrescentar:

```

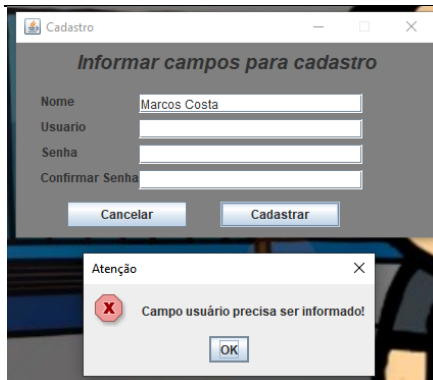
162 //Ação no botão para cadastrar Usuário
163 btnCadastrar.addActionListener((ActionEvent e) -> {
164     //Instancio a classe TelaCadastro
165     TelaCadastro tCadastro = new TelaCadastro();
166     tCadastro.abreTela();
167     dispose();
168 });

```

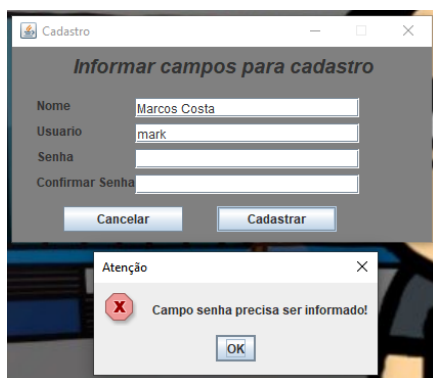
Assim feito, temos a inserção e validações de usuário implementadas em nosso projeto, testando ficaria assim:



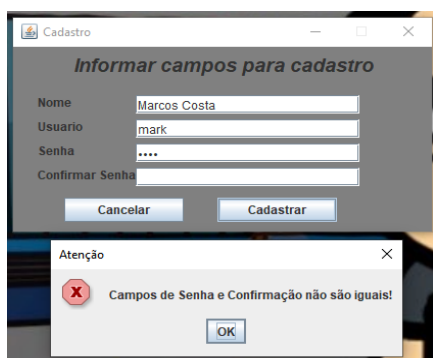
Nome do usuário precisa ser informado



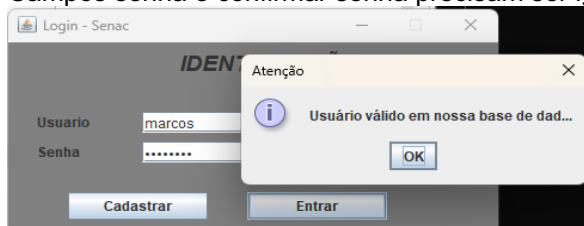
Usuário precisa ser informado



Senha precisa ser informada



Campos senha e confirmar senha precisam ser iguais



Usuário e senha corretos.

Após a inserção o sistema será direcionado para a tela de login.